altairsim — User Manual

2026-09-04 · f570347

A simulation of the MITS Altair 8800 and the S-100 bus

What altairsim is

`altairsim` simulates the **MITS Altair 8800** and the **S-100 bus** it was built around.

It boots real software — not software written to work with it, but the actual artifacts, byte for byte, as they were sold: Altair BASIC off a cassette, CP/M 2.2 off an 8″ floppy, MITS Programming System II off paper tape. None of it has been patched, and none of it knows it is not running on a real machine.

The S-100 bus is a real object in the program, not a wiring diagram implied by the code. Boards plug into it, contend for addresses, pull the interrupt line, and float the data bus when nobody is driving it — getting the answer wrong in exactly the ways real boards did. That makes the machine a **bench**: a board you have not built yet can be fitted here, and the driver you have not finished can be run against it, before either exists in copper. And it makes bugs findable — you can stop the machine mid-instruction, ask which board answered and which stayed silent, and run the same thing again and get the same answer, because nothing here is intermittent and nothing is hidden.

What it does

- **An 8080 and its binary-compatible successor the 8085**, with a Z80 alongside — each faithful down to the flags, the carries and the undocumented behaviours, and each checked against the standard processor exercisers before any board is built on it. The Altair 680b's Motorola 6800 is here too.
- **A board for most of the machine**, all but one modelled from its own manual: CPU boards, RAM/ROM, serial boards, cassette interfaces, floppy and disk controllers, a line-printer controller, video displays, the Sol-PC's integrated I/O, a vectored-interrupt/real-time-clock board, the front panel, and one board of our own for moving files in and out. The boards chapter has the whole list.
- **A monitor** — the prompt you get when the machine is not running — with breakpoints (plain or conditional), single-stepping, disassembly, memory examine and deposit, a bus-cycle trace and a history ring, and a view of the bus itself: who decodes what, who is pulling which interrupt line, and where two boards are fighting.
- **Real I/O.** A serial board can be wired to your terminal, to a TCP socket (so you can telnet into the guest), or to an actual serial port on your machine, with the modem control lines wired through. A line-printer board can be wired to a **real print queue**, so a listing from 1977 comes out of the printer on your desk.
- **File transfer** between the host and CP/M, sandboxed. A board in the machine does the moving; the things you actually *type* — `HDIR`, `R` and `W` — are ordinary CP/M programs that run at the `A>` prompt, like `PIP` or `STAT`.

What it does not do

This section is here because a manual that only lists strengths is an advertisement.

- **The 8085's on-chip serial and interrupt *pins* connect to nothing.** Every documented 8085 instruction runs, and so do the undocumented opcodes — but no card on the S-100 bus carries the `SID`/`SOD` serial lines or the `TRAP`/`RST 5.5`/`6.5`/`7.5` inputs, so code that bit-bangs a console on those pins has nothing on the other end. Ordinary interrupts, over the shared `INTR` line, work exactly as for the 8080.
- **You can save state, but not replay.** `SNAPSHOT` writes the machine's whole state to a file and `RESTORE` reads it back. What you cannot do is *record* a session and step backwards through it — there is no rewind.
- **The Altair itself had no video and no audio.** A terminal on a serial port is its display, as it was — but **video cards existed for the S-100 bus, and several are here** (the VDM-1, the Dazzler, the SD Systems VDB-8024, the Sol-PC). Each opens a real window when the program was built with SDL3, and falls back to a headless display when it was not. **There is no audio output.** Cassette `.WAV` files are read and written as *files*, which the tapes chapter covers.
- **Not every S-100 board is here**, and timing, while honest, is not a circuit simulation: instructions cost the right number of T-states and a cassette takes the right number of them to load, but propagation delays and analogue behaviour are not modelled — and no software from the period could tell.

What is in the box

You have the `altairsim` program, this manual, and a folder of worked examples with their media — complete machines that boot, each with a README of its own. More machines are built into the program itself. It boots something the moment you unzip it; the next chapter says what, and where further disks and tapes come from.

There is no installer and nothing to set up. Unzip it and run it.

What is in the package

```
altairsim                 the program. One file, nothing to install.
altairsim-manual.pdf      this.
altairsim-changelog.pdf   what changed in this release, and the ones before it.
altairsim-cheatsheet.pdf  every command and option, rendered for reading.
altairsim-monitor.pdf     the altairsim> prompt: driving the machine from the console.
altairsim-debugger.pdf    breakpoints, stepping, and looking at the bus itself.
DRIVING-WITH-AI.md        for an AI assistant driving the machine; see below.
cheatsheet.md             the same reference as plain text, for the AI to read.
LICENSE                   the MIT licence this is published under.
LICENSE-SDL3              the licence of SDL3, which is built into the program.
examples/                 machines that boot, media included.
cpm/                      the file-transfer utilities (in cpm/hostbridge/): source, HEX, COM.
```

That is the whole archive. There is no library to install, no runtime, and no configuration file you must write before the program will start.

`altairsim-changelog.pdf` is the release history — what this version does that the last one did not, and the same for the versions before it. It is a separate document from this manual on purpose: the manual describes the program as it is *now*, and a record of what changed reads better on its own than as a chapter that would have to grow one section per release.

`altairsim-monitor.pdf` and `altairsim-debugger.pdf` are two more documents beside this one. They are about driving the program itself — the `altairsim>` prompt where you start and stop the machine, and the debugger you reach for at that prompt when something has gone wrong — rather than the emulated hardware this manual describes. Open them the way you open this one; each says at the top where to start if it is the first one you picked up.

`altairsim` is a single self-contained program. The one outside library it uses — **SDL3**, which opens the window the video boards draw into — is compiled *into* it rather than shipped beside it, so there is nothing to install and nothing that can go missing. `LICENSE-SDL3` is that library's licence, and it is in the package because its code is in the program.

The **Developer Guide** is not in here — it is a separate download from the same release page this came from, and you want it only if you intend to build a board of your own.

`DRIVING-WITH-AI.md`

This one is not for you, exactly. It is a briefing document for an **AI assistant**: drop it in a working directory, start an assistant there, and say "*using altairsim, boot CP/M and show me what is on the disk.*" It tells the assistant how to drive the machine over the program's MCP interface. Ignore it if that is not how you work — nothing else depends on it.

The quick reference travels beside it: the whole command surface — every option, every monitor command, every board and machine — generated from this very program so it matches the binary you have. It ships in two forms of the same content. `altairsim-cheatsheet.pdf` is the one for you — open it the way you open this manual. `cheatsheet.md` is the same thing as plain text, there for the assistant to read.

The machines are in the program

You do not need any files to get a running machine — the machine descriptions are compiled into the binary, and naming one boots it:

```
$ altairsim --list                what the built-in names are
$ altairsim altmon                a monitor in ROM, on a terminal
$ altairsim sol20                 a Processor Technology Sol-20, running SOLOS
```

A built-in is an ordinary machine file that happens to live inside the executable — the same TOML format you would write yourself, and `CONFIG SAVE mine.toml` writes any running machine out as one you can edit. **Several carry their software in ROM and need nothing else** (`altmon`, `amon`, `sol20`, `vdm1`, `rombasic`, the SD Systems `sbc200`/`sbc200v`, among others); the rest carry at most a boot PROM and come up with empty drives, wanting media — which the next section is about. The machines chapter has the full story.

The examples, media included

`examples/` holds complete machines. **Each is a folder with the media in it**, so every one of them comes up the moment you unzip the archive — nothing to fetch, nothing to mount.

Every folder carries its own README, in Markdown and as a PDF beside it, and that README is the description of that example: what the machine is, what is in the drive or the deck, what to type, and what it should print back. So the list of examples is not written down in this manual — look in `examples/`, and read the README of whichever one you want.

```
$ ls examples/
$ altairsim examples/cpm/cpm22-buffered.toml
```

The folder is the unit, and you may move it anywhere. A path written *inside* a machine file resolves against **that file**, not against wherever you were standing when you ran the program — so `examples/cpm/cpm22-buffered.toml` names its disk as plain `cpm22b23-56k.dsk`, the one lying next to it, and the folder still boots after you copy it to your desktop, rename it, or mail it to somebody.

(The other half of that rule matters just as much: a path *you type* at the prompt is relative to **your shell**, because you are the one who can see your own directory. The machines chapter covers both halves.)

The examples chapter walks through some of them at length. Where an example carries period documentation of its own — a game's own printed manual, say — that travels in the folder too, and its README says so.

The file-transfer utilities

`cpm/hostbridge/` holds the programs the file-transfer chapter uses to move files between CP/M and your host — `R`, `W` and `HDIR`. Both the 8080 **source** and the assembled `.HEX` and `.COM` are in there. You do not need them to *use* the utilities on the shipped CP/M disk, which already carries the `.COM`; they ship for the other case that chapter covers — putting the utilities onto a disk that has not got them, where you paste `R.HEX` in through the console.

What is *not* in the package: everything else to run

What is in `examples/` is the whole of the shipped media. The other built-ins that want a disk or a tape — `basic8k`, `ps2`, `minidisk` and the rest — start up perfectly well, with an empty drive:

```
$ altairsim -x "SHOW MOUNTS" basic4k
altairsim> SHOW MOUNTS
  UNIT       KIND  HOLDS
  acr0:tape  tape  (empty)

  Paths are AS WRITTEN.  SHOW PATHS says what they are relative to.
```

You supply the media and `MOUNT` it. The disks and tapes chapters describe how — and where those chapters name an image that is not in `examples/`, they are showing you the shape of the command, not a file you already have.

> **Where the rest will come from.** A separate **`altairsim-packages`** repository is planned to hold the wider collection of disks, tapes and machine files, packaged the same way — each example a self-contained folder you can drop anywhere. **It is not published yet**, and exactly which images go in it has not been settled, so there is nothing to link to here yet.

The bulk of the media is kept out of the program's own archive on purpose: an image is large, most of the good ones are not ours to redistribute, and the simulator's version and the software's have no reason to move together. The ones that ship are the ones that make the manual's first chapters true.

What is *not* in the package: the source

The **source code** is not here — the simulator's, that is. `altairsim` is an open project under the MIT licence, and the source is a separate thing to fetch:

https://github.com/deltecent/altairsim

(The one bit of source that *does* ship is `cpm/hostbridge/` above — but that is 8080 program source for the file-transfer utilities, not the simulator's own code.)

Nothing in this manual requires it. The one exception worth naming: **if you want to build a board of your own** — which is what the simulator is really for — you need the source, and you want the *Developer Guide*, which is a different document. This one is about driving the machine, not extending it.

Reporting a bug, or asking for something

Both go in the same place — the **Issues** tab of that repository:

https://github.com/deltecent/altairsim/issues

Search it first; if nobody has raised your problem, open a new issue. You need a GitHub account, and nothing else.

What makes a bug report useful is enough for somebody else to see what you saw:

- The **version** — the line `altairsim` prints at startup, or `altairsim --version` — and which operating system. Paste it whole: the commit in the parentheses is what says which source built your copy, and between releases the number alone names them all the same. `SHOW VERSION` prints that from inside the monitor, plus a `video` row saying whether this copy can open a window — worth including in anything about the video boards.
- The **machine**: the built-in's name, or the machine file itself, which is a small text file you can paste.
- **What you typed and what happened.** Paste the terminal, prompt and all. The monitor echoes every command, so a pasted session is a complete record of what was asked of it.
- What you expected instead, when that is not obvious.

If the guest software misbehaved rather than the simulator, say which software and where you got it — a period program failing on real hardware in 1976 is a fair thing for it to do here too, and knowing the image is how that gets untangled.

A feature request is an issue as well, and does not need an apology. Say what you are trying to do rather than only which knob you want, because the machine often has a way in already; and if it does not, the shape of the problem is what decides the shape of the answer. A missing S-100 board is a particularly good request when you can name the manual it was documented in — every board here was modelled from its own documentation, and a board with no surviving source is one nobody can build honestly.

Running it

`altairsim` is a single executable. Unzip the package and run it from a terminal.

```
$ ./altairsim
AltairSim 1.0.0 -- 8080, full speed.
machine: default.  HELP for commands.
altairsim>
```

The part in parentheses is **the commit this binary was built from**, and yours will differ. Between releases the version number alone names every build alike, so it is the commit that says which source produced the program in front of you — quote it in a bug report. `SHOW VERSION` prints it on its own, and says whether the tree had uncommitted edits in it at the time.

That prompt is **the monitor**. The machine exists — it has memory, a processor, a console board and a floppy controller in it — but it is not running. Nothing has been started. This is the equivalent of standing in front of the real Altair with the power on and your hands on the switches.

You are not obliged to keep it in the current directory. Put it on your `PATH` and `altairsim` works from anywhere.

macOS: the first run

macOS marks anything that arrives from the internet and refuses to run it until you say otherwise. If you see "*cannot be opened because the developer cannot be verified*", clear the mark:

```
$ xattr -dr com.apple.quarantine ./altairsim
```

Do that once. It is not a comment on the program; it is what macOS does to every unsigned binary that arrives in a zip, whoever wrote it.

The mark is put there by whatever *fetched* the file — a browser, a mail client — so if you pulled the zip down with `curl` or `scp` there may be nothing to clear. The command says nothing and succeeds either way, which is why it is `-dr` and not `-d`: plain `-d` reports an error when the flag is already absent, and that error is not a problem.

Getting help, and getting out

Type	To
`HELP`	list every command
`HELP DUMP`	the usage and worked examples for one command
`QUIT`	leave

There is no `EXIT`. `QUIT` is the word, and `Q` is enough of it.

Commands resolve by **prefix**, so you type as much as it takes to be unambiguous and no more — `HELP` shows each command with its shortest form in brackets: `D[UMP]`, `DE[POSIT]`, `RES[ET]`. Type the part before the bracket. And **commands are not case-sensitive**, nor are the names of the boards in the machine; this manual writes them in capitals only because it is easier to read.

Which machine you get

Running `altairsim` with no arguments gives you a machine called `default` — a 56K Altair with a console and a floppy controller, which is the machine most period software expects.

Naming something gets you something else:

```
$ altairsim examples/cpm/cpm22-buffered.toml     a machine file: this one boots CP/M
$ altairsim basic4k                     a BUILT-IN machine, by name
$ altairsim --list                      what the built-in names are
```

A **built-in** is a machine file that lives inside the program. There is nothing special about it — it is written in the same format as the ones under `examples/`. To see what is actually in one, boot it and look:

```
$ altairsim -x BOARDS basic4k
```

…and if you want it as a file you can edit, `CONFIG SAVE mine.toml` writes out the machine you are actually running, and what it writes will boot.

The full story — every command-line option, and how `altairsim` decides whether a word is a built-in name or a filename — is in the machines chapter.

Quick start: CP/M in one command

```
$ altairsim examples/cpm/cpm22-buffered.toml
```

That is the whole of it.

```
startup> RUN FF00
[console -- ^E returns to the monitor]

56K CP/M 2.2b v2.3
For Altair 8" Floppy

A>
```

You are in CP/M. It is 1977. Type `DIR`:

```
A>DIR
A: L80      COM : LADDER   COM : ED       COM : ASM      COM
A: DUMP     COM : XSUB     COM : PCGET    COM : LS       COM
A: SUBMIT   COM : LOAD     COM : SURVEY   COM : VIEW     COM
A: LADDER   DAT : LUNAR    BAS : M80      COM : MAC      COM
A: MBASIC   COM : PIP      COM : STAT     COM : DDT      COM
A: MOVCPM8  COM : NSWP     COM : SYSGEN   COM : ACOPY    COM
A: OTHELLO  COM : STARTRK  BAS : TICTAK   BAS : WM       COM
A: WM       HLP : CRC      COM : PCPUT    COM : AFORMAT  COM
A: STARINS  BAS : IOBYTE   TXT
A>
```

An assembler, a debugger, Microsoft BASIC, a text editor, and Star Trek. Run one:

```
A>MBASIC
```

What actually happened

Nothing was faked, and it is worth knowing what the one command did, because the rest of the manual is built on it.

The machine file named a **56K Altair with an 8″ floppy controller and a boot PROM at `FF00`**, put the disk image in drive 0, and then did one more thing: it typed `RUN FF00` for you. That is what the `startup>` line is telling you. **There is no `BOOT` command in this program** — booting a disk on an Altair meant setting the address switches to `FF00`, pressing EXAMINE, and then pressing RUN, so that is what the machine file says, in the operator's own words. (EXAMINE is the step that matters: the switches by themselves change nothing, and it is EXAMINE that loads them into the program counter. `RUN FF00` is precisely those two presses — see the *Monitor* document.) Anything you can type, a machine file can do; it gets no special powers.

From there it is all real: the PROM read sector 0 off track 0, that loader pulled CP/M into high memory and jumped into the BIOS, and the BIOS printed its banner.

Getting back out — `^E`

Press **`^E`** (Control-E). This is **ATTN**, and it is how you take the keyboard back from a running program:

```
A>
ATTN -- the machine is still at CA9C. RUN resumes.
C0Z1M0E1I0 A=00 BC=007F DE=CA01 HL=BC0E SP=BC37 IE=1 PC=CA9C  CALL CA78
altairsim>
```

You are back at the monitor, and **the machine is stopped exactly where it stood**. The processor executes nothing while this prompt is up: the `PC=CA9C` above is where it will still be in an hour. That is what makes the prompt useful — you can read memory, single-step, and set a breakpoint, and none of it is a moving target.

Stopped is not **lost**. ATTN is not RESET and it is not POWER: every register, every byte of memory, the disk in the drive and the guest's place in its own program are all exactly as they were. That is the whole content of "*the machine is still at CA9C*" — it is telling you the machine is intact and says where to pick it up.

Why `^E` and not `^C`? Because **`^C` belongs to the guest** — CP/M warm-boots on it and BASIC breaks on it — so the host intercepts `^E` before the guest is ever offered the byte, and no program inside the machine can take it from you. Everything else, `^C` included, goes straight through. (If `^E` collides with something you need, `CONSOLE attn=1D` moves it to `^]`.)

Going back in — `RUN`

```
altairsim> RUN
```

That is all. The machine never stopped, so it simply picks up where it was, and your `A>` prompt is where you left it.

Leaving — `QUIT`

```
altairsim> QUIT
```

There is no `EXIT`. `Q` will do.

The three things to remember

`^E`	out of the guest, back to the monitor. The machine stops where it stands, and loses nothing.
`RUN`	back into the guest.
`QUIT`	done.

Careful: the disk is real, and there is no undo

The disk image is mounted **read/write**, because that is what a machine with a disk in it is — so anything you do in CP/M happens to the file on your host, with nothing keeping a copy. Two ways to be safe:

- **Write-protect it.** `MOUNT dsk0:drive0 examples/cpm/cpm22b23-56k.dsk RO` refuses every write at the controller, so the file cannot change however the guest behaves. Use it to *look around*. But the guest is not *told* the disk is protected, so a program that means to write may not survive being refused — mount `RO` to read, not to run a CP/M you expect to save your work.

- **Copy the folder** when you actually intend to write. It is self-contained and boots from anywhere:

  ```
  $ cp -R examples/cpm my-cpm
  $ altairsim my-cpm/cpm22-buffered.toml
  ```

One non-obvious trap: **this BIOS does not write to the disk when CP/M closes a file** — it buffers a whole track and flushes it the next time it reads the console. So **get back to the `A>` prompt before you quit or copy the image**, or the last write never lands. The disks chapter explains all three in full.

No disk? Start with a tape instead

If you would rather see something boot from nothing at all — no disk, no PROM, the bootstrap toggled in by hand exactly as MITS printed it in the manual — turn to the tapes chapter and load Altair 4K BASIC 3.1. It is the more instructive machine, and it is the one that shows you what an Altair actually was.

Quick reference

Getting out, and back in

Key	Does
`^E`	**ATTN —** stop the machine and take the keyboard back. Nothing is lost.
`RUN`	Resume, at the exact instruction it stopped on.
`QUIT`	Leave. (There is no `EXIT`.)

Editing the command line

`Tab` completes what you are typing — a command, then a board id, then its property names, then a property's values (`SET mem0 fill=` then `Tab`); a second `Tab` lists the choices. `Up` / `Down` walk the command history, saved per directory in `.altairsim_history`.

Key	Does
`Ctrl-A` / `Ctrl-E` (or `Home` / `End`)	start / end of line
`Alt-B` / `Alt-F` (or `Ctrl-Left` / `Ctrl-Right`)	back / forward one word
`Ctrl-W` / `Ctrl-K` / `Ctrl-U`	erase word behind / to end of line / whole line
`Backspace` / `Delete`	erase before / under the cursor

Command line

```
altairsim [options] [machine]

  machine            a built-in name, or a config file (has a '/' or ends .toml).
                     Omitted: ./altairsim.toml if there is one, else `default`.
  -m, --machine <n>  ALWAYS a built-in name -- never a file.
  -f, --file <path>  ALWAYS a file -- never a built-in name.
  -n, --none         empty backplane: no boards, no memory, nothing.
  -l, --list         list the built-in machines and exit.
  -s, --script <f>   run a command script, then exit with its status.
  -x, --exec <cmd>   run one monitor command (repeatable), then exit.
  -i, --interactive  after --script/--exec, stay in the monitor.
      --mcp          MCP server on stdio.
  -v, --version      print the version and exit.
  -h, --help         print this help and exit.
```

Monitor commands

Type the part before the bracket.

<table>
<tr><th>Command</th><th>Does</th><th>Usage</th></tr>
<tr><td>BO[ARDS]</td><td>List, add, or remove boards on the backplane.</td><td>BOARDS [LIST]|ADD <type> <id> [k=v...]|REMOVE <id></td></tr>
<tr><td>B[REAK]</td><td>Set a breakpoint on an address, memory/I/O access, or tape stop.</td><td>BREAK [<addr> | MEM R|W <addr> | IO R|W <port> | TAPE STOP] [IF <expr> | LOADS <expr>] [TRACE ON|OFF]</td></tr>
<tr><td>COM[PARE]</td><td>Compare a range of memory against another address.</td><td>COMPARE <range> <addr></td></tr>
<tr><td>C[ONFIG]</td><td>Load or save the whole machine as a TOML file.</td><td>CONFIG LOAD <f.toml> | CONFIG SAVE <f.toml></td></tr>
<tr><td>CONN[ECT]</td><td>Attach a serial unit to an endpoint (console, socket, file, ...).</td><td>CONNECT <id>:<u> <endpoint></td></tr>
<tr><td>CONS[OLE]</td><td>Show or set the host console's properties.</td><td>CONSOLE [<k>=<v>...]</td></tr>
<tr><td>DE[POSIT]</td><td>Write bytes into memory at an address.</td><td>DEPOSIT <addr> <bytes...></td></tr>
<tr><td>DI[SASM]</td><td>Disassemble memory into instructions.</td><td>DISASM [<addr>|<range>] [n] [CPU=8080]</td></tr>
<tr><td>DISC[ONNECT]</td><td>Unplug the endpoint from a serial unit.</td><td>DISCONNECT <id>:<u></td></tr>
<tr><td>D[UMP]</td><td>Show memory as hex and ASCII.</td><td>DUMP [<addr>|<range>] [WIDTH=16]</td></tr>
<tr><td>E[DIT]</td><td>Enter bytes into memory interactively from an address.</td><td>EDIT <addr> [ROM]</td></tr>
<tr><td>EX[AMINE]</td><td>Point the front panel at an address (and show that byte).</td><td>EXAMINE [<addr>]</td></tr>
<tr><td>F[ILL]</td><td>Fill a range of memory with a byte.</td><td>FILL <range> <byte></td></tr>
<tr><td>HE[LP]</td><td>Show help for a command.</td><td>HELP [<command>]</td></tr>
<tr><td>H[ISTORY]</td><td>Replay the recent instruction (or bus-cycle) history.</td><td>HISTORY [BUS|CPU] [n]</td></tr>
<tr><td>I[N]</td><td>Read a byte from an I/O port.</td><td>IN <port></td></tr>
<tr><td>L[OAD]</td><td>Load a file into memory (binary, Intel hex or S-record).</td><td>LOAD <file> [AT <addr>] [FORMAT=BIN|HEX|SREC] [ROM]</td></tr>
<tr><td>M[OUNT]</td><td>Put a disk or tape image into a drive; a .imd is converted to a raw .dsk beside it.</td><td>MOUNT <id>[:<u>] <file> [WP] [CREATE] [extract[=<base>]] [k=v...]</td></tr>
</table>

`MOV[E]`	Copy a range of memory to another address.	`MOVE <range> <dest> [ROM]`
`N[EXT]`	Step one instruction, running any CALL/RST to completion.	`NEXT`
`NO[BREAK]`	Remove a breakpoint, or all of them.	`NOBREAK [id]`
`O[UT]`	Write a byte to an I/O port.	`OUT <port> <byte>`
`P[OWER]`	Power-cycle the machine -- the only thing that clears RAM.	`POWER`
`Q[UIT]`	Leave the simulator.	`QUIT`
`REGI[ON]`	Add a RAM or ROM region to a memory board.	`REGION ADD <id> type=ram|rom at=<addr> [size=|mount=]`
`RE[GS]`	Show the CPU registers (SET REG changes one).	`REGS | SET REG <r>=<v>`
`RES[ET]`	Reset the machine, keeping RAM (RESET CPU resets just the processor).	`RESET [CPU]`
`REST[ORE]`	Load machine state back from a snapshot.	`RESTORE <file>`
`R[UN]`	Start or resume the machine, optionally at an address.	`RUN [addr]`

`SA[VE]`	Write a range of memory out to a file.	`SAVE <file> <range> [FORMAT=BIN|HEX|OCTAL|PRN]`
`SEA[RCH]`	Find bytes or a string in a range of memory.	`SEARCH <range> <bytes...>|"str"`
`SE[T]`	Change a property of a board, the console, display, a register, or the bus.	`SET <id>[:<u>]|CONSOLE|DISPLAY|REG|BUS <k>=<v>`
`SH[OW]`	Display the state of a board, the bus, or the machine.	`SHOW <id>|BOARDS|BOARD <type> [UNITS]|MACHINES|MACHINE [<name>]|BUS [MAP|IO|IRQ|CONTENTION]|ROMS|MOUNTS|PATHS|CONSOLE|DISPLAY|SYMBOLS|VERSION`
`SN[APSHOT]`	Save the whole machine state to a file.	`SNAPSHOT <file>`
`S[TEP]`	Run one instruction (or n), showing the registers after each.	`STEP [n]`
`SY[MBOLS]`	Load or clear a symbol table for disassembly.	`SYMBOLS LOAD <file> [REPLACE] | SYMBOLS CLEAR`
`T[RACE]`	Log every bus cycle while the machine runs.	`TRACE ON|OFF [file] [MASK=IN,OUT,IRQ,DMA,CONTENTION]`
`TY[PE]`	Feed text to the guest as if typed at its keyboard.	`TYPE "text"`
`U[NMOUNT]`	Take a disk or tape out of a drive.	`UNMOUNT <id>:<u>`

`W[HO]`	Say which board answers an address or I/O port.	`WHO <addr> | WHO IO <port>`

Boards

CPU

Type	What it is
`6800`	Altair 680b CPU board: a Motorola 6800 at 500 KHz. Decodes nothing -- it drives the bus. The 88-CPU's twin, one core down, with memory-mapped I/O
`8080`	MITS 88-CPU: an 8080A at 2 MHz. Decodes nothing -- it drives the bus
`8085`	Generic 8085 CPU board. Decodes nothing -- it drives the bus. The 88-CPU's twin, with an 8085 core (RIM/SIM + TRAP/RST 5.5/6.5/7.5)
`z80`	Generic Z80 CPU board. Decodes nothing -- it drives the bus. The 88-CPU's twin, with a Z80 core

Memory

Type	What it is
`bankmem`	S-100 bank-switched RAM. One card, four decoders (card=vector|cromemco64kz|northstar|expandoram2): a write-only select port swaps which RAM plane(s) drive the bus. Each card owns its own decode -- one-hot select (Vector 40), 8-bit bank mask (Cromemco 40), on/off+one-hot toggle (North Star C0), or PROM page-select (ExpandoRAM II FF, approximated)
`memory`	RAM/ROM board: a list of regions and PHANTOM* -- plain, unbanked memory (bank switching is its own board, `bankmem`)
`v2z80rom`	S100Computers V2 Z80 CPU board -- its onboard paged monitor EEPROM (the Z80 itself is board 'z80cpu'). An 8K 28C64 at F000-FFFF holding two 4K pages, builtin:master0 (low) / master1 (high), selected by OUT D3H bit1 (bit0=1 inactivates the EEPROM so RAM shows through). Shadows RAM in its window while enabled. Cold-start the MASTER monitor with startup=["RUN F000"]; the 'I' command boots CP/M 3 off a dualsd card

Disk

Type	What it is
`16fdc`	Cromemco 16FDC: WD FD1793 soft-sector floppy (single + double density), up to 4 drives. Disk registers at 30-34, a TMS 5501 console UART at 00-09 (unit 'tty'), and a 4K RDOS 2.52 boot PROM at C000 (OUT 40H banks it out, RESET restores it). Boots CDOS
`64fdc`	Cromemco 64FDC: the 16FDC's 1983 successor -- same FD1793 + TMS 5501, carrying an 8K RDOS 3.12 boot PROM at C000-DFFF (OUT 40H banks it out, RESET restores it). Boots CDOS
`dcdd`	MITS 88-DCDD: 8" hard-sector floppy, up to 16 drives. Three ports at BASE+0..2. INVERTED status bits
`dualide`	S100Computers IDE-AB (CF): the IDE/CompactFlash half of the IDE+ESP32 combination board -- two CF sockets (drives 0/1 = A:/B:) for CP/M 3. Five 8255 ports at BASE+0..4 (default 30): A/B data, C control lines, mode config, drive select. Programmed-I/O ATA register engine (LBA read/write, 512-byte sectors). No boot PROM -- the CPU board's monitor boots CP/M from the CF. Mounts the SAME card image as dualsd (a .img with a .geo geometry sidecar); pair with dualsd for the full A:/B:+C:/D: system
`dualsd`	S100Computers Dual SD: two microSD sockets (drives 0/1) presented as raw 512-byte-sector CF/SD cards, for CP/M 3. Two ports at BASE+0..1 (default 80): status/command + data. Programmed-I/O command/handshake engine (33H-lead + 8 commands). No boot PROM -- the CPU board's monitor loads CP/M from track 0. Mount a card image (a .img with a .geo geometry sidecar)
`hdsk`	MITS 88-HDSK Datakeeper: Pertec hard disk, 256-byte sectors from a linear .DSK. Eight ports at BASE+0..7 (default A0). Command/handshake protocol, four page buffers
`icom`	iCOM FD3712/FD3812 8" floppy: a programmed-I/O command/handshake controller on the S-100 Interface board. Two ports at BASE+0..1 (default C0) plus a boot PROM and 6810 scratch RAM in high memory (rom=builtin:icom-fd3712-cpm | icom-fd3712-fdos | icom-fd3812-cpm). Boots CP/M 2.2 (single and double density) and FDOS. Up to 4 drives
`mds`	MITS 88-MDS: 5.25" minidisk, 4 drives. Same three ports as the dcdd -- but 300 RPM, 64 us/byte, and a motor that stops after 6.4 s
`tarbell`	Tarbell #1011: single-density WD FD1771 floppy, up to 4 drives. Eight ports at BASE+0..7 (default F8). Carries a 32-byte boot PROM that shadows 0000 over PHANTOM* -- boots CP/M automatically at reset (bootstrap=on)
`tarbelldd`	Tarbell #2022: double-density WD FD1791 floppy (mixed-density media, SD track 0), up to 4 drives. The single-density card's twin with a bitmap OUT-FC latch and a port-FD DMA/ext-addr register. Same 32-byte boot PROM
`versafloppy`	SD Systems VersaFloppy I/II: WD FD177x soft-sector floppy, up to 4 drives. Eight ports at BASE+0..7 (default 60). variant=vfi (FD1771, single density) | vfii (FD1791, single+double). Boots SDOS with the SBC-200 + DDBIOS

Serial

Type	What it is
`2sio`	MITS 88-2SIO: two 6850 ACIAs, units 'a' and 'b'. Four ports at BASE+0..3
`680io`	Altair 680b onboard I/O: a 6850 ACIA console ('tty') at F000/F001 and the config-strap read port at F002. Memory-mapped
`gsio`	Generic SIO: a strap-configurable serial board with TWO independent channels, units 'a' and 'b' (configure each under [board.unit.a] / [board.unit.b]). Basic transmit/receive only -- no programmable word length, parity, stop bits or framing/overrun status; a specific card that needs those is a separate emulated board. Per channel you strap: a status/control port (status_port -- read synthesizes DAV/TBMT at bit positions you pick, write is discarded) and a data port (data_port); one inverter_gate knob inverts both status bits together. Built-in profiles preset the straps: profile=sior0 (MITS SIO Rev 0, the default) | tuart (Cromemco TU-ART) | imsai-sio2 | compupro-if2 (CompuPro Interfacer II) | compupro-ss1 (CompuPro System Support 1). Channel a defaults to ports 0/1, b to 2/3. Polled, no interrupts. CONNECT each channel to a file, socket, serial port, in:/out:
`io4`	SSM IO-4 (2P+2S): the real Solid State Music board -- two full-duplex serial channels AND a four-port parallel section. Serial units 'a'/'b' are real 1602-family UARTs with programmable word length (data_bits 5-8), parity and stop bits (unlike the generic gsio), plus the full status-word strap-up (stat_* map, invert_status, port_reversal) and named profiles (default altair-rev1). A 4-port block set by switch S3 (default 0-3): Serial A status/data at BASE+0/+1, Serial B at BASE+2/+3. Parallel units 'pa'/'pb' are 8212 latched ports (input latch + service request, output latch) on their own 2-port block set by switch S4 (par_port, default 4-5): Parallel A at PAR+0, B at PAR+1; a byte the far end sends is strobed in, a write latches out, and the §3.2.2 status/data console flag is strappable (dav_bit/dav_source/dav_active_low). If the two blocks overlap, neither section answers the shared ports. Each serial channel's receive (DAV) and transmit (TBMT) and each parallel input (service request) can raise an interrupt, strapped on header W4 to a VI line, pin 73 (int), or none (rx_int/tx_int per serial channel, int per parallel port) -- there is no software enable, the strap is the enable, and a parallel input interrupt rises even when the port is not addressed. Configure each unit under [board.unit.a] / [board.unit.pa] etc.; CONNECT each to a file, socket, serial port, in:/out:
`pmmi`	PMMI MM-103: Bell 103 modem on an S-100 card, unit 'line'. Four ports at BASE+0..3 (default C0), read/write different registers. Transmit/receive over a ByteStream; CONNECT it to in:/out: files. No dialer; modem status is a fixed stub
`propio`	S100Computers Console IO Board (Parallax-Propeller console), unit 'serial'. A strap-configurable serial subtype preset to the board's documented convention: status/data at 00/01, RX-ready = status bit 1, TX-ready = status bit 2, both active high. Every strap (status_port/data_port/dav/tbmt/inverter_gate) is still overridable -- the real board is jumpered. Polled, no interrupts. CONNECT it to a file, socket, serial port, in:/out:
`sbc`	SD Systems SBC-100/200: Z80 single-board computer. One 8-port block (78-7F): Intel 8251 console (unit 'tty', data 7C / status 7D, RxD->/DSR auto-baud for MSMONR21), Z80-CTC (78-7B) whose ch1 raises a mode-2 keyboard interrupt (vector 0x82) off the 8251 RxRDY, and a parallel port (7E/7F) whose OUT 7F bit 1 switches the onboard PROM out. Optional onboard boot PROM via [[board.socket]] (at+mount). variant=sbc100|sbc200
`sio`	MITS 88-SIO: one COM2502 UART, unit 'tty'. Two ports at BASE+0..1. INVERTED status bits

`turnkey`	MITS 8800b Turnkey Module: phantom boot PROM (FC00-FFFF), integrated 6850 SIO (unit 'tty', default 0x10), sense switches at FF, and the Auto-Start JMP jam. Sockets via [[board.socket]]

Tape

Type	What it is
`680kcacr`	Altair 680b KCACR audio-cassette interface: a 1602-family UART recording Kansas City Standard FSK, memory-mapped at F010 (status/control) and F011 (data), active-LOW. Adds software motor control (control D7=on, D6=off) and interrupt-driven transfer (D0/D1 enables pull the 6800 IRQ). Reuses the 88-ACR tape machinery -- MOUNT a tape, WIND/REWIND it
`acr`	MITS 88-ACR: cassette. An 88-SIO B + an FSK modem, unit 'tape'. Brings the WIND/REWIND/EXTRACT verbs and a tape counter
`uio`	MITS 88-UIO: serial + cassette on one board. A 6850 (unit 'serial', default 0x10) and an 88-ACR cassette section (unit 'tape', default 0x06) with motor control and a SW-1 MITS/Kansas-City modulation switch. Defaults reproduce the standard 0x10 + 0x06 layout

Parallel and printer

Type	What it is
`4pio`	MITS 88-4PIO: up to four 6820 PIAs, sections ja/jb.. per port. 16 ports from BASE (default 20). Software-set direction; CONNECT each section
`680uio`	Altair 680b Universal I/O: a second 6850 ACIA serial port ('serial') and a 6820 PIA parallel port (sections 'p1a/p1b', 'p2a/p2b' with pias=2) in an S9-relocatable window (default base F000: serial F006/F007, PIA F008-F00F), plus fixed switch inputs at F003 and a non-latched output at F010-F013. Memory-mapped, active-high
`c700`	MITS 88-C700: Centronics line-printer controller, unit 'prn'. Two ports at BASE+0..1 (default 02). Output-only; CONNECT it to a file, a socket, or a real printer queue
`d7a`	Cromemco D+7A: analog + parallel I/O. Eight ports from BASE (default 18): one parallel port + seven two's-complement A/D-in/D/A-out channels. Reads 1-2 JS-1 joysticks from the host
`lpc`	MITS 88-LPC: 88-LP line-printer controller, unit 'prn'. Two ports at BASE+0..1 (default 02). Line-buffered: 6-bit codes + PRINT/LINE FEED/CLEAR. CONNECT it to a file, a socket, or a real printer queue
`pio`	MITS 88-PIO: 8-bit parallel port, units 'out'/'in'. Two ports at BASE+0..1 (default 04). CONNECT a printer, a keyboard, a socket

Video

Type	What it is
`dazzler`	Cromemco Dazzler: color graphics from a framebuffer in main RAM. Two ports at BASE+0..1 (default 0E): control/status and format. 32x32 to 128x128, 16 colors/greys. Needs a Display
`vdb8024`	SD Systems VDB-8024: an 80x24 video terminal on one board -- the video console for an SBC-100/200 (the alternative to the 8251). Two I/O ports at BASE+0..1 (default 00): status/keyboard/display. Unit 'keyboard' (CONNECT). Optional keyboard-strobe interrupt strap (interrupt=vi0..vi7) for the SBC-200's CTC to vector -- what the SD video CBIOS needs; polled by default. Boots sdmonv21. Needs a Display
`vdm1`	Processor Technology VDM-1: memory-mapped 16x64 video, screen RAM at BASE (default CC00), scroll/status port (default CC). Needs a Display

Systems

Type	What it is
`sol`	Processor Technology Sol-PC I/O: serial, keyboard, parallel, CUTS tape as one board. Seven ports F8..FE. Units serial/printer/keyboard (CONNECT) and tape1/tape2 (MOUNT). Brings the WIND/REWIND/EXTRACT verbs and a tape counter

PROM programmer

Type	What it is
`pb1`	SSM PB1: 2708/2716 EPROM programmer + on-board EPROM board. A 4K programming-socket window (default D000, sockets U22=2708/U23=2716) and one control port (default 10): OUT arms the board and picks the chip (D0=2708, D1=2716), then a window write burns a byte and a window read disarms it. Save the burn to a host hex file with `SAVE file window`. Optional read-only on-board EPROM area via [[board.prom]] (at + mount). The board ships no firmware -- run any 2708/2716 burner; SSM's own from the PB1 manual is in examples/pb1

Other

Type	What it is
`fp`	Altair front panel: the SENSE switches a guest reads at IN 0FFH -- a configured byte (SET fp0 sense= or TOML), not toggled here. No OUT
`hostbridge`	Host Bridge: guest <-> host file transfer, sandboxed. OUR OWN BOARD, not a period one. Two ports at BASE+0..1. R.COM/W.COM/HDIR.COM
`ss1`	CompuPro System Support 1: multifunction S-100 board. Dual 8259A interrupt controllers in a master/slave cascade (master/slave at base+0..+3; master watches VI0-6 and drives pin 73, slave takes the timer OUTs and the UART's Rx/TxRDY), an 8253 interval timer (three counters + control at base+4..+7, 2 MHz clock), the OKI MSM5832 battery-backed real-time clock/calendar (command/data at base+10/+11) and a 2651 UART serial channel (base+12..+15); base default 50H. The 9511/9512 math socket is unpopulated
`virtc`	MITS 88-VI/RTC: vectored interrupts (VI0-VI7 -> RST n) and a real-time clock. One port at FE

Machines

Machine	What it is
`8085`	A minimal 8085 machine: an `8085` CPU, 64K of RAM, and a 2SIO console.
`acuter`	ACUTER at F000 -- CUTER on a plain Altair, with a terminal instead of a VDM.
`altair680`	The Altair 680b -- MITS's second machine, and a different animal from the 8800.
`altmon`	An Altair with a monitor in ROM and a terminal on it.
`amon`	AMON 3.1 in a 4K EPROM at F000 -- Martin Eberhard's full-featured Altair monitor.
`bankmem`	A bank-switched RAM machine: a Z80, a console, and a Vector Graphic 64K bankmem.
`basic4k`	The machine Altair 4K BASIC was sold to run on: an 88-SIO Teletype, a cassette in the ACR.
`basic8k`	The machine Altair 8K BASIC was sold to run on: an 88-2SIO terminal, a cassette in the ACR.
`cdbl`	The `default` machine with the Combo Disk Boot Loader in the PROM socket.
`compupro`	A stock Altair with a CompuPro System Support 1 board for its clock/calendar.
`cuter`	CUTER 1.3 driving a Processor Technology VDM-1 -- the real Sol/CUTS monitor.
`dazzler`	A Cromemco Dazzler in an Altair -- the bench for the S-100's first color graphics card.
`default`	The machine you get when you name none: 56K, and the DBL boot PROM at FF00.
`dualide`	S100Computers "IDE-AB CF" machine -- a V2 Z80 CPU board booting CP/M 3 off a CompactFlash card.
`dualidesd`	S100Computers "IDE-AB CF+ESP32" machine -- both boards, CP/M 3 on CF drives A:/B: and SD drives C:/D:.
`dualsd`	S100Computers "Dual SD" machine -- a V2 Z80 CPU board booting CP/M 3 off a microSD card.
`icom`	iCOM FD3712 8" floppy machine -- boots CP/M 2.2 off a single-density iCOM disk.
`lineprinter-lpc`	The `default` machine with an 88-LPC line printer at port 02; CONNECT `lpt0:prn` to a file or the console.
`lineprinter`	The `default` machine with an 88-C700 line printer at port 02; CONNECT `lpt0:prn` to a file or the console.
`minidisk`	The Altair Minidisk: an 88-MDS at 08 and the MDBL boot PROM. You supply the 5.25" disk.
`original`	The Altair as it actually left Albuquerque.
`parallel`	The `default` machine with two MITS parallel boards: an 88-PIO and an 88-4PIO.
`ps2`	The machine MITS Programming System II ran on: basic8k's cards, but not basic8k's bootstrap.
`ps2int`	MITS Programming System II, WITH INTERRUPTS. `ps2` with A9 down and an 88-VI/RTC in it.
`rombasic`	MITS Extended ROM BASIC 16K -- Extended BASIC that runs directly out of ROM.

`sbc200`	SD Systems SBC-200 -- a 4 MHz Z80 single-board computer running the MSMONR21 monitor.
`sbc200v`	The SD Systems SBC-200 with a VDB-8024 VIDEO console: SDMONV21 on an 80x24 screen.
`sol20`	The Processor Technology Sol-20 -- an integrated 8080 machine, running SOLOS.
`tarbell`	Tarbell #1011 single-density floppy machine -- boots CP/M 2.2 the moment a disk is in it.
`tarbelldd`	Tarbell #2022 double-density floppy machine -- boots CP/M 2.2 the moment a disk is in it.
`turnkey`	The MITS 8800bt -- an Altair with a Turnkey Module where the front panel used to be.
`vdm1`	A Processor Technology VDM-1 in an Altair, and a demo that draws on it.
`z80`	A minimal Z80 machine: a `z80` CPU, 64K of RAM, and a 2SIO console.

A machine file, in one look

```
[machine]
name    = "mine"
base    = "default"         # start from a machine, and say what is DIFFERENT
startup = ["RUN FF00"]      # the operator's own keystrokes. There is no BOOT verb.

[[board]]                   # type + a NEW id      -> ADD the card
type = "2sio"               # type + an id from the base -> REPLACE it outright
id   = "sio0"               # NO type + an id      -> MODIFY the one already there
port = 10                   # remove = true        -> PULL THE CARD OUT

  [board.unit.a]            # a unit's own settings
  connect = "console"

  [[board.region]]          # a list the card owns (memory)
  type = "ram"
  at   = 0000               # hex: it is an address
  size = "56K"              # decimal: it is a size

  [[board.drive]]           # a list the card owns (disk controllers)
  unit  = 0
  mount = "cpm.dsk"         # relative to THIS FILE

[console]                   # the HOST's terminal -- not a board
strip7out = true
base      = octal           # read/print the wire class in split octal (MITS style)
```

Paths: a path *inside* a machine file is relative to **that file**. A path you *type* is relative to **your shell**.

Endpoints — `CONNECT <id>:<unit> <endpoint>`

Endpoint	Is
`console`	the host terminal. Exactly one unit may hold it.
`null`	nowhere. Writes vanish, reads never come.
`loopback`	itself — what you write comes back.
`scripted`	a caller in place of a human — what MCP and the tests type into.
`socket:PORT`	**listens** — this is telnet-in.
`socket:HOST:PORT`	**calls out**.
`serial:DEVICE`	a real serial port on this host.
`in:PATH`	a host file as a reader (paper tape). `?cps=N` paces it.
`out:PATH`	a host file as a punch — 8-bit clean, never truncating.
`terminal`	a window the simulator draws itself (SDL builds). `?emulation=vt100|adm3a|vt52|h19`, `?size=COLSxROWS`.
`printer:QUEUE`	a real print queue on this host.
`<endpoint>|FILE`	a tap: append `|FILE` to any endpoint to also log the line.
`<endpoint>|socket:PORT`	a live mirror: `telnet` in to watch and take over. `?ro` = watch-only.

Machines

A **machine** is a backplane with boards in it. Which boards, with what settings, in what state — that is all a machine is, and it is the only thing `altairsim` needs to be told.

You tell it in one of three ways: name a **built-in**, name a **file**, or say **nothing** and take the default. This chapter is about how that choice is made, and about the one rule that decides what a path means.

The command line

```
altairsim [options] [machine]
```

Option	
`machine`	a built-in name, **or** a config file if it contains a `/` or ends in `.toml`
`-m, --machine <name>`	**always** a built-in name — never a file
`-f, --file <path>`	**always** a file — never a built-in name
`-n, --none`	an empty backplane. No boards, no memory, nothing
`-l, --list`	list the built-in machines and exit
`-s, --script <file>`	run a command script, then exit with its status
`-x, --exec <cmd>`	run one monitor command, then exit. Repeatable
`-i, --interactive`	after `--script` / `--exec`, stay in the monitor
`--mcp`	run as an MCP server on stdio
`-v, --version`	print the version and exit
`-h, --help`	print this help and exit

Give exactly one machine. A positional name *and* a `-m`, or a `-f` *and* a `-n`, is an error — the program says *give ONE machine* and stops. It does not guess which one you meant.

How a bare word resolves — and why it never looks at your disk

```
$ altairsim basic4k                     a BUILT-IN, by name
$ altairsim examples/cpm/cpm22-buffered.toml            a FILE, by path
```

The rule is **purely syntactic**. The filesystem is **never probed**:

The word	What it is
contains `/` or `\`	a **file**
ends in `.toml`	a **file**
anything else	a **built-in name**

That is deliberate, and it is worth being clear about why, because a simulator that guessed would be more convenient exactly until the day it was not.

`altairsim basic4k` means `basic4k` in every directory on earth. It means the same thing on your machine and on mine. It cannot be hijacked by a file called `basic4k` that happens to be lying next to you — because the program never asks whether such a file exists. A command in a script, a line in a README, a habit in your fingers: all of them keep meaning what they meant.

The cost is that `altairsim mymachine.toml` needs the extension, and `altairsim ./mymachine` needs the `./`. That is a small price, and if you want the question settled explicitly, settle it:

```
$ altairsim -m basic4k                  a built-in. Stop looking for a file
$ altairsim -f ./basic4k                a file called `basic4k`, no extension, right here
```

`-m` and `-f` are how you say what you mean when the syntax will not say it for you.

The one file the simulator *finds*

If you name **nothing at all**, and the working directory contains a file called `altairsim.toml`, that machine is loaded — and it says so:

```
$ altairsim
altairsim: no machine named -- using ./altairsim.toml (`-m default` for the built-in).
AltairSim 1.0.0 -- 8080, full speed.
machine: bench.  HELP for commands.
altairsim>
```

The first line is the announcement, and it is printed before anything else so you cannot miss it. The `machine:` line after it names the machine *the file* declares — `bench` here, not the file it came out of — so the two lines together say both halves: where it came from, and what it turned out to be.

This is the **only** file the simulator finds rather than is given, and it only happens when the command line names nothing whatsoever. Name a built-in, a file, or `-n`, and `./altairsim.toml` is ignored — you asked for something, so you get it.

Put one in a project directory and `altairsim`, bare, is your machine. **It announces itself when it does**, so you are never running something you did not know about.

With no `altairsim.toml` and no arguments, you get the built-in `default`.

The built-in machines

A **built-in is a TOML machine file compiled into the binary.** There is nothing privileged about it: same format, same keys, same rules as one you write yourself. It is in the program only so that it is always there.

```
$ altairsim --list
```

names them, one to a line, each with a sentence saying what it is — and it is the live list, so it cannot be short of a machine the way a list typed into a chapter can. The machine reference at the back of this manual is that same table. From the `altairsim>` prompt the list is `SHOW MACHINES`.

To see what is in one — its backplane and its startup — name it:

```
$ altairsim -x 'SHOW MACHINE' basic4k
```

or, at the prompt, `SHOW MACHINE basic4k`. A bare `SHOW MACHINE` there is the machine you are running now.

And to get it as **text you can edit** — the actual machine file, every board, every setting:

```
$ altairsim -x 'CONFIG SAVE mine.toml' basic4k
$ altairsim mine.toml
```

`CONFIG SAVE` writes the machine you are actually running, and it round-trips. **Which makes every built-in a worked example.** Find the one closest to what you want, save it out, and edit it.

Or better, do not copy it at all — start *from* it with `base`, and write down only what is different. The configuring chapter is about that.

The empty backplane — `-n`

```
$ altairsim -n
```

No boards. No memory. No processor. `-n` is a bare chassis, and every `BOARDS ADD` from there is yours. It is the honest starting point when you are building a machine up board by board, and it is the one way to be certain nothing is in there that you did not put there.

The path rule, and it has two halves

This is the rule that lets an example directory be copied anywhere and still boot. It is one principle stated twice:

> **A path written inside a machine file is relative to that file.**
>
> **A path you type at the prompt is relative to your shell.**

Both are true at once, and they do not conflict, because they are the same idea: **a path means what its author could see.**

The author of a machine file could see the directory the machine file is in. When `examples/cpm/cpm22-buffered.toml` says `mount = "cpm22b23-56k.dsk"`, it means *the disk lying next to me* — and it goes on meaning that after you copy the folder to your desktop, rename it, or mail it to someone. That is why the examples are self-contained directories, and why the quick start's `cp -R` actually works.

You, at the prompt, can see your own working directory. When you type

```
altairsim> MOUNT dsk0:drive1 scratch.dsk
```

you mean *the file next to me*, because that is the only thing `scratch.dsk` could sensibly mean when you are the one typing it. The simulator does not go rummaging in the machine file's directory for a file you named with your own hands.

The consequence is worth stating, because it catches people: **a path in a `startup` command is inside the machine file, so it is relative to the machine file** — even though `startup` commands look exactly like things you type. They were written by the file's author, so they mean what the file's author could see.

When it bites, and what it looks like

The rule is invisible until a file is missing, and then it can look like a typo that is not one. Keep your machine files in a `machines/` folder, write a path meaning *the folder you launched from*, and you get the one confusing case:

```
[[board.drive]]
unit  = 0
mount = "disks/Kermit/cpm.dsk"      # meant: the disks/ I can see from my shell
```

`altairsim -f ./machines/8800c.toml` then says:

```
./machines/8800c.toml: dsk0: 'machines/disks/Kermit/cpm.dsk': no such file
  ('disks/Kermit/cpm.dsk' is relative to the machine file that wrote it, in ./machines/)
```

The disk is not missing. It was looked for beside the machine file, because that is where the rule says a machine file's paths point. Write it the way the machine file sees it:

```
mount = "../disks/Kermit/cpm.dsk"   # up out of machines/, then down into disks/
```

…or keep the machine file next to what it mounts, which is what every shipped example does.

Note what is *not* affected: once the machine is up, `MOUNT dsk0:drive1 disks/Kermit/cpm.dsk` typed at the prompt needs no `../`, because you are the one typing it. The `../` belongs to the file, not to you.

Ask the machine, rather than working it out

You do not have to hold this in your head. `SHOW PATHS` prints every base at once, for the machine you are actually running:

```
altairsim> SHOW PATHS
  what you type      /home/you/altair
                     MOUNT, LOAD, SAVE and -s scripts resolve against this.

  machine file       /home/you/altair/disks/cpm22
                     `mount`, `base` and the MOUNT/LOAD lines in `startup`
                     resolve against THIS, not the cwd -- so a machine file
                     names the disks lying beside it and goes on naming them
                     from wherever you launch it.

  hb0 sandbox        /home/you/altair/disks/cpm22/xfer
                     THE GUEST'S SANDBOX, and the only real fence here:
                     R.COM/W.COM cannot leave it. It is not a base for
                     anything you type. Set with `hostdir`.
```

Three lines, three different directories, and they are allowed to differ — that is the rule working, not a fault.

Boot a **built-in** machine and the middle line says so, because then there is no file and nothing was re-based:

```
  machine file       (none -- this machine is built in to the binary)
```

`SHOW MOUNTS` is the companion: every disk, tape and ROM in the machine and what is in each, across all the boards at once.

```
altairsim> SHOW MOUNTS
  UNIT         KIND  HOLDS
  dsk0:drive0  disk  cpm22b23-56k.dsk
  dsk0:drive1  disk  (empty)
  dsk0:drive2  disk  (empty)
  dsk0:drive3  disk  (empty)
  mem0:rom0    rom   builtin:dbl  (read-only)

  Paths are AS WRITTEN.  SHOW PATHS says what they are relative to.
```

Empty drives are listed, not hidden. The 88-DCDD has four, one disk is in it, and the other three doors are open — which is the machine, and worth seeing.

That last line is the command telling you what the middle column is worth, and it is why the two belong together: `SHOW MOUNTS` tells you what the machine was told, and `SHOW PATHS` tells you what that meant.

None of this is a sandbox

The path rule decides **where a path points**, and confines nothing. A machine file may mount any file on your disk — with `..`, or with an absolute path — and it will be opened.

The one real fence is the Host Bridge's **`hostdir`**, which limits how far a CP/M program running *inside* the machine can reach when it reads and writes host files. That is a different mechanism for a different purpose — see *Moving files in and out* — and nothing you write in a machine file moves it.

Running a command and leaving — `-x` and `-s`

`altairsim` does not have to be interactive.

```
$ altairsim -x 'SHOW MACHINE' default
$ altairsim -x 'DUMP 0 F' examples/cpm/cpm22-buffered.toml
```

`-x` runs one monitor command against the machine and exits. It is **repeatable**, and the commands run in the order you gave them:

```
$ altairsim -x 'MOUNT dsk0:drive0 mine.dsk' -x 'RUN FF00' -i examples/cpm/cpm22-buffered.toml
```

`-i` is the difference between a query and a start-up: without it the program exits when the commands are done; with it you are dropped into the monitor with the machine exactly as your commands left it. `-i` alone, with no `-x` or `-s`, does nothing.

`-s` runs a **script** — a file of monitor commands, one per line, the same ones you type:

```
$ altairsim -s boot.cmd examples/cpm/cpm22-buffered.toml
```

The exit status is non-zero if any command failed. That is the whole point: `altairsim -s` is a program you can put in a shell script, a Makefile, or a build, and test the result of.

```
if altairsim -s check.cmd mine.toml; then
    echo "machine is sane"
fi
```

Which chapter next

The **configuring** chapter is the machine file itself: every table, every key, and the four things a `[[board]]` entry can mean. The **boards** chapter is what the boards *are*.

The machine file

A machine file is **TOML**. It lists the boards in the backplane, what is set on each of them, and what to do once the power is on. That is all it does, and there is nothing else it can do.

This chapter is the definitive description of the format.

What TOML is

The machine file is written in **TOML** — a plain configuration format meant to be written by hand and read back a year later without a manual. You need only a handful of rules to read every example below:

- **`key = value`** — one setting per line: `clock_hz = 2000000`.
- **Quotes are for text.** A string is quoted (`name = "cpm22"`); a number or a true/false is bare (`size = 256`, `idle = true`).
- **`[table]` appears once** — `[machine]` is the machine, `[console]` is your terminal.
- **`[[table]]` — doubled brackets — repeats.** Each `[[board]]` starts another board.
- **A nested `[board.unit.x]` belongs to the block above it**, by name — the indentation in these examples is only for the eye.
- **`#` starts a comment** to the end of the line. A comment that begins **`#>`** is a *note* — the file prints it to you when it loads (see below).

That is the whole of the syntax. Everything below is which keys go where.

The one thing to know first

Anything you can type, a config can do — and nothing more. A machine file has no special powers. It cannot boot a disk, because there is no `BOOT` verb; what it can do is *type `RUN FF00` for you*, which is what the operator did. Every board setting in a machine file is a setting you could have made with `SET` at the prompt, and every `SET` you make at the prompt is a key you could have written in the file. **A board's properties *are* its TOML keys.**

There is no separate config schema anywhere in the program. That is why the board reference at the back of this manual is exhaustive, and why it cannot drift out of date: it is printed from the same table the monitor resolves against.

Nothing is silently ignored

Any unknown table or key is a hard error, with a sentence saying so. The machine does not load.

```
mine.toml: unknown [machine] key 'widget'
mine.toml: [[board]] cpu0: cpu0 has no property 'frobnicate'. Known: clock_hz idle achieved_hz
```

This is not fussiness. **A configuration that looks like it set something and did not is worse than one that will not load**, because you will spend the afternoon debugging the machine instead of the typo. A misconfiguration in this program cannot be silent.

Notes the operator sees — `#>`

An ordinary `#` comment is for whoever *reads the file*. A comment that begins `#>` is for whoever *loads it*: its text is printed to you when the machine comes up, right after the `loaded …` line and before the `startup` commands run. It is where the file's author leaves the one or two sentences you need to actually use the machine.

```
#> Boots CP/M 2.2 from drive A.
#> Type DIR at the A> prompt, and DIR B: for the blank second disk.

[machine]
name = "cpm22"
base = "default"
startup = ["RUN FF00"]
```

- **One `#>` line is one printed line.** Write as many as you like, in a row or scattered through the file — they print in the order they appear.
- **`#>` on its own prints a blank line**, so you can space a note into a short paragraph.
- **A `#>` can trail a setting**, too: `name = "cpm22"  #> the buffered variant`.
- It is still a comment. It **sets nothing**, and `CONFIG SAVE` does not write it back — a saved machine is the backplane, not the prose around it. If a note is worth keeping, keep it in the file you wrote by hand.

An ordinary `#` comment, as always, is seen by no one but the reader of the file.

The tables

Table	What it is
`[machine]`	the machine's identity. Three keys, no more
`[[board]]`	a board. One entry per board
`[board.unit.<name>]`	one unit *on* the board above — a serial channel, a tape deck
`[[board.region]]`	a memory region on a `memory` board
`[[board.drive]]`	a drive on a disk controller
`[console]`	**your terminal.** Not a board — see below
`[display]`	**your video window.** Not a board either — see below

`[machine]` — and it has exactly three keys

```
[machine]
name    = "cpm22"
base    = "default"
startup = ["RUN FF00"]
```

`name`

What the machine is called. That is the whole of it.

`base` — start from a machine, and say what is *different*

```
base = "default"          # a built-in
base = "../cpm22/cpm22.toml"   # or a file
```

The value resolves by **the same syntactic rule as the command line**: contains a `/` or ends in `.toml` → a file; otherwise → a built-in name. (The machines chapter explains why the filesystem is never probed.) A file path is relative to *this* file.

Two rules about where it goes:

- **`base` is processed before every other key**, whatever order the file is written in.
- **`base` must appear before the first `[[board]]`.** You cannot inherit a backplane you have already started modifying.

`base` nests **up to 8 deep**. A machine built on a machine built on `default` is fine.

This is the key that makes the format worth using. Without it, every variant of a machine is a copy of a hundred lines, and the day you change one of them you change it in six files. With it, a machine file says only **what is different**, and reads as the diff it actually is.

`startup` — the operator's keystrokes, written down

```
startup = ["RUN FF00"]
```

An array of **ordinary monitor commands**, run once the machine is built. Any command. They run in order, and you see them run — that is the `startup>` line in the quick start.

Paths inside a `startup` command are relative to the machine file, not to your shell, because the file's author wrote them and the file's author could see the file's directory. This is the one place the two halves of the path rule sit next to each other, and it is worth remembering.

What `[machine]` will *not* take

```
[machine]
clock_hz = 2000000        # ERROR
sense    = 0x80           # ERROR
```

Both are **rejected, with an explanation**:

```
mine.toml: clock_hz belongs to the CPU BOARD, not to [machine] --
  the crystal is on the board. Put it in the CPU's [[board]]:
      [[board]]
      type     = "8080"
      id       = "cpu0"
      clock_hz = 2000000
```

The crystal is soldered to the **88-CPU card**. The sense switches are on the **front panel**. Neither is a property of "the machine" — the machine is just the box they are plugged into. If you pull the CPU card out, the crystal goes with it.

These two get a custom error rather than the generic *unknown key* because they are the two people reach for first, and being told *where the thing actually lives* is more use than being told it isn't here.

`[[board]]` — and it has four forms

This is the heart of the format. **What a `[[board]]` entry means depends on whether it has a `type`, and whether its `id` is one the base already used.**

Write	And it means
`type` + a **new** `id`	**ADD** the board
`type` + an id **from the base**	**REPLACE** the board outright
no `type` + an id	**MODIFY IN PLACE**
`remove = true` + an id	**PULL THE BOARD OUT**

`id` is always mandatory. It is how you refer to the board at the prompt, and how a later file refers to it here.

ADD — `type` + a new id

```
[[board]]
type = "virtc"
id   = "vi0"
```

A board that was not there is now there. **In a file with no `base`, this is the only form** — there is nothing to modify, replace or remove.

REPLACE — `type` + an id the base already used

```
[[board]]
type = "2sio"
id   = "sio0"
port = 0x20
```

If the base had a board called `sio0`, it is **gone** — pulled out and thrown away — and a fresh `2sio` is fitted in its place. **Everything the base set on that board is lost**, including the settings you did not mention. You get the type's defaults, plus whatever you write here.

That is what "replace" means, and it is almost never what you want. You want:

MODIFY IN PLACE — no `type`

```toml
[[board]]
id    = "cpu0"
clock_hz = 2000000
```

Leave the `type` out and you are reaching into the board that is already there. Everything the base set on `cpu0` stays set; you change the crystal and nothing else.

The absence of `type` is the whole signal. It reads oddly for about a day and then reads as exactly what it is: *I am not fitting a board, I am adjusting one.*

REMOVE — `remove = true`

```toml
[[board]]
id     = "acr0"
remove = true
```

The board is pulled out of the backplane. Its ports stop being decoded. Nothing else in the file may mention it.

The error that catches a copy-paste

`type` + an id that *this same file* has already declared is an error. Not a replace — an error. Within one file, declaring the same board twice is never something you meant; it is a block you copied and forgot to rename. The file will not load, and it will tell you which id.

(Across files it is different: an id from your *base* is a board you inherited, and replacing it is a legitimate thing to want.)

Everything else on a `[[board]]` is a property

`type`, `id` and `remove` are the only keys the config layer understands. **Every other key is handed straight to the board.**

```toml
[[board]]
type = "2sio"
id   = "sio0"
port = 0x10          # the 2sio knows what a port is. The config layer does not.
```

The config layer knows nothing about ports, baud rates, sense switches or drive counts. It cannot, and it does not try. It routes the key to the board and the board accepts it or rejects it by name:

```
mine.toml: [[board]] cpu0: cpu0 has no property 'frobnicate'. Known: clock_hz idle achieved_hz
```

The full key list for every board is the board reference at the back of this manual. The boards chapter says what the boards *are*.

`[board.unit.<name>]` — settings that belong to one unit

Some boards carry more than one independent thing. An 88-2SIO is **two 6850 ACIAs**, not one chip with two channels: unit `a` and unit `b` have their own baud rate, their own interrupt strap, their own endpoint, and they share nothing at all. So they get their own tables.

```
[[board]]
type = "2sio"
id   = "sio0"
port = 0x10                        # the BOARD's property -- both chips live at this base

  [board.unit.a]
  baud    = 9600                   # channel A's property
  connect = "console"

  [board.unit.b]
  baud    = 1200                   # channel B is a different chip. It does not care.
  connect = "socket:2323"
```

A key in `[board.unit.a]` is exactly the key `SET sio0:a baud=9600` takes at the prompt. It is the same property, reached two ways.

The board reference lists which boards have units, and what each unit takes.

`[[board.region]]` — memory

A `memory` board is **a list of regions**, which is why one physical card can carry 56K of RAM and a boot PROM at the top of memory. The regions are the board.

```toml
[[board]]
type = "memory"
id   = "mem0"

  [[board.region]]
  type = "ram"
  at   = 0x0000            # HEX -- it is an address
  size = "56K"             # DECIMAL -- it is a count

  [[board.region]]
  type   = "rom"
  at     = 0xFF00
  size   = 256
  mount = "turnmon.bin"    # relative to THIS FILE
```

Key	
`type`	**required.** `ram` or `rom`
`at`	the address it decodes. **Hex**
`size`	how big. **Decimal**; `K` and `M` suffixes work
`mount`	a ROM image: a file path, or `builtin:<name>`

(A size with a suffix is written as a string — `size = "56K"` — because `56K` is not a number TOML will accept bare. A plain count needs no quotes: `size = 256`.)

An empty socket

A `rom` region with no `mount` is an empty socket. It decodes nothing, and reads there float to `FF` — because that is what an S-100 bus with nobody driving it does. It is not zeros, and it is not an error. It is an unpopulated socket on a card that has one, which is a thing a real machine could be, and software that reads it gets `FF`.

`[[board.drive]]` — disks

A disk controller addresses drives; a drive holds an image.

```toml
[[board]]
id = "dsk0"

  [[board.drive]]
  unit     = 0            # DECIMAL -- it is a drive number
  mount    = "cpm.dsk"    # relative to THIS FILE
  readonly = false
```

Key	
`unit`	the drive number. **Decimal**
`mount`	the image file
`readonly`	refuse every write at the controller, so the host file cannot change. For a disk you mean to read — see the disks chapter. `writeprotect` is the same key under the name the rest of the program uses; write either
`media`	force a format instead of probing the image
`create`	make the file, empty, if it is not there — then mount it. `MOUNT … CREATE`, written down

`media` is the escape hatch. The controller normally works out the format from the image, and normally it is right; when it is not — a headerless image, an unusual geometry — you say so. It is also what says how big a **blank** disk is, since a blank one matches no format at all. The disks chapter covers the formats, and `create`.

Without `create`, a `mount` naming a file that is not there is an **error and the machine does not load** — the same rule as everywhere else here, that a thing which looks like it worked and did not is the worst outcome available.

Numbers: hex on the wire, decimal for counts

The machine file follows the same number rule as the monitor (the *Monitor* document states it in full): **anything the 8080 sees on the bus is hex; anything that never reaches the bus is decimal.**

```toml
port  = 10        # 0x10 -- a port is on the wire, so it is HEX. This is SIXTEEN.
at    = 0xFF00    # an address
sense = 80        # 0x80
baud  = 9600      # a rate  -- DECIMAL
size  = 56        # a count -- DECIMAL
```

`port = 10` is port sixteen. That is the line that catches people: a port is on the wire, ports are hex, and every listing from 1976 wrote the 2SIO at 10. In your own files, be explicit — the forcing markers all work here: `0x10` / `$10` / `10h` for hex, `0o20` / `20q` for octal, `0b10000` for binary, `#16` for decimal, and a `K` / `M` suffix is always a decimal count. To read and print in octal throughout, set `[console] base = octal` (below) — that changes the base, not the rule. Board settings keep their own base regardless, so `SHOW` prints a port as `0x20` however you wrote it.

`[console]` — your terminal, which is not a board

```
[console]
attn      = 0x05      # the key that gets you back to the monitor. ^E
base      = hex       # hex | octal -- how you read/write addresses, ports, bytes
upper     = false
strip7in  = false
strip7out = false
crlf      = false
echo      = false
bell      = true
bsdel     = "off"
```

`[console]` is **not a** **`[[board]]`** and it is not in the backplane. It describes *the terminal you are sitting at* — a piece of equipment on your desk, on the far end of a cable, in 2026. The Altair never knew anything about it.

Key	
`attn`	the escape byte. **Hex.** Default `05` = `^E`
`base`	`hex` | `octal` — how the monitor reads and prints the wire class (addresses, ports, bytes). `octal` is split octal, the MITS front-panel convention
`upper`	fold input to upper case
`strip7in`	clear bit 7 of everything the guest receives
`strip7out`	clear bit 7 of everything the guest sends
`crlf`	translate line endings
`echo`	echo typed characters locally
`bell`	let the guest ring your terminal's bell
`bsdel`	`off` | `bs` | `del` — what your Backspace key sends

Because they are the terminal's, they reach as far as the terminal does and no further: send a board's console unit somewhere else — `CONNECT sio0:a socket:2323`, or out a real serial port — and the far end gets the bytes exactly as the guest wrote them, all eight bits. The settings are not undone; the byte just no longer passes through the console, and every line in the machine is 8-bit clean. The serial chapter has the full story, and what to set instead.

`[display]` — your video window, which is not a board either

```
[display]
focus = true
crt   = true
```

Same idea as `[console]`, one table down: it describes *the window on your desk in 2026*, not the card that draws into it. A VDM-1 has no opinion about window managers, and a machine with two video boards still has one operator with one keyboard — so these settings live here, once, rather than on each board. (Window *size* is the exception: how big a picture opens is the board's own, so `width` is a property of each video board — see Boards.)

Key	
`focus`	whether the video window comes to the front and takes keyboard focus when it opens. Default `false`
`keyboard`	whether a focused window's keystrokes reach the machine's console: `console` (default) or `none` (display-only). See below
`crt`	paint the window like the original monitor — a soft phosphor glow and the tall 4:3 tube — instead of crisp square pixels. Default `false`. See below

With `focus = false` — the default — the terminal keeps the keyboard. The window opens behind whatever you were doing, and when the guest stops you can type at `altairsim>` immediately. You can still click into the window and type there; the keys join the terminal's on one stream.

With `focus = true` the window comes to the front when it opens and keeps the keyboard when the guest stops. That is what a **Sol-20** wants, because there the window *is* the console and the terminal is the back door.

How *big* the window opens is set per video board, not here — see the `width` property in Boards. Each board's video-out could drive its own monitor on a real Altair, so the size belongs to the board whose picture it frames — and each video board opens its **own** window. The `[display]` keys here are **shared across all of a machine's windows**: one operator has one keyboard, so `focus`, `keyboard` and `crt` are a session-wide choice, not a per-window one.

`keyboard` decides whether a video window is a *keyboard* at all — which is separate from whether it has focus. Default `console`: a focused window is a keyboard, and its keys join the terminal's on the one console stream, which is right for a **Sol-20** where the window *is* the console. Set it to `none` for a **display-only** board like a **Dazzler**: the window still shows the picture and still comes to the front, but its keystrokes do **not** reach the console — they drive a joystick (a `d7a`, if the machine has one), and only `Ctrl-E` in the window is honored, stopping the guest and handing you back the monitor. This is why typing in a Dazzler game window does not land at the CP/M prompt.

`crt` changes how the picture is *painted*, not what the machine draws. With `crt = false` — the default — you get today's look: the board's pixels as crisp squares, scaled up a whole number of times so a 1970s pixel stays a sharp square on a modern panel. Set it to `true` for the period monitor instead. The short, wide raster these boards scan — a VDM-1 is 512×208, a VDB is 640×240 — was never square: it was stretched to fill a 4:3 tube, and the beam softened each row into the next rather than drawing a grid of hard dots. `crt = true` reproduces both, stretching the picture to the 4:3 shape and softening the raster so the text reads as phosphor glow instead of sharp squares. It is a matter of taste — some prefer the crisp look, some the tube — and you can flip it with `SET DISPLAY crt=on` / `off` at the monitor, and the open window re-fits at once — no need to reopen it. Under the tube look a window opened at a specific `width` fills exactly that many pixels; the crisp look keeps the whole-number scaling that makes its squares sharp.

On a build without SDL3 these keys are still accepted and simply have no window to apply to, so a machine file that asks for them stays portable.

The transform chain belongs to the console

The `[console]` keys above (`strip7out`, `upper`, `crlf` …) are the **only** thing in the simulator that alters a byte, and they belong to the console because that is where a human is reading text. **Every serial line is 8-bit clean** — there is no bit-masking strap on any board, because a line may carry XMODEM and a line that eats bit 7 is a line you cannot trust with a file. So the fix for MITS BASIC's garbled `MEMORY SIZ?` prompt is `strip7out` on the console, never `data_bits = 7` on the card. The serial chapter explains why in full.

A complete machine file

Small, whole, and it works. No `base` — so every board is an ADD:

```toml
[machine]
name    = "tiny"
startup = ["RUN 0"]

[[board]]
type = "fp"                # the front panel: the sense switches at port FF
id   = "fp0"
sense = 0x00

[[board]]
type     = "8080"          # the CPU is a board. The crystal is on it.
id       = "cpu0"
clock_hz = 0               # 0 = flat out. This is the default.

[[board]]
type = "2sio"              # the console board
id   = "sio0"
port = 10                  # HEX. Port SIXTEEN.

  [board.unit.a]
  baud    = 9600           # DECIMAL. Nine thousand six hundred.
  connect = "console"

[[board]]
type = "memory"
id   = "mem0"

  [[board.region]]
  type = "ram"
  at   = 0x0000
  size = "16K"

[console]
strip7out = true
```

A `base` delta — and this is the one you will actually write

This is genuine. It is how the CP/M example is built, and it is nine lines:

```
[machine]
name     = "cpm22"
base     = "default"        # a front panel, an 8080, a 2SIO console, a floppy
                            # controller, 56K of RAM and the boot PROM at FF00
startup = ["RUN FF00"]      # the operator's own keystrokes, written down

[[board]]
id = "dsk0"                 # NO type: modify the controller the base already has

  [[board.drive]]
  unit  = 0
  mount = "cpm.dsk"         # relative to THIS FILE
```

Everything a 56K CP/M machine is, `default` already was. The only thing this file has to say is *which floppy is in drive 0*, and *press RUN at FF00*. **That is the whole of the difference, and so that is the whole of the file.**

Note what is not here: no `type` on the `[[board]]`, because `dsk0` already exists and we are adjusting it, not fitting it. Had we written `type = "dcdd"`, we would have thrown the base's controller away and got a fresh one with default settings — and it would still have worked, and we would never have known we had done it. Leave the `type` out.

Saving and loading at the prompt

```
altairsim> CONFIG SAVE mine.toml
altairsim> CONFIG LOAD mine.toml
```

`CONFIG SAVE` writes the machine you are actually running — every board, every property, as it stands right now, including everything you changed with `SET` since you started. It **round-trips**: load what it wrote and you get the machine back.

`CONFIG LOAD` is the whole machine, so it replaces the one you have — the same thing that naming the file on the command line does, and there is no undo but the file you saved it to. It is also **all or nothing**: the machine is built off to one side first, so a file that will not load leaves you exactly where you were rather than halfway between two machines.

Which makes it the fastest way to write a machine file. Build the machine at the prompt with `BOARDS ADD` and `SET` until it is what you want, then save it, then edit the file down to the parts you care about — or give it a `base` and delete the rest.

Boards

An Altair is a **backplane**. Everything else is a board in it — the memory, the serial ports, the disk controller, the front panel, and the processor itself. There is no "machine" underneath the boards doing the real work; take the boards out and there is nothing left but a bus.

`altairsim` is built that way on purpose, and it is the reason the CPU's crystal is a property of the CPU *board* and the sense switches are a property of the *front panel*. Not nitpicking: it is what lets you pull a board out, put a different one in, and find out what the software does about it.

This chapter says what the boards **are** — what the real hardware was, what it is for, and what will bite you. **It does not list their parameters.** Every key of every board is in the board reference at the back of this manual, printed from the program's own tables, which is why it cannot be wrong.

The boards

Grouped by what they do — the same order as the sections below.

Memory

Type	What it is
`memory`	RAM and ROM, as a list of regions
`bankmem`	bank-switched RAM — Vector Graphic, Cromemco 64KZ, North Star HRAM, ExpandoRAM II
`v2z80rom`	S100Computers V2 Z80 CPU board's onboard MASTER monitor ROM — a paged EEPROM, not a processor

Processors

Type	What it is
`8080`	the MITS 88-CPU
`8085`	an 8085 CPU board — the 8080's binary-compatible superset, with RIM/SIM and the TRAP/RST interrupts
`z80`	a Z80 CPU board — the same bus, a different instruction set
`6800`	a Motorola 6800 — the CPU of MITS's *other* machine, the Altair 680b. See the note below

Serial ports and consoles

Type	What it is
`2sio`	MITS 88-2SIO — two serial ports. The usual console
`sio`	MITS 88-SIO — one serial port. MITS's first
`sbc`	SD Systems SBC-100/200 — a Z80 single-board computer's serial console
`propio`	S100Computers Console I/O — a Propeller-based serial console
`pmmi`	PMMI MM-103 — a Bell 103 telephone modem on one card

Cassette

Type	What it is
`acr`	MITS 88-ACR — the cassette interface
`uio`	MITS 88-UIO — a serial port and a cassette, on one card

Printers

Type	What it is
`c700`	MITS 88-C700 — the line-printer controller. Capture to a file
`lpc`	MITS 88-LPC — the other line-printer controller, line-buffered

Parallel and analog I/O

Type	What it is
`pio`	MITS 88-PIO — an 8-bit parallel port, in and out
`4pio`	MITS 88-4PIO — up to four programmable parallel ports
`d7a`	Cromemco D+7A — analog and parallel I/O; reads joysticks

Floppy and disk controllers

Type	What it is
`dcdd`	MITS 88-DCDD — the 8″ floppy controller
`mds`	MITS 88-MDS — the 5¼″ minidisk controller
`hdsk`	MITS 88-HDSK — the Datakeeper hard-disk controller
`versafloppy`	SD Systems VersaFloppy I/II — a soft-sector floppy controller. Boots SDOS
`tarbell`	Tarbell #1011 — a single-density floppy controller with its own boot PROM. Boots CP/M by itself
`tarbelldd`	Tarbell #2022 — the double-density twin, mixed-density disks
`icom`	iCOM FD3712/FD3812 — an 8″ floppy controller with its own boot PROM. Boots CP/M and FDOS
`dualsd`	S100Computers Dual SD — two microSD cards as CP/M drives. Boots CP/M 3
`dualide`	S100Computers IDE-AB — two CompactFlash cards as CP/M drives. Boots CP/M 3

Video and display

Type	What it is
`vdm1`	Processor Technology VDM-1 — memory-mapped video. Needs a display
`dazzler`	Cromemco Dazzler — color graphics. Needs a display
`vdb8024`	SD Systems VDB-8024 — an 80×24 video terminal on one board. Needs a display
`sol`	Processor Technology Sol-PC — the Sol-20's onboard I/O, on one card

Interrupts and the clock

Type	What it is
`virtc`	MITS 88-VI/RTC — vectored interrupts and a clock
`ss1`	CompuPro System Support 1 — a multifunction board: real-time clock, serial channel, interval timer, and dual interrupt controllers

The whole machine

Type	What it is
`fp`	the front panel
`turnkey`	MITS 8800b Turnkey Module — the front-panel-less Altair, on one card

Host integration

Type	What it is
`hostbridge`	file transfer to your host. **Ours, not a period card**

PROM programmers

Type	What it is
`pb1`	SSM PB1 — burn a 2708/2716 EPROM, then save it as a hex file

---

Memory

RAM, ROM, and the boards that switch banks of it.

`memory` — RAM and ROM

A memory board is **a list of regions**, and the regions are the board. That is not a modelling convenience; it is what an S-100 memory board was. One physical card carried banks of chips decoding whatever ranges its jumpers said, and a card with 56K of RAM low and a 256-byte boot PROM at `FF00` is a perfectly ordinary card.

So `default` has exactly one memory board in it, and that board is the 56K *and* the PROM.

`PHANTOM*` — how a boot PROM gets out of the way

The bus has a line called `PHANTOM*`. **A board pulls it to switch another board off.** When the PROM at `FF00` is being read, it asserts `PHANTOM*`, and the RAM card underneath — if it is jumpered to honour it — shuts up. Two boards decode `FF00`; only one answers.

This is how an Altair with disks boots. The PROM overlays the RAM at the top of memory, the loader runs out of it, and then **the loader gets out of the way** and the RAM underneath is uncovered — which matters, because CP/M wants that memory back.

Whether a board honours `PHANTOM*`, and whether it asserts it, are **jumpers**. They are on the board and they are yours to set. Getting them wrong produces a machine that does not boot and does not say why — which is precisely what it did in 1977, and the bus view in the monitor will show you both boards claiming the page.

Banking is its own board

Sixty-four kilobytes was not enough for very long, and the industry's answer was **bank switching**: several cards' worth of RAM at the same addresses, with a write-only port that says which is live. Nobody agreed on how — so banking is **not a knob on `memory`**. It is its own board, **`bankmem`** (below), and each real card it models owns its own decode. A plain `memory` board is exactly that: plain, unbanked RAM and ROM.

`bankmem` — bank-switched RAM

When 64K stopped being enough, S-100 makers put several planes of RAM at the same addresses and a write-only **select port** that chose which plane the CPU saw. Every maker did it differently, so `bankmem` is **one board with four decoders**, chosen by `card`:

`card`	Real board	Select port	What a write does
`vector`	Vector Graphic 64K	`40`	**one-hot —** `01`→bank 0, `02`→1, `04`→2 … `80`→7
`cromemco64kz`	Cromemco 64KZ / 64KZ-II	`40`	**8-bit mask —** bit *N* turns bank *N* on; **several at once** (`28`→banks 3 and 5)
`northstar`	North Star HRAM	`C0`	bit 0 = on/off, bits 1–7 = which bank; banks toggle **one at a time**
`expandoram2`	SD Systems ExpandoRAM II	`FF`	the byte is a **page number** (an approximation — see below)

`banks` sets how many planes the card carries (one per real board): up to 8 for `vector` and `cromemco64kz`, 6 for `northstar`, 10 for `expandoram2`. `fill` and `seed` behave exactly as they do on `memory`. The select port is write-only and the **guest** drives it; from the monitor you can drive it yourself with `OUT`, and `SHOW` lists every plane and which is live.

There is no banked operating system in the box to boot, so this board is here to be *driven* — the quickest way to see it work is from the monitor:

```
altairsim bankmem
OUT 40 01            ; select bank 0
DEPOSIT 1000 A0
OUT 40 08            ; select bank 3 (one-hot 0x08, not bank 8)
DEPOSIT 1000 B3
OUT 40 01            ; back to bank 0 — DUMP reads A0
DUMP 1000-1000
OUT 40 08            ; bank 3 — DUMP reads B3; the plane really swapped
DUMP 1000-1000
```

> **`expandoram2` is an approximation.** The real board decodes the page number through an on-board PROM into a 32K or 48K partition; that decode is not published in a form we can reproduce faithfully, so `bankmem` models a plain page-select over 64K planes and says so here. The other three cards are exact.

`v2z80rom` — the S100Computers V2 Z80 CPU board's onboard monitor ROM

This board is not a processor — it is the **monitor ROM** that a real S100Computers V2 Z80 CPU board carries on it: an **8K EEPROM** at `F000`–`FFFF` holding John Monahan's MASTER V6.6 ROM monitor. It is a separate board from the CPU on purpose — a machine that uses it still needs a `z80` (below) beside it for the processor, exactly as the real card is a Z80 with its own onboard firmware.

The EEPROM is **paged**: two 4K halves both live at `F000`–`FFFF`, and a write to port `D3` chooses which is visible (and can switch the EEPROM off altogether, so the RAM underneath shows through). That is how CP/M gets a flat 64K after boot — it inactivates the EEPROM and the monitor's window becomes ordinary memory. While it is on, the EEPROM shadows the RAM in its window for reads.

There is no `BOOT` verb — **the monitor is the boot command**. `startup = ["RUN F000"]` cold-starts it, and at its `->` prompt the **`I` command** boots CP/M 3 off a Dual SD card (below). This is the board that makes `altairsim dualsd` go.

The CPU boards — one plugs in, and it drives the bus

`altairsim` has three S-100 processor boards — `8080`, `8085`, and `z80` — and everything in this shared section is true of all three. Each is described on its own below; what they have in common is here.

A fourth CPU, the Motorola `6800`, is not one of them. It drives MITS's *other* machine, the Altair 680b — a different bus, with memory-mapped I/O rather than S-100 ports, so little of this section applies to it. The `6800` and the 680b's own I/O boards (`680io`, `680uio`, `680kcacr`) are catalogued in the board reference at the back, and the `altair680` machine boots MON680 out of the box.

The processor is a board like any other. It plugs into the backplane, it can be removed, and with `-n` you can build a machine that does not have one. It decodes no ports and answers no addresses. What it does is **drive the bus** — which makes it unlike every other board in the box, and is exactly what a CPU card did. Put a `z80` where an `8080` would go and the bus, the boards, and the debugger neither know nor care; that is the whole point of keeping the processor a board.

Every CPU board carries the same three properties: **`clock_hz`** (the crystal), **`idle`** (stands the processor down at a prompt), and the read-only **`achieved_hz`** (the speed it actually reached).

The crystal is on the board — `clock_hz`

Because the crystal is soldered to the CPU card, `clock_hz` is the *board's* property, not the machine's. **`clock_hz = 0` is the default, and it means run flat out** — on a modern host, north of a hundred times a real Altair.

```
SET cpu0 clock_hz=2000000
```

buys back the real 2 MHz machine, and it is worth doing once. **What the guest sees is identical either way**: instructions cost the same T-states and a cassette loads in the same number of them, so the crystal buys period *feel*, not period *behaviour*.

That holds everywhere except at the edge of the machine. A guest counts instructions to measure time, so flat out it retires a "three-second" timeout in milliseconds of yours — which is why anything the guest times against the *outside* world (XMODEM through a serial port) wants the real crystal. The troubleshooting chapter has the full story.

`idle` — the CPU stands down at a prompt

At a prompt a guest is only spinning on the serial status register waiting for a keystroke, and flat out that pins a core to accomplish nothing. **`idle` (on by default) stands the processor down while the guest is polling an empty keyboard** — a pinned core becomes a few percent — and **the guest cannot tell**, because the moment a byte arrives the processor is back before the next poll. An XMODEM transfer through an idling machine is byte-exact.

`8080` — the MITS 88-CPU

The original — the board the Altair shipped with, and the one the other two processors stand in for. As a board it is the plain case of everything above: it drives the bus, decodes nothing, and carries `clock_hz`, `idle`, and `achieved_hz`.

`8085` — an 8085 CPU

The 8080's own successor, and a binary *superset* of it — every 8080 program runs on an 8085 unchanged, which makes it the closest of the three to the 88-CPU. Everything in *The CPU boards* above applies unchanged; what follows is only what the 8085 adds.

Over the 8080 it adds `RIM` and `SIM` (read and set the interrupt mask and the SID/SOD serial pins) and the on-chip interrupts — `TRAP` plus `RST 5.5`, `6.5` and `7.5`, layered on the 8080-style `INTR` line. The documented set is faithful (including the one instruction that differs, `ANA`/`ANI` always setting the auxiliary carry), and the undocumented opcodes execute too, with the extra V and K flag bits some of them set; `DISASM` marks each undocumented byte the way `DDT` does. The built-in `8085` machine is a minimal one — an `8085`, 64K of RAM, and a 2SIO console.

What no board drives is the 8085's on-chip *pins*: the `SID`/`SOD` serial lines and the `TRAP`/`RST 5.5`/`6.5`/`7.5` interrupt inputs. Nothing on the S-100 bus carries them, so no card asserts them; ordinary interrupts over the `INTR` line work as they do for the 8080.

`z80` — a Z80 CPU

The other processor you can drop into the slot — the same backplane, a different instruction set behind it. Everything in *The CPU boards* above applies unchanged. The built-in `z80` machine is a minimal one — a `z80`, 64K of RAM, and a 2SIO console — for putting it through its paces.

Serial ports and consoles

The boards that carry a serial port — a console, a modem, or both.

`2sio` — MITS 88-2SIO

Two **6850 ACIAs**, units `a` and `b`, four ports at BASE+0 through BASE+3. Base defaults to `10` hex, which is where every listing from the period expects it.

This is **the usual console board**, and it is what `default` has. If you are running CP/M or Microsoft BASIC, this is the board the software is talking to.

The two halves share nothing

Not the baud rate, not the endpoint, not the interrupt strap. **They are two independent chips that happen to be bolted to the same board**, and the model says so: `a` and `b` are separate units with separate properties.

So a console on `a` at 9600 and a modem on `b` at 1200, one interrupting and one polled, is not a configuration you have to work around. It is Tuesday.

The serial chapter covers what a channel can be *connected* to: your terminal, a TCP socket, or a real serial port on your host, with the modem control lines wired through.

`sio` — MITS 88-SIO

One **COM2502 UART**, unit `tty`, two ports. This was **MITS's first serial card** — it predates the 2SIO, and the earliest Altair software talks to it. `basic4k` uses it.

Its status bits are inverted

A clear bit means ready. Read that twice, because every instinct you have says otherwise, and because it will make you certain you have found a bug in the simulator.

You have not. **It is a fact about the chip**, not a quirk anyone invented: the COM2502's status lines came out of the package active-low, MITS wired them to the data bus as they were, and every program that drove an 88-SIO was written knowing it. `basic4k`'s I/O routine masks and branches on zero, and it is right to.

The port must be **even**: control at BASE, data at BASE+1.

`sbc` — SD Systems SBC-100/200

SD Systems built S-100 boards for people who wanted a whole computer on as few cards as possible, and the **SBC-100** and **SBC-200** are the heart of one: a **Z80 single-board computer** — processor, some memory, and a serial console — on a single card. `altairsim` models the console half, which is the part the software talks to.

That console is an **Intel 8251 USART**, not the 6850 the MITS boards use — unit `tty`, with data at `7C` and the status/command register at `7D`. Software written for a 2SIO will not drive it; the SBC's own **SD monitor** will.

It measures your terminal's speed

The board's one memorable trick is **auto-baud**. Run `altairsim sbc200` and the SD monitor is waiting — not at a fixed rate, but for you to **press Return**. It times the bits of that one character and sets its own baud to match, so a terminal at any common speed just works. Nothing happens until that first Return, which surprises people: it is not hung, it is listening.

The `sbc200` machine boots the **SD monitor**. Give the machine the **DDBIOS** disk BIOS in a PROM socket and a `versafloppy` controller beside it, and the monitor's `C` command boots **SDOS** — see the VersaFloppy below, and the SD Systems example in `examples/`. `variant` picks the generation (`sbc200` or `sbc100`). The parallel ports, timer and interrupts of the real card are a later phase; the console is here now.

`gsio` — the generic strap-configurable serial board

Most serial boards in this list model **one specific chip** — the 2SIO's 6850, the SBC's 8251, the 88-SIO's COM2502 — so the software has to be written for that chip. **`gsio`** is the other kind: a generic serial board with no fixed chip to imitate. You **describe** the port with straps — which port is status/control, which is data, which status bit means a byte is waiting (`dav`), which means the transmitter is ready (`tbmt`), and whether the shared **`inverter_gate`** flips their sense — and that description *is* the port. It is the general-purpose serial board for reaching whatever polled "read a status bit, then a data byte" interface a piece of software expects.

It carries **two independent serial channels**, units **`a`** and **`b`**, each with its own straps, `baud` and `connect` endpoint, configured under its own `[board.unit.a]` / `[board.unit.b]` table in a machine file. By default `a` answers ports `0` / `1` and `b` answers `2` / `3`.

You rarely set the straps by hand. A **profile** presets them to imitate a known card: **`sior0`** (MITS SIO Rev 0 — the default, what the SSM 8080 monitor expects), `tuart` (Cromemco TU-ART), `imsai-sio2`, `compupro-if2` (CompuPro Interfacer II), `compupro-ss1` (CompuPro System Support 1). Pick a profile per channel, then override any individual strap afterward — a jumpered board, and so is this one. The board is polled, with no interrupts, and does **basic transmit and receive only**: it does not emulate programmable word length, parity or stop bits. A specific card that needs those is a separate, fully emulated board.

`io4` — SSM IO-4 (2P + 2S)

Where `gsio` describes a serial port with straps, **`io4`** is the specific card `gsio` declines to be: the **SSM IO-4**, a combined S-100 board with **two full-duplex serial channels *and* four latched parallel ports** on one card, modeled as the real silicon it is. Each serial channel is a real UART with **programmable word length (5–8 bits), parity, stop bits and baud**, and the full status-word strapping the card was famous for.

Its **two serial channels** are units **`a`** (Serial A) and **`b`** (Serial B), each with its own `[board.unit.a]` / `[board.unit.b]` table, its own `baud` and `connect` endpoint. The four ports of the serial half sit on a four-port block that must start on a four-boundary (default `0`–`3`): `a` answers `0`/`1`, `b` answers `2`/`3`.

You strap it, or you name a personality

The IO-4 was a heavily jumpered card, and every jumper is a property here. Which data-bus bit carries "a byte is waiting" (`dav`), which carries "the transmitter is ready" (`tbmt`), and four more status signals; whether the whole status byte is inverted (`invert_status`); and whether the status and data ports are swapped (`port_reversal`). You rarely set those one at a time. A **`profile`** presets them to imitate a known host: **`altair-rev1`** — the default, the MITS SIO Rev-0 console the SSM 8080 monitor expects — plus `altair-rev0`, `i8251`, `proctech`, `imsai`, and `custom` (every strap left free to roll your own). Pick a profile, then override any individual strap afterward, exactly as you would move a jumper.

The three UART **error flags** — parity, framing and overrun — are strappable to the status byte, but they always read inactive: the serial line here carries exact bytes and models no line noise, so there is nothing for them to report. The card's current-loop and EIA/RS-232 electrical options are an electrical choice, not a programming one; they are not modeled.

The parallel half

The other half is **four 8212 latched ports** — two input, two output — as units **`pa`** and `pb` (Parallel A and B), on their own two-port block (default `4`/`5`) that must start on a two-boundary. Each input latches a byte on its strobe and raises a service-request flip-flop; a read hands the byte over and acknowledges it. `CONNECT io40:pa …` wires a port to a byte source or sink like any serial line. The `dav_bit`, `dav_source` and `dav_active_low` straps set up the "status byte here, data byte there" console idiom the card supports across a pair of ports.

The two halves are mutually exclusive on overlap. Set the serial four-port block and the parallel two-port block to overlapping addresses and **neither section responds** in the contended ports — a deliberate design of the card, not a bus fight.

Interrupts, if you strap them

The card's **W4 header** routes interrupts, and `io4` follows it: each serial channel's receive (`rx_int`) and transmit (`tx_int`), and each parallel input (`int`), can be strapped to a vectored interrupt line, to the plain interrupt pin, or to `none`. There is **no software enable** — the strap *is* the enable, just as on the card — so a stock board with the header bare (the default) boots polled. A strapped transmit interrupt is a level, asserted whenever the transmitter is idle; a parallel input raises its interrupt on the strobe even when the port is not being read. Wire these to a `virtc` and the receive of a character, or a parallel strobe, vectors to the RST you strapped.

`SHOW io40` reports every unit, its ports and its live line state. The SSM 8080 monitor booting on a stock `io4` console is in the examples.

`propio` — S100Computers Console I/O

A **serial console** of the reproduction era: the S100Computers Console I/O board, built around a Parallax Propeller instead of a 6850 or 8251, with status at `00` and data at `01`. It is a polled console, unit `serial` — you `CONNECT` it to a terminal, a file, a socket or a serial port like any other serial board. It is the console the Dual SD machine uses.

Underneath, `propio` is just a **preset**: it is the strap-configurable serial engine (the same one behind the `gsio` board) with this board's documented convention filled in — the ports, and which status bit means receive-ready and which means transmit-ready. Because the real board is jumpered, every one of those straps is still yours to override, so a differently-strapped Console I/O board needs no new board type, just a property or two.

`pmmi` — PMMI MM-103 modem

A **Bell 103 telephone modem on one S-100 card** — the first S-100 modem approved for direct connection to the phone line. In a real machine it dialed, answered, and carried a serial link over the line at up to 600 baud. Here it is the card's **transmit and receive path**: unit `line`, four ports from a base that must sit on a four-port boundary (default `C0`; the DIP switch on the real card set it, and PMMI's own North Star software used `E0`).

Its four ports are the card's quirk: **read and write at the same address are different registers.** Writing `BASE+0` sets the character format (data bits, parity, stop bits) and the modem-control bits; *reading* it gives you UART status. `BASE+1` is transmit on a write, receive on a read. `BASE+2` writes the baud-rate divisor and reads modem status; `BASE+3` writes the modem chip's control word and, read, returns nothing the card drives. The three control registers are **write-only** — the program keeps its own copy of what it wrote, exactly as it had to on the hardware.

There is no telephone network in the box, so **`CONNECT` its `line` to a byte source and sink**: the straightforward test is a pair of paper-tape-style files — `CONNECT pmmi0:line in:incoming.tap,out:outgoing.tap` — where what the guest sends lands in one file and what it receives is read from the other. The bytes are exact; the Bell 103 tones are not simulated, because the line here is a byte stream, not audio.

What it does *not* do yet, on purpose. It does not dial: the make-and-break of the hook relay a period dialer program produces is not decoded into a phone number, and no number picks a far end — **placing the call is `CONNECT`'s job**, and reading a number out of the hook would be a behaviour this card never had. Modem status (dial tone, ringing, carrier, clear-to-send) reads a fixed "connected and ready" value rather than following a real handshake, and the card raises no interrupts. `SHOW pmmi0` reports the live frame, baud, UART flags and modem lines alongside the base address. To try one, add it to `default` — `BOARDS ADD pmmi` fits it at `C0`.

Cassette

Boards that read and write cassette tape.

`acr` — MITS 88-ACR

The **cassette interface**: an 88-SIO channel B with an FSK modem on the end of it, so a byte on the bus becomes an audible tone on a tape and back again. Unit `tape`, default port `06`, and it runs at 300 baud because that is what an audio cassette could carry.

It brings verbs of its own — **`WIND`**, **`REWIND`** and **`EXTRACT`**. The first two are there because a tape has a **position** and a disk does not, and pretending otherwise would help nobody: `WIND` puts the head at a time on the tape (`mm:ss`, or `START` / `END`), so a tape holding several programs one after another is reachable, `REWIND` is the common case of `WIND START`, and `SHOW` reads the counter back the same way. **`EXTRACT`** is a different job — it demodulates a mounted `.WAV` and writes each program it finds out as its own `.TAP` file, so an audio recording becomes something you can mount directly.

This is the board that shows you what an Altair actually was: no disk, no PROM, a bootstrap you toggle in by hand, and eight minutes of listening to a cassette. **The tapes chapter is the one to read**, and Altair 4K BASIC 3.1 is the machine to run.

`uio` — MITS 88-UIO

A serial port and a cassette interface on one board — the Universal I/O card, which is very nearly what a 2SIO channel and an 88-ACR are when you put them on the same card and let them share the parts. It comes up looking like the two boards it replaces: a **6850 serial port** at `10` (unit `serial`) and a **cassette section** at `06` (unit `tape`), the standard addresses, so software that expects a 2SIO console and an ACR tape finds both where it left them.

What it adds over two separate cards is **motor control** and a **modulation switch**. `SW-1` chooses between the two encodings the era used — the MITS 300-baud format and the Kansas City standard — because the UIO was sold to talk to either. The tape half brings the same `WIND`/`REWIND`/`EXTRACT` verbs and the position counter the 88-ACR does.

Printers

Line-printer controllers — each captures its output to a file or a print queue.

`c700` — MITS 88-C700

The **line-printer controller**: an output-only board that sends characters to an Altair C700 printer. Unit `prn`, default port `02` — the MITS default, with Control/Status at `02` and Data at `03`.

There is no printer in the box, so **`CONNECT` its `prn` line wherever you want the output**: a file (`CONNECT lpt0:prn out:printout.txt`), the `console` to watch it print live, a `socket:`, a real `serial:` printer, or a **real print queue on your host** (`CONNECT lpt0:prn printer:linewriter`) — that last one where your build found a print system, and the serial chapter has the job-submission options. The capture is byte-for-byte — the bytes the program sent, control codes and all, not a reformatted page.

Drive it **polled or on interrupts**. Polled: write a character to the data port (`03`), then poll the status port (`02`, bit 0 ACKNOWLEDGE, set = ready) before the next. Or arm its **single-level interrupt** (control bit 1) and let the printer interrupt the CPU each time it takes a byte, so the handler just feeds the next one. The `interrupt` strap picks the wire — pin 73 (`int`, the default), a vectored-interrupt level (`vi0`..`vi7`), or `none`; `interrupt_after` is the card's SW2 #4, firing after every character or only after a CR/LF.

The **`lineprinter`** machine is `default` with one of these already fitted; its output starts on `null`, so `CONNECT lpt0:prn out:printout.txt` to capture it to a file, or `console` to watch it live (that takes the console from the terminal).

`lpc` — MITS 88-LPC

The **other** line-printer controller — for the 88-LP printer, where the `c700` drives the C700. Same two-port shape (control at an even base, data above it; MITS default `02`), but it drives the mechanism the way it really worked, and the difference shows in what you capture.

The C700 is a **transparent byte pipe**: the bytes the program sends are the bytes you get, control codes and all. The **LPC is line-buffered**. The guest loads a **6-bit character code** at a time into an **80-character line buffer**, and nothing prints until it sends a **PRINT** command — or the buffer fills. **LINE FEED** and **CLEAR** are commands too. So the capture is the printed *page* — the codes decoded to their glyphs, one text line per printed line — not a byte stream, because on this board the line breaks are commands, not data.

`CONNECT` its `prn` line wherever a line can go — an `out:` file, the `console`, a `socket:`, a real `printer:` queue. The **`lineprinter-lpc`** machine has one fitted at `02`, its output on `null` until you `CONNECT` it. It is polled; unlike the `c700`, this card's interrupt is not modeled.

Parallel and analog I/O

Parallel ports, and one board that also reads analog inputs.

`pio` — MITS 88-PIO

An **8-bit parallel port**, in and out — the simplest way to move a byte that is not a serial character. It has **two lines you `CONNECT` independently**: `out` (an output device — a printer, a socket) and `in` (an input device — a keyboard, another socket), so one board can punch to an `out:` file *and* read a keyboard off the `console` at once, because the two directions are two separate connections. Default ports `04` / `05`. It is **polled**: a byte moves when a driver polls the status port for it.

`4pio` — MITS 88-4PIO

The programmable cousin of the `pio`: up to **four Motorola 6820 PIAs** on one card, whose **data direction the guest sets in software** rather than the board fixing it. Each populated port is a section — `ja`, `jb`, and so on — and each section is its own connectable line, so a card with two PIAs fitted gives you several independent ports to `CONNECT`. Sixteen ports from a default base of `20`. Polled, like the `pio`.

Between them the two parallel boards cover the span from *a fixed eight bits each way* to *however the software wants it configured* — the same range the real MITS parallel line did. The **`parallel`** machine has a `pio` fitted and capturing to a file.

`d7a` — Cromemco D+7A

An **analog and parallel I/O card** — one parallel port and **seven analog channels** in a block of eight ports (default base `18`). Each analog channel is an **A/D converter when you read it and a D/A converter when you write it**, in 8-bit two's-complement: `00` is 0 V, `7F` about +2.5 V, `80` about −2.5 V. On a real bench it read sensors and drove instruments.

Here its job is the input end of a **game console**. It reads **one or two JS-1 joysticks**: the X and Y pots on analog channels, and the four buttons — **active-low** — packed into the parallel byte, low nibble for one stick, high nibble for the other. The sticks come from your host through the same kind of injected service the display uses: a **USB gamepad** where SDL3 is present, or the **keyboard** as a fallback (arrows and a few keys), and nothing at all in a headless build, which still runs.

`joystick1` / `joystick2` choose which host device drives each console. Both default to `auto`, which claims a **different** gamepad per console — console 1 takes gamepad 0, console 2 gamepad 1 — so two controllers work with no configuration, each falling back to the keyboard when its gamepad is absent. `SHOW <id>` shows what each console currently resolves to (a named controller, the keyboard, or nothing), and `SHOW JOYSTICKS` lists the controllers your host actually sees. The Dazzler examples in `examples/` pair the board with color graphics and set the video window to be a **display, not a keyboard** (`[display] keyboard = none`), so your keystrokes drive the stick instead of landing at a prompt. (The board's designed-but-unbuilt sound output — a JS-1's speaker is a D/A the CPU writes a waveform to — is not here yet.)

Floppy and disk controllers

The disk controllers, from 8-inch floppies to CompactFlash — several boot an operating system by themselves.

`dcdd` — MITS 88-DCDD

The **8″ hard-sector floppy controller**, up to sixteen drives, three ports at `08`, `09` and `0A`. **This is the board CP/M booted from**, and it is in `default`.

It also carries the **8 MB medium** — a large-capacity format the same controller can address, on an image **you supply**; no 8 MB disk is in the package.

Its status bits are **inverted**, for the same reason the 88-SIO's are and with the same consequence: a clear bit means ready.

The disks chapter is where this board lives: formats, mounting, write protection, and the track-buffer trap that means you should get back to the `A>` prompt before you stop the machine.

`mds` — MITS 88-MDS

The **5¼″ minidisk**, four drives. **The same registers as the DCDD** — a program written for one will drive the other — but **different physics**:

	`dcdd`	`mds`
Spindle	360 RPM	**300 RPM**
Byte time	32 µs	**64 µs**
Motor	always turning	**stops after 6.4 seconds**

The minidisk's motor is not permanently on. It spins up, and if nobody touches the drive it spins down again — which the software has to cope with, and which you can watch it cope with by setting the motor to `real`. By default the motor is `free`: always at speed, no waiting. The DCDD needs no such switch, because its spindle never stopped.

A minidisk image is one you supply. The board is here and the `minidisk` machine boots its PROM, but no 5¼″ image is in the package, so the drives come up empty.

It cannot share a machine with a `dcdd`

Same three ports. Two controllers decoding `08` is not a limitation of this program; it is the MITS address map, and a real Altair with both cards in it would have had them fighting on the data bus. Fit both here and the bus view will name the contention rather than leave you wondering why the guest has gone strange.

Pick one.

`hdsk` — MITS 88-HDSK Datakeeper

A **hard-disk controller** — the "Datakeeper" — and the board CP/M boots from when it is not booting from a floppy. It is an outboard controller with a **command/handshake protocol** and four internal **256-byte page buffers**, so unlike the floppy cards it **moves whole sectors for you** rather than shifting bits in real time. Eight ports, default `A0`–`A7`.

The **HDBL** boot PROM at `FC00` reads the disk's descriptor page and brings the system up. There is a ready-made machine for it in `examples/`, image and all: run it and you land at `A>` on a multi-megabyte CP/M 2.2 platter, **read/write**, with CP/M saving to it. Its README says how.

`versafloppy` — SD Systems VersaFloppy I & II

SD Systems' **soft-sector floppy controller**, built around a **Western Digital FD177x** — and the board that boots **SDOS**, a CP/M work-alike, on an SBC-200.

It is **one board covering both generations**, chosen with `variant`: **`vfii`** (the default) is the double-density **FD1791** VersaFloppy II, and **`vfi`** the single-density **FD1771** VersaFloppy I. They differ only in the controller chip and a few control bits; the port block (eight ports, default `60`) and the driver family are the same. Neither carries a boot PROM — the bootstrap BIOS lives on a separate PROM, which on the SBC-100/200 is the onboard socket.

Fit a `vfii`, mount an 8″ double-density disk, and with the SBC-200's **DDBIOS** the monitor's disk commands come alive: **`C`** cold-boots SDOS, **`R`** and **`W`** read and write sectors. The SD Systems example in `examples/` is that machine with the disk already in it: press Return for the auto-baud, type `C`, and *32K SD-OS* comes up to its `[A]` prompt.

What it will not do

`Z` **formats** a disk, and here it cannot make one from nothing. A raw disk image is only its data — it has none of the gaps and address marks a real format writes *between* the sectors — so there is nothing for a low-level format to lay down, and the controller says so (a WRITE FAULT) rather than pretending. This is the honest limitation every soft-sector controller here shares: mount a disk that already carries a format and read and write work; ask the board to create a blank one and it tells you it can't. For a fresh SDOS disk, copy one that is already formatted.

`tarbell` — Tarbell #1011 single-density floppy

The **Tarbell Electronics #1011** (July 1977) was the S-100 floppy interface that booted CP/M on a whole generation of Altair and IMSAI machines. It is a **Western Digital FD1771** soft-sector controller — eight ports, default `F8` — and, unlike the VersaFloppy, it carries **its own 32-byte boot PROM**. That is the whole experience of this card: you do not type a boot command.

Power on with a disk in drive 0 and the machine boots itself. RESET arms the PROM at address 0000, where it shadows the bottom of memory; the PROM reads the first sector off the disk, and the moment the loaded code runs, the shadow falls away and CP/M comes up. There is a Tarbell example in `examples/` with a disk in the drive: run it and you land at `A>` with no monitor in between. With no disk in the drive the PROM has nothing to load and simply halts — put a disk in and reset.

The `bootstrap` switch turns the PROM off, leaving a plain disk controller for a machine that boots some other way. Four drives, selected by the software; the disks are 8″ single-density, 128-byte sectors, and the card recognises them by size when you mount one.

`tarbelldd` — Tarbell #2022 double-density floppy

The **#2022** (1979-80) is the #1011's twin with a **Western Digital FD1791**, which reads **double-density** as well as single. It boots exactly the same way — the same automatic boot PROM, the same `F8` ports — from a **mixed-density** disk: the first track is single density (so the boot PROM can read it), and the rest are double density. The same Tarbell example folder carries a double-density machine that boots CP/M 2.2 off one to `A>`.

Everything the `tarbell` card does, this one does; it adds only the second density and a disk format that carries more per track. Choose it when your disk is a double-density Tarbell image; choose `tarbell` for a single-density one. The card tells the two apart by the size of the image you mount.

`icom` — iCOM FD3712 / FD3812 8″ floppy

An **8″ floppy controller** of a different kind: not a bit-shifting floppy card but a **command/handshake** one, like the Datakeeper — it buffers a whole sector and the CPU moves bytes through two ports (default `C0`–`C1`), while the operating system's disk driver lives in a **boot PROM** up in high memory. One board covers both iCOM generations: the single-density **FD3712** and the double-density **FD3812**, whose disk is mixed density (a single-density track 0, then double-density tracks). The card tells them apart by the size of the image you mount.

Which system it boots is set by its PROM, chosen with `rom`:

- **`builtin:icom-fd3712-cpm`** (the default) boots **CP/M 2.2** single density from the PROM at `F000`.
- **`builtin:icom-fd3812-cpm`** boots **CP/M 2.23** double density, also at `F000`.
- **`builtin:icom-fd3712-fdos`** boots iCOM's own **FDOS** disk operating system from the PROM at `C000` — not CP/M, but iCOM's `!`-prompt executive, with its own `LIST`, `EDIT`, `ASMB` and the rest.

`altairsim icom` is CP/M 2.2 the moment a disk is in it. The `examples/` folder carries ready-made machines for all three — single- and double-density CP/M and FDOS-III — image and all: start one and you land at the prompt, read/write, with the guest saving to the disk. Read and write of existing disks work; like the other controllers here, it will not lay down a **blank** format from nothing.

`dualsd` — S100Computers Dual SD

A **modern** disk controller among the period ones: the S100Computers Dual SD board puts **two microSD cards** on the S-100 bus as raw 512-byte-sector drives, so a Z80 machine runs **CP/M 3** off flash. Like the iCOM and the Datakeeper it is a **command-and-handshake** card — two ports (default `80`–`81`) and a byte-at-a-time protocol to an onboard microcontroller that does the actual card I/O — not a floppy-shift card. Each SD card is one CP/M drive, and the drive letter comes from the **socket** it sits in: socket 1 is A:, socket 2 is B:.

It has no boot PROM — the CPU board's monitor boots it. That is why `altairsim dualsd` is the three boards together: a `z80` for the processor, the `v2z80rom` monitor EEPROM whose `I` command reads CP/M 3 off the card, and this controller. Bring the machine up, type `I` at the `->` monitor prompt, and CP/M 3 signs on at `A>`.

Mount a card in both sockets. The CP/M 3 boot ROM checks that each socket has a card before it will come up — a card in socket 1 alone boots the loader and then stops. So the example fits the bootable system card in socket 1 and a blank spare in socket 2.

A card is a raw **`.img` with a sibling `.geo`** file beside it that declares the card's true size. The `.img` can be **shorter** than the card — just the live filesystem — and every sector past its end reads back as an erased card would (all `FF`), which is why a card that would be hundreds of megabytes on real flash ships here as a couple of megabytes. `MOUNT … CREATE` creates a **blank** data card (an empty `.img` and its `.geo`) for the guest to format; a *bootable* card, like the other controllers here, has to come from a real image — its system tracks cannot be invented. See `examples/dualsd/`, which boots CP/M 3 with the host bridge fitted so `R`/`W`/`HDIR` move files to and from your host at the `A>` prompt.

`dualide` — S100Computers IDE-AB (CompactFlash)

The other half of the same physical S100Computers card. Where the Dual SD engine drives microSD, the **IDE-AB** side drives **two CompactFlash cards** through an 8255 parallel port wired to a CF card's IDE bus (default ports `30`–`34`). It presents its two cards as CP/M drives A: and B:, boots the same **CP/M 3** off flash, and — because it lays a card out byte-for-byte the way the SD side does — **a card image is interchangeable between the two**.

It has no boot PROM either. `altairsim dualide` is the `z80`, the `v2z80rom` monitor EEPROM, and this controller; at the `->` monitor prompt the **`P` command** (rather than the Dual SD's `I`) reads CP/M 3 off CompactFlash drive 0, and CP/M signs on at `A>`.

Cards behave exactly as on the Dual SD board: a raw **`.img` with a sibling `.geo`**, truncatable to the live filesystem with `FF` past its end; `MOUNT … CREATE` creates a blank data card, while a bootable one has to come from a real image. See `examples/dualide/` for the CompactFlash-only machine, and `examples/dualidesd/` for the **whole combination board** at once — CompactFlash as A:/B: and microSD as C:/D:, one CP/M 3 system spanning all four drives.

Video and display

Boards that put a picture on a display window.

`vdm1` — Processor Technology VDM-1

Memory-mapped video, and the first board here that is not a MITS one. A 1K screen of **16 rows by 64 columns** lives in the machine's own address space — by default at `CC00` — so a program puts a character on the screen by *storing a byte*, with no port and no driver. That is why it is fast enough to be worth having, and why it needs no `CONNECT`: the screen is memory.

One port (default `CC`) does the rest: writing it sets which row is at the top, which is how the VDM-1 scrolls — the text does not move, the *window* does. Reading it gives back two timing bits the software uses to avoid writing while the beam is in the way.

It needs a display. Built with SDL3, it opens a real window; built without, it runs headless and everything else still works — a program writing to the screen simply has nowhere to show it.

Closing that window stops the machine, it does not quit the simulator. The close box is the operator talking, so it does what `ATTN` does: the guest stops at an instruction boundary and you get the monitor prompt back, with the machine exactly where it was. `RUN` resumes it into the same window; `QUIT` is still how you leave.

Which window has your keyboard is yours to say. By default the terminal keeps it: the video window opens behind whatever you were doing, and when the guest stops you can type at `altairsim>` straight away. You can still click the window and type into it — keys typed there and keys typed at the terminal reach the guest as one stream — but the next time the guest stops, the keyboard goes back to the terminal.

That is the right default for a machine you drive from the monitor, and the wrong one for a Sol-20, where the window *is* the console. So:

```
altairsim> SET DISPLAY focus=on
```

and the window comes to the front when it opens and keeps the keyboard when the guest stops. `SHOW DISPLAY` says which way it is set, and a machine file can ask for it directly:

```
[display]
focus = true
```

It is a setting of the **display**, not of this board — a machine with two video boards still has one operator with one keyboard — so it reads the same whichever board is drawing. Setting it says what should happen from now on; it does not go back and re-focus a window that is already open.

Bit 7 of each byte is the **cursor/blink** flag rather than part of the character, so the board draws 128 glyphs from a real character ROM, not 256.

Two machines fit one: **`vdm1`**, which is an Altair with a VDM-1 and a demo that draws on it, and **`cuter`**, which runs the period CUTER monitor with its own built-in VDM-1 driver.

How big the window opens — the `width` property

Every video board — the VDM-1, the Dazzler, the VDB-8024 — carries a **`width`** property that sets how wide its window opens, in pixels. `auto` (the default) opens the window about **half the screen wide**; a number like `width = 1024` asks for that many pixels. The **height follows the board's own aspect** — you set width, height comes with it — and in the crisp default the picture is drawn at a whole-number multiple of the board's pixels so a 1970s frame stays a crisp grid rather than a blur (the leftover is a thin dark border). Under the period tube look (`[display] crt = true`) the window instead fills exactly the width you asked for, since the soft raster has no square pixels to keep sharp. A width that would run off the screen is brought down to fit.

The built-in `terminal` window (see the Serial ports chapter) sizes itself the same way, through a `width=` option on its connect string rather than a board property.

`width` lives on the board, not on `[display]`, because on real hardware each board has its own video-out and could drive its own monitor — and the simulator matches that: **each video board opens its own window**, sized by its own `width`. Fit a VDM-1 and a Dazzler in one machine and you get two pictures on two windows, just as the real cards would drive two monitors. The keyboard and focus stay shared, though — one operator, one keyboard (see `[display]` in the Configuring chapter), so whichever window you type into, the keys reach the machine.

A stopped machine's window is responsive, but it does not redraw

The window stays live even when the machine is stopped: you can move it, and its close button works — clicking it at the monitor prompt closes the window (`RUN` opens a fresh one the next time a program draws). What a stopped machine does **not** do is *repaint*, because drawing is the running machine's job. Two consequences, and they are the same fact:

- **A change like `SET vdm0 video=reverse` does not appear** until you `RUN` again — the setting is remembered, but nothing redraws the screen to show it until the machine does.
- **The cursor does not blink** while the machine is stopped. To watch it blink, use `sol20`, whose SOLOS sits in a loop rather than halting.

`vdm1`'s demo halts once it has drawn its banner — that is the point — so the window you are left looking at belongs to a stopped machine: still there, still closeable, just not redrawing.

Closing the window of a **running** machine is not the same as quitting: it stops the guest and gives you the monitor prompt, leaving the machine exactly where it was and the window on screen. `RUN` goes back into it; `QUIT` exits.

`dazzler` — Cromemco Dazzler

The **first color-graphics card for the S-100 bus**, and the second video board here that is not a MITS one. Where the VDM-1 paints text, the Dazzler paints a **picture**, and it does it the same clever way: out of a **framebuffer in the machine's own RAM**. You point the board at a 512-byte or 2 KB block anywhere on a 512-byte boundary and it scans that memory onto the screen — so a program draws by *storing bytes*, with no port in the inner loop.

Two ports (default `0E`/`0F`) set the rest: on/off and where the framebuffer is, then the format — resolution, size and color. Four modes fall out of it: **32×32 or 64×64** color or grey elements, and **64×64 or 128×128** on/off elements, in **16 colors** or 16 greys. Small numbers — but this was 1976, and it was in color.

It needs a display, and like the VDM-1 it draws into it: an SDL3 build opens a window, a headless build runs and simply has nowhere to show the picture. The Dazzler example in `examples/` comes up running

Li-Chen Wang's Kaleidoscope, a four-way-mirrored pattern turning over in the window (`ATTN` breaks back to the monitor); the `dazzler` machine is the bare board to build on. Because a 64×64 frame is tiny, the board's `width` property (above) sizes the window up to land near a VDM-1's size on your screen rather than a sixth of it.

`vdb8024` — SD Systems VDB-8024

An **80-column by 24-line video terminal on one board** — the SD Systems answer to a serial console. Where the `sbc` card gives an SBC-100/200 a serial port for a teletype or a glass terminal, the VDB-8024 gives it *the terminal itself*: a whole intelligent display, keyboard and all, plugged straight into the backplane.

Despite the name, it is not memory-mapped. Nothing of its screen lives in the machine's address space. To the computer it is simply **two I/O ports** — a status port and a data port, at `00` and `01` — that behave like a terminal on a wire: the program reads the status to see whether a key is waiting or the display is ready, writes a character or a control code to the data port, and reads a typed key back from it. The screen, its memory and its own processor all sit behind that pair of ports, invisible, exactly as they were on the real card. The real card's pair was wired at `00`, not jumpered; the board here still carries a `port` so you can move it, the same liberty the `sol` takes below.

It runs the **SD monitor's video build, `sdmonv21`**, which is the same monitor as the serial `sbc200` machine (same commands, same `.` prompt) built to talk to this board instead of the 8251. There is **no "press Enter first"** here — the VDB is not a serial line with a speed to measure, so the prompt is on the screen the moment the machine starts:

```
altairsim sbc200v
```

comes up at the monitor's `.` prompt **on the video display**, and the SD Systems example in `examples/` carries the same machine as a file you can read and change.

It needs a display, like the VDM-1: built with SDL3 it opens a real window in the board's own character font; built without, it runs headless and the text simply has nowhere to show. And like a Sol-20, **the window is the console** — the machine asks for `focus=on`, so the window keeps the keyboard while the guest runs, and window keys and terminal keys reach the monitor as one stream. The characters are drawn from the board's own character-generator font, with true lower-case descenders on `g`, `j`, `p`, `q` and `y`, the way the hardware's socketed font PROM did it.

The board understands the control codes its firmware did: carriage return, line feed, backspace, tab, cursor up and right, clear-screen and home, and the `ESC` sequences that position the cursor and erase to the end of a line or the screen. A line that fills wraps, and a line feed at the bottom scrolls the page up.

`sol` — Processor Technology Sol-PC

The **Sol-20's onboard I/O, as one card** — because on a real Sol-20 that is what it was. The Sol was not an Altair with cards in it; it was an integrated machine whose serial port, keyboard, parallel port and cassette interface were all on the one processor board, at `F8`–`FE`. On the real hardware those addresses were wired, not jumpered; here the board still carries a `base` so you can move it, which is the one liberty taken and the reference chapter records it.

So this board carries four things at once, and you reach them as units: `serial`, `printer` and `keyboard` are lines you `CONNECT`, and `tape1`/`tape2` are cassette transports you `MOUNT`. The keyboard is connected to the console by default, so it simply takes what you type — from the display window when there is one, and from your terminal when there is not.

The keyboard's special keys

The Sol's keyboard is not a subset of a modern one. Eight of its keys send codes with no ASCII equivalent at all, and they are how you drive SOLOS and the screen:

Key	Sends	What it does	Press	Or type
`←`	`81`	Cursor left one	←	Ctrl-A
`→`	`93`	Cursor right one	→	Ctrl-S
`↑`	`97`	Cursor up one	↑	Ctrl-W
`↓`	`9A`	Cursor down one	↓	Ctrl-Z
`HOME CURSOR`	`8E`	Cursor to the top left, screen untouched	Home	Ctrl-N
`MODE SELECT`	`80`	Return to the command mode, restarting the command line	F1	Ctrl-@
`CLEAR`	`8B`	Erase the screen, cursor home	F2	Ctrl-K
`LOAD`	`8C`	Nothing — neither SOLOS nor CONSOL ever claimed it	F3	—

The `Press` column works in the video window only. Your keyboard's own arrows and Home, and the function keys F1–F3, send these codes when the window has focus. They cannot work from a terminal: there an arrow or function key sends an escape sequence rather than a single byte, and `ESC` is a character the guest legitimately needs, so there is nothing to safely match on. From a terminal, use the last column.

A PC keyboard has nothing that reads as `MODE SELECT`, `CLEAR` or `LOAD`, so the function keys stand in for them: **F1** is `MODE SELECT`, **F2** is `CLEAR`, **F3** is `LOAD`. On a Mac the function keys reach the window only if the system is set to send F1, F2, … as plain function keys (otherwise hold `fn`).

That last column is not a hack — it is how the hardware was built. Each special key's code is exactly `80` plus the control code for the same action, because SOLOS's display driver masks the top bit off everything it is handed before it looks the character up. So `CLEAR` and Ctrl-K arrive at one routine, `HOME CURSOR` and Ctrl-N at another. The command-mode reader does the same masking, which is why a NUL byte is `MODE SELECT`: type Ctrl-@ (Ctrl-Space on many keyboards) and SOLOS abandons the line you were typing and gives you a fresh prompt.

The console is 8-bit clean, so if you have some way of sending the byte itself — a paste, a script, a terminal macro — the real code works too, and behaves identically.

One register (`FA`) reports the state of *all* of them at once — and it does so with **mixed polarity**, the keyboard and parallel bits reading active-low while the tape bits read active-high. That is not a bug in the board or in this simulator; it is what the hardware did, and the period software inverts what it needs.

The cassette decks

The Sol's cassette interface is on the motherboard, not a card, and it has **two decks** — `tape1` and `tape2` — where the 88-ACR has one. Everything in the tapes chapter applies to them; only the unit name changes, and you must always name the deck (a bare `WIND sol0` is refused rather than guessing which tape to move):

```
altairsim> MOUNT sol0:tape1 "mytape.tap"
altairsim> SET  sol0:tape1 mode=record
altairsim> REW  sol0:tape1
```

Three things differ from the ACR, and each is the hardware talking:

- **The Sol can work the motors.** `OUT 0FAh` starts and stops each transport, and SOLOS does it for you — `SAVE` spins the deck up, writes, and spins it down. So a Sol tape plays only while the guest is running it, and a deck whose motor is off yields nothing at all rather than merely nothing yet. That dropped motor line is a *stop*, so it writes a recording out — something the 88-ACR, which cannot see the deck at all, has no equivalent of. It still cannot **wind** the tape on its own: a motor line only says *turn*, not which way, so `WIND`/`REWIND` is your finger here too.
- **The speed is the guest's.** The ACR's 300 baud is soldered; the Sol's cassette runs at 300 or 1200 and `OUT 0FAh` bit D5 picks, at run time. SOLOS's `SE TA` command is that bit.
- **The modulation is CUTS, not the ACR's FSK.** A Sol tape is `cuts1200` (1200/600 Hz) at 1200 baud, an octave below the ACR's 2400/1850 Hz; it also reads Kansas City (`kcs300`, 2400/1200). This is why a Sol tape mounted on an 88-ACR is refused — the tapes chapter has that story.

Once a tape is in, SOLOS's own commands work:

```
>SA MYPROG 0100 01FF        (save memory to the tape)
>GE MYPROG                  (find it again and load it)
>CA                         (catalog what is on the tape)
```

Writing audio a real Sol will load takes more than the right tones — it takes the right *shape* and level. A genuine Sol CUTS modem is a flip-flop dividing a master clock into a square, rounded by an RC network, recorded at a modest level. Three properties reproduce that when the board writes a `.WAV` back, and their defaults are measured off a real archived dub:

Property	Default	What it does
`leader`	`3`	Seconds of steady tone before the data — the real dub carries 3.05 s (the ACR's is longer, 15 s).
`trailer`	`2`	Seconds of tone after it — the dub's measured 1.93 s.
`rc`	`4000`	Edge-rounding low-pass corner, Hz. Rounds the square's edges the way the modem's RC network and the cassette's own bandwidth do, so the tone curves like a real dub instead of sitting on a flat top. CUTS only.

`level` (percent of full scale, default `36`) matters on both boards and is covered in the tapes chapter; the point of these Sol defaults together is that a CUTS tape the simulator writes lays its tones on the same clock grid a real Sol expects, at a real dub's level — built to load on the hardware, not only to read back here.

Fit it with a `vdm1` and you have the **`sol20`** machine, which cold-starts the real SOLOS operating system.

Interrupts and the clock

Vectored interrupts and a real-time clock, on one board.

`virtc` — MITS 88-VI/RTC

Two things on one board, which is why it has an awkward name.

Vectored interrupts. Eight lines, **VI0 through VI7**. A device is strapped to a line; when it interrupts, **level *n* becomes `RST n`** — the processor jumps to `8×n` and the right handler runs without anybody having to poll anything. **VI0 is the highest priority**, and the board enforces that: a lower level cannot interrupt a higher one that is being serviced.

This is what turns a machine that busy-waits into a machine that gets on with something else. Every board with an interrupt strap in its properties — the serial boards, the floppy controllers — is strapped to one of these lines, or to `int`, or to nothing at all.

A real-time clock, on the same board: a periodic interrupt off the 60 Hz line or off the system clock, divided down.

One port at `FE`, and it is **write-only**. There is nothing to read back. Interrupt boards are the easiest thing in a machine to get subtly wrong, and this one is worth reading the reference for before you strap anything to it.

`ps2int` is the machine that shows it working — with a MITS Programming System II tape **you supply**, since none is in the package. Its cassette deck comes up empty.

`ss1` — CompuPro System Support 1

A **multifunction** board: on the real card, one 16-port block holds a serial channel, an interval timer, two interrupt controllers, a battery-backed **real-time clock/calendar**, and a socket for a math coprocessor. This board is its **clock**, its **serial channel**, its **interval timer** and its **dual interrupt controllers** — everything but the (empty) math-chip socket.

The clock is the OKI MSM5832. It comes up reading **your host's own date and time**, so a guest program that reads it gets the real wall clock for free. A guest can also **set** it, and once set it is battery-backed — the time survives a RESET, exactly as the real chip's battery does. The block sits at base **`50H`** by default (the CompuPro convention), movable with the `base` property.

The serial channel is a **2651 UART** at `5C`–`5F`. It is a spare serial port — a guest programs its rate, frame and RS-232 handshaking through the mode and command registers — that you point at something with `CONNECT ss1:serial <endpoint>`. Its `baud`, `interrupt` and `connect` are unit settings, shown under `SHOW ss1`.

The interval timer is an **Intel 8253** at `54`–`57`: three independent 16-bit counters a guest can program as timers or square-wave/rate generators. The counters tick at 2 MHz. Their outputs feed the interrupt controllers. `SHOW ss1` prints a live `timer` line with each counter's mode, count and output.

The interrupt system is two **Intel 8259A** controllers at `50`–`53`, cascaded master and slave. The master watches the eight vectored-interrupt lines of the bus and drives the CPU's interrupt pin; the slave gathers the on-board sources — the three timer outputs and the UART's transmit- and receive-ready signals — and feeds them up through the master. A guest programs them the usual 8259A way (the ICW/OCW words), unmasks the sources it wants, and on an interrupt the controller hands the CPU a `CALL` to the vector for the winning source. `SHOW ss1` prints a live `pic` line with each controller's request, in-service and mask registers. Because the master is the machine's interrupt priority encoder, do not also fit an `88vi` — the two would fight over the acknowledge.

`compupro` is the machine that fits one: a stock Altair with the System Support 1 added, its console still on the 2SIO. The clock lives at `5A` (command) / `5B` (data); the digit map, the read/set sequences and the register layouts are in the board's reference.

The whole machine

The front panel, and the turnkey board that stands in for it.

`fp` — the front panel

The sense switches. The panel is **a board**, because on a real Altair it was one — it plugged into the bus like everything else, and a machine without it is a machine you cannot toggle a bootstrap into.

The SENSE switches, at port `FF`

The eight switches on the left of the address bank, `SA8`–`SA15`. **`IN FFH` reads them.** They are **read-only**: an `OUT FF` is not this board's business and goes nowhere at all.

`SA15` is the **top** bit of the byte the program reads and `SA8` is the bottom, so the switches line up left to right the way they sit on the panel:

```
switch   SA15  SA14  SA13  SA12  SA11  SA10   SA9   SA8
bit         7     6     5     4     3     2     1     0
value    0x80  0x40  0x20  0x10  0x08  0x04  0x02  0x01
```

That is what makes a period boot procedure followable. "Raise A15 and A11" is `0x80` plus `0x08` — or, written so it looks like the panel, `0b10001000`.

They are not decoration. Period bootstraps read the sense switches to decide **what to boot from** — which device, at which port, at which speed. That is why every tape machine in this package sets one:

```
sense = 0x80        # basic4k: load from the 88-SIO at port 00
sense = 0b10000000  #          the same eight switches, drawn
sense = 0x8E        # ps2:     the 2SIO, and interrupts off
sense = 0b10001110  #          again, one digit per switch
```

Binary is nothing special to this board — `0b` works anywhere a number does, at the prompt and in a machine file alike, and the *Monitor* document's number rule has the whole list of prefixes. It is just that eight switches would rather be eight digits than two hex ones. Whichever way you write it, `SHOW fp` reads the byte back in hex.

Get it wrong and the loader sits there reading a device that is not there. It will not tell you. It has no way to.

`sense` is a **board property**, because the switches are on the panel. There is no machine-level `sense`, and asking for one is an error that says so.

`turnkey` — the MITS 8800bt, on one card

The 8800b "turnkey" system had **no front panel**. One board — the Systems Turnkey Module — did the panel's job and more: it carried the boot PROM, the terminal serial port, the sense switches, and a circuit that booted the machine the moment you switched it on. This board is that card, so a `turnkey` machine has **no `fp` and no separate `2sio`** — all three live here. `altairsim turnkey` is the bare machine. Give it a disk and one of the two boot loaders below — `start = "FF00"` for the floppy, `FC00` for the hard disk — and CP/M comes up the moment it powers on.

It boots itself

There is no front panel to toggle a bootstrap in from, so the card has an **Auto-Start** circuit. `RUN 0000` starts the processor at address 0, and the card jams a `JMP` onto the bus so the first thing that runs is the boot PROM — exactly what pressing the panel's START switch did. The `start` property is the START ADDR switches: `FF00` runs the floppy loader, `FC00` the hard-disk loader. A `startup = ["RUN 0000"]` in the machine file makes it happen at launch.

The boot PROM gets out of the way

The PROM sits at `FC00`–`FFFF` and **shadows the RAM underneath it for reads** — until the first `IN` from port `FE` or `FF`, when it switches itself out and the machine has the **full 64 KB** of RAM. That is why this machine has 64K where a front-panel Altair stops at 56K: the PROM is not permanently taking up the top of memory, it is a boot device that steps aside. The same `IN FF` that reads the sense switches is what triggers it — which is exactly how period software (Altair BASIC, the CP/M loaders) frees the whole 64K without knowing the trick.

The console and the sense switches

The serial console is the `tty` unit, at port `10h`, and it behaves like the A channel of a `2sio` — so the same software drives it. The sense switches answer port `FF`, as on the front panel. Because this card owns `FF`, **do not put an `fp` in a `turnkey` machine**: they would both try to answer the port.

The PROM sockets are a list, like a memory card's regions:

```
[[board.socket]]
at    = 0xFC00      # socket L1 — the hard-disk loader
mount = "builtin:hdbl"
```

Host integration

Not period hardware — a board that bridges to your host.

`hostbridge` — ours, not a period card

This card never existed. It is the one thing in the machine that is not history, and the manual says so plainly rather than letting you discover it in a museum catalogue.

It moves **files between the guest and your host**, in both directions, and it is **sandboxed**: the guest sees one directory you choose and **cannot escape it**. Not by `..`, not by an absolute path, not at all. That is a hard requirement, not a setting with a default.

Default port `B0`. **Nothing of the utilities is in the board.** `R`, `W` and `HDIR` are ordinary CP/M `.COM` programs — they live on a disk, they run at the `A>` prompt, and they talk to the board through its two ports exactly as any other CP/M program would. What is unusual is only where they came from: they were assembled *inside the machine*, by the machine's own assembler, from 8080 source that ships with it. So they are readable code rather than a magic trick the simulator performs on your behalf.

The file-transfer chapter is where this board is explained. It is the fastest way to get your own code into CP/M, and it beats XMODEM by a distance — but XMODEM works too, over an ordinary serial line, exactly as it did.

PROM programmers

A board here does not run software — it *burns* it. You load a period EPROM-programmer routine, point it at the board, and the bytes it "programs" pile up in a socket you can then save to a file.

`pb1` — SSM PB1

The **SSM PB1** (Solid State Music) is a 2708/2716 EPROM programmer. It puts a **4K window** in memory (default `D000`) for its two sockets — **U22** holds a 2708 (1K), **U23** a 2716 (2K) — and one **control port** (default `10`). The burn is done by *software*: an `OUT` to the control port arms the board and picks the chip (write `01` for the 2708, `02` for the 2716), and from then on every byte the guest **writes into the window** is programmed into the socketed chip. A **read** of the window disarms the board again — which is exactly how the period routines finish and turn the LED off.

Two things follow from "programming is a write, disarming is a read", and they are worth knowing before you chase a bug:

- **A socket starts erased** — every byte `FF` — and **programming can only clear bits to 0**, never set them, just like a real EPROM. So burn a *blank* chip; a second burn over the first only ANDs.
- The burn lives in the board, not on the host. To keep it, **`SAVE` it to a file** (below). A power-cycle re-reads the sockets from their mounts, so an unsaved burn is gone.

Burning an EPROM, and making a hex file

This is the whole point of the board (and the answer to "do I need a burner board, or is a bus write enough?" — you need the board, because you want to run the *software*). The example under `examples/pb1` is a stock Altair with a `pb1` in it, plus SSM's own burner routines from the PB1 manual: `PROG2708.HEX` (a 2708) and `PROG2716.HEX` (a 2716). Each copies bytes from `4000` into the socket. So put the data you want to burn at `4000`, run the burner, and save the result:

```
cd examples/pb1
altairsim pb1.toml
FILL 4000-43FF E5                ; the 1K you want to burn (or LOAD your own data there)
LOAD PROG2708.HEX                ; the SSM 2708 burner
BREAK F021                       ; where the burner "returns to the monitor"
RUN 100                          ; run it: it arms the board and burns the 2708
SAVE eprom.hex D000-D3FF         ; the burned chip, as an Intel HEX file on your host
```

The `BREAK F021` is there because SSM's routine ends by jumping to its own system monitor, which this machine does not carry — the breakpoint catches that jump once the burn is done. `SAVE` then reads the socket window back off the bus and writes it out — the socket reads like any other memory once it is programmed — so `eprom.hex` is an ordinary Intel HEX image of the chip you burned, ready to hand to another tool or `LOAD` back later. Your own burner works the same way: put your data in RAM, `OUT 10,01` (or `,02` for a 2716), write it into the window, and `SAVE`.

The board also has an optional **on-board EPROM area** (the real card's U11–U14): read-only chips you mount above `8000` with `[[board.prom]]` (`at` + `mount`), handy as the *source* for copying one EPROM to another.

Working with the backplane at the prompt

Everything a machine file can do to a board, you can do by hand.

Command	
`BOARDS`	what is in the backplane
`SHOW BOARDS`	the board types you can add
`SHOW BOARD <type>`	one type's description and its settings (add `UNITS` for just the units)
`BOARDS ADD <type> <id>`	fit a board
`BOARDS REMOVE <id>`	pull one out
`SHOW <id>`	one installed board's settings, with the legal values
`SET <id> <key>=<value>`	change one

```
altairsim> BOARDS ADD virtc vi0
altairsim> SET vi0 rtc_source=line
altairsim> SHOW vi0
```

The keys are the same keys. `SET cpu0 clock_hz=2000000` at the prompt and `clock_hz = 2000000` in a machine file are the same property reached two ways — there is no separate config schema, which is the whole reason the board reference at the back can be exhaustive.

And when you have the machine you want:

```
altairsim> CONFIG SAVE mine.toml
```

writes it out, and it round-trips.

Looking a board type over before you fit it

`SHOW BOARD <type>` reads the *catalog*. It builds one of that board, describes it, lists its settings, and throws it away — nothing is added to the backplane. A disk controller, for instance, ends by naming the units it carries:

```
altairsim> SHOW BOARD dcdd
  dcdd  MITS 88-DCDD: 8" hard-sector floppy, up to 16 drives...
  ...
  This board has units: drive
  SHOW BOARD dcdd UNITS for their properties.
```

Add `UNITS` and you get just the units — each under a heading that names its **kind** and, after it, the **verb that fills it**:

```
altairsim> SHOW BOARD sol units

  Unit 'serial'  (serial, CONNECT)
  connect    The endpoint on the other end of this line (CONNECT sets this)
  ...

  Unit 'tape1'  (tape, MOUNT)
  mode       Which way the bytes go: play loads from the file, record saves to it
  ...
```

A **serial** unit is an endpoint, so you `CONNECT` it (to `console`, a socket, a real port). A **disk**, **rom** or **tape** unit holds an image, so you `MOUNT` a file into it. A **cpu** unit is soldered on — neither — and shows only its kind. The heading tells you which verb a unit takes without your having to fit the board and find out.

Where a unit is filled from a *machine file* rather than by hand, `UNITS` shows the TOML table and its keys instead — `[[board.drive]]` for a disk controller, with its `mount`, `readonly` and `media` keys — so the file form is as discoverable as the prompt form.

Contention

Two boards decoding the same port is **contention**, and it is a real thing that real backplanes did. Both boards answer, both drive the data bus, and what the processor reads is neither.

The simulator does not stop you. It does something more useful — **it tells you**:

```
altairsim> SHOW BUS CONTENTION
```

which names the port and names both boards. Fit an `mds` in a machine that already has a `dcdd` and this is what you will see, and it is a great deal more helpful than a guest that has mysteriously gone mad.

Being able to build a machine that does not work is not a defect. **It is the point** — this is a bench for developing hardware, and hardware that cannot be wired up wrong cannot be wired up at all.

Disks

A disk on an Altair is really three things: a **controller** board on the bus, some number of **drives** hanging off it, and a **disk image** sitting in one of those drives. The three are separate, and the manual keeps them separate, because the machine did.

This chapter is about the last two — the **image** and the **drive it goes in** — and the `MOUNT` workflow that puts one in the other. That workflow is the same whichever controller you have. **`altairsim` has many disk controllers** — the **controllers themselves** are boards, one per section in the Boards chapter; this chapter uses just the MITS hard-sector pair (`dcdd` and `mds`) for its examples, because that is what the shipped machines boot from.

The controller is a board. It goes in the machine file. The drives are part of the controller — a MITS 88-DCDD addresses up to sixteen of them, whether or not you own sixteen. The disk goes in a drive, and you may put it there either way: name it in the machine file, or `MOUNT` it at the prompt. They do the same thing.

> **A disk is not a cassette, and the difference is real.** A machine file *may* name the floppy that is in drive 0 — the CP/M example does exactly that — but it may never name the tape in the recorder. A floppy drive is wired to its controller and the guest can tell what is in it. **A cassette recorder has no motor control on the card**: nothing in the machine can start the tape, sense it, or know it is there. So the tape is something a human puts in and presses PLAY on, and it stays at the prompt where humans are. See the tapes chapter.

The controllers this chapter uses

`altairsim` has many disk controllers — the two MITS hard-sector boards below, plus the Datakeeper hard disk (`hdsk`), the SD Systems VersaFloppy (`versafloppy`), the Tarbell single- and double-density boards (`tarbell`, `tarbelldd`), the iCOM 8″ (`icom`), and the S100Computers CompactFlash and SD boards (`dualide`, `dualsd`). Each has its own section in the **Boards** chapter, which is where the hardware detail lives.

This chapter uses just the two the shipped machines boot from — the MITS hard-sector pair — because everything it teaches about the *image* and the `MOUNT` workflow is the same on all of them:

Board	What it is	Drives	Ports
`dcdd`	**MITS 88-DCDD —** the 8″ hard-sector floppy controller. The one CP/M booted from.	up to 16	08, 09, 0A
`mds`	**88-MDS —** the 5¼″ minidisk. Smaller, slower, four drives.	4	08, 09, 0A

Read the ports column again. **They are the same ports**, which means these two boards cannot coexist in one machine. That is not a limitation of the simulator; it is the MITS address map. A real Altair with both would have had two cards decoding 08 and fighting on the data bus, and the bus view in the monitor will tell you so if you try it. Pick one.

Drives are units on the board: `drive0`, `drive1`, and so on up to the board's limit.

Putting a disk in — `MOUNT`

```
MOUNT <id>[:<unit>] <file> [WP] [CREATE]
```

```
altairsim> MOUNT dsk0:drive1 games.dsk
```

That is a floppy going into drive 1 of the controller called `dsk0`. The socket was empty before; it isn't now.

The file has to be there already. A name that does not exist is not a new disk, it is a typo, and you are told so rather than handed a disk you did not mean to make:

```
altairsim> MOUNT dsk0:drive1 my-scratch.dsk
dsk0: 'my-scratch.dsk': no such file
dsk0: to make a blank one, add CREATE: MOUNT dsk0:drive1 my-scratch.dsk CREATE
```

`CREATE` is how you make one on purpose, and "Making a scratch disk" below is what to do with it once you have.

An `.imd` is converted on the way in

`MOUNT` on a file whose name ends `.imd` (an **ImageDisk** file, the format much archived 8″ and 5¼″ software is distributed in) does not mount the `.imd` itself. It **reads it, converts it to a raw sector image, writes that beside it as `foo.dsk`, and mounts the `.dsk`** — because a controller reads raw sectors, not ImageDisk's track records. The conversion is reported so you can check it against the disk you expected:

```
altairsim> MOUNT dsk0:drive0 cpm.imd
dsk0: converted cpm.imd -> cpm.dsk
dsk0:   IMD: CP/M 2.2 system disk
dsk0:   337568 bytes
dsk0:drive0: mounted cpm.dsk
```

It **never overwrites a `.dsk` that is already there** — the same caution as `CREATE`. If `cpm.dsk` already exists you are told to remove it or mount it directly, so a re-mount cannot silently discard edits you made to the converted disk. From then on the `.dsk` is the disk; the `.imd` is just where it came from.

`WP` **write-protects the disk**, and it does what a real diskette's write-protect notch did: the guest may read the disk, and a write is refused at the controller and never reaches the file on your host.

A real 8" or 5.25" diskette had no movable tab — a notch in the jacket was either covered or it was not, and the drive's photocell read it. (The two sizes read that notch in *opposite* senses, which is a fact about the drive, not about the disk, and nothing here depends on it: a disk is either protected or it is not.)

```
altairsim> MOUNT dsk0:drive2 golden-master.dsk WP
```

`RO` is accepted and means exactly the same thing. It is the right word for a ROM socket, which is read-only because of what it is — for a floppy, `WP` is the thing you are actually doing.

The guest is not told, and cannot be — see "Read-only is not an error message" below. Mount `WP` for a disk you intend to read.

A protected disk says so wherever it is listed, so you never have to remember which one you did it to:

```
altairsim> SHOW MOUNTS
  UNIT         KIND  HOLDS
  dsk0:drive2  disk  golden-master.dsk  (write-protected)
```

And if the **host** would not let us write the file — the permissions say no — the disk still mounts, protected, and says that it was not your idea:

```
  dsk0:drive2  disk  master.dsk  (write-protected -- THE HOST WON'T LET US WRITE IT; you did not ask for this)
```

That one is worth reading closely. You did not ask for the protection, so CP/M is about to bounce every write off a disk you believe is writable.

And taking it out:

```
altairsim> UNMOUNT dsk0:drive1
```

The socket is empty again. The guest sees a drive with no disk in it, which is a thing a real drive could be.

Names, and what you may leave out

Board names are **case-blind everywhere** — `dsk0`, `DSK0` and `Dsk0` are the same board.

Beyond that you may omit **what carries no information**:

- the **trailing index**, when only one board of that type is in the machine. One floppy controller means `dsk` finds `dsk0`.

- the **unit**, when the board has only one thing you could possibly mount into. `MOUNT ACR tape.bin` needs no unit, because a cassette recorder has one slot.

A floppy controller does not qualify for the second one. It has several drives — four by default, and up to sixteen on an 88-DCDD — and which drive you meant is real information, so you must say it. **Anything genuinely plural, you must name.**

…or put it in the machine file

`MOUNT` at the prompt is for the disk you are dealing with *now*. A disk that belongs to a machine belongs in the machine file, and that is how the shipped examples do it:

```
[[board]]
id = "dsk0"                   # no `type`: the controller is already there

  [[board.drive]]
  unit  = 0
  mount = "cpm22b23-56k.dsk"  # relative to THIS FILE
  # readonly = true           # refuse every write; your file cannot change
  # create   = true           # make it blank if it isn't there -- CREATE, in a file
  # media    = "8in"          # what a blank one is; see "The geometry is probed" below
```

`writeprotect = true` says the same thing, and is there because everything *else* here calls it write-protect — `MOUNT` takes `WP`, `SHOW MOUNTS` prints `(write-protected)`, and on the real diskette it was a notch. `readonly` is the spelling a machine file is saved with.

The two forms do the same thing. The difference is only *when*: one is the drive as the machine ships, the other is you, at the prompt, changing your mind.

Note the path. It names the file lying **beside the machine file**, with no directory at all — which is what lets the whole folder be copied somewhere else and still boot. A path inside a machine file is resolved against **that file**; a path you type is resolved against **your shell**. The machines chapter has the rest of that rule.

…or fetch it over the network

Everything so far assumed the image is a file on your host. It need not be. Point `MOUNT` at a **TNFS server** — the network file system the FujiNet project speaks — and the disk is fetched across the network instead of read off your disk:

```
altairsim> MOUNT dsk0:drive0 tnfs://fileserver/cpm/games.dsk
altairsim> MOUNT dsk0:drive1 tnfs://fileserver:16384/scratch.dsk WP
```

The form is `tnfs://<host>[:<port>]/<path>`. The port may be left off and defaults to **16384**, the TNFS port; the path is where the image lives on the server. A machine file may name one the same way — `mount = "tnfs://fileserver/cpm/games.dsk"` on the drive — and because a `tnfs://` name carries its own server with it, it is **not** re-based against the machine file or your shell the way a bare filename is. It means the same thing wherever you write it.

What happens after that is what happens with any disk. The image is pulled in **once, at mount**, and from then on it behaves exactly like a local one: the guest reads and writes it, the geometry is probed from its size (below), `SHOW MOUNTS` lists it, and your changes are written back to the server when you `UNMOUNT` or the guest flushes. `WP` write-protects it just as it does a local disk, and if the **server** will not let us write — the file is read-only there — the disk still mounts, protected, and says the protection was not your idea, exactly as a host file whose permissions say no.

A network can go away in the middle of a session, and a local disk cannot — so this one case is worth watching for. If the server stops accepting writes while you are using the disk, altairsim **tells you**: it prints a line saying it can no longer save changes to that mount, and that they are being held in memory only. The guest keeps running, and altairsim keeps trying; when the server comes back it says so and the held changes are saved. But until then, treat those changes as not yet safe — if you quit or the server never returns, what was written after the warning is lost.

There is one limit worth knowing. TNFS carries the **single-file images** — a floppy, a minidisk, an 8 MB `fdc8mb`, a cassette tape — the ones small enough to hold whole in memory. The larger card images that keep their geometry in a companion file (the CompactFlash and SD boards' `.img`) are **not** mountable over TNFS; those stay on your host.

The geometry is probed, not declared

You do not tell `altairsim` what kind of disk you just mounted. It **looks at the file's byte count** and works it out:

Format	Tracks	Sectors	Bytes/sector	File size
`8in`	77	32	137	**337,568**
`minidisk`	35	16	137	**76,720**
`fdc8mb`	2048	32	137	**8,978,432**

A file of 337,568 bytes is an 8″ floppy. There is nothing else it could be. This is why the quick start never mentions a format: there was nothing to mention.

The `8in` and `fdc8mb` rows belong to the `dcdd`; `minidisk` is the `mds`'s one format, and the probe only ever chooses among the formats the board it is on can take.

The size in that column is the disk, and the file you have may be slightly bigger. Almost every image in circulation was moved by XMODEM at some point, which pads to a 128-byte block — the CP/M disk in this package is 337,664 bytes rather than 337,568, and the minidisk images you will find are 76,800 rather than 76,720. The probe allows that padding, so those are 8″ floppies and minidisks like any other. Do not go looking for the exact number on your own disk; it is the format's size, not a checksum.

Only 8″ floppies ship. Every disk image in the package is one — a `minidisk` or an `fdc8mb` image is one **you supply**. The `mds` board and the 8 MB medium are both here and both work; what is not here is a disk to put in them.

And a file that matches nothing at all

A blank disk is a file of no size, and a size of zero is not in the table. It mounts anyway, because refusing it would leave you no way to make a disk:

A file whose size matches no row is mounted UNFORMATTED, at the widest format the board can reach — `fdc8mb` on a `dcdd`, `minidisk` on an `mds` — and the guest's own formatter fills it in. The file **grows as it is written**, out to as far as the head can step. That is what makes `CREATE` and a period FORMAT program add up to a disk.

So the thing to know about a blank one is that it starts out as big as the controller goes. If you want a blank 8″ floppy rather than a blank 8 MB disk — because you are about to format it with something that expects 77 tracks — say so with `media`:

```
[[board.drive]]
unit   = 1
mount  = "scratch.dsk"
create = true
media  = "8in"        # not "as far as a dcdd can step"
```

`media` is a machine-file key on the drive, not something you can set at the prompt. It also settles a truncated image, or a format you are inventing, where the byte count genuinely cannot decide.

Why an 8 MB disk works at all

`fdc8mb` is a 2048-track disk on a controller MITS designed for 77 tracks, and it works through the **stock card with the stock PROM** — because the controller cannot tell the two apart. It steps the head and shifts bytes; it never asks how big the disk is. All the intelligence about *where track 1500 is* lives in the BIOS, which is software.

The useful upshot is that **format and spindle are per drive, not per controller**, so mixed geometry on one board is the intended arrangement: period 8 MB CP/M BIOSes expect an 8 MB disk on A:/B: and ordinary 77-track floppies on C:/D:, so you can `PIP` between them.

Both are images you supply — neither is in the package, so the lines below are the shape of the command, not files you have. The period 8 MB CP/M image is one you supply; the package chapter explains where the wider collection of disks and tapes will come from.

```
altairsim> MOUNT dsk0:drive0 big.dsk
altairsim> MOUNT dsk0:drive2 floppy.dsk
altairsim> SHOW dsk0
```

THE TRACK BUFFER TRAP

Read this section. It is the one thing about disks in this simulator that will cost you work if you do not know it, and it is not the simulator's doing — **it is what a BIOS may do**, and the CP/M in this package is one that does.

A track-buffering BIOS does not write to the controller when CP/M closes a file.

`BIOS WRITE` copies your data into a **track buffer in memory** — 32 sectors of 137 bytes, one whole track — marks it dirty, and returns. Nothing has touched the disk. Nothing will touch the disk until something calls the flush. And the flush is called from **`CONIN`**: the BIOS console-input routine. **Reading the console is the flush trigger.**

This is not madness. A track write is one seek and one revolution instead of thirty-two, and the moment the machine sits down to wait for a human to type something is precisely the moment it has spare time and nothing to lose. It is a very good piece of engineering. It is also a loaded gun.

Not every CP/M does this

Buffering is not a property of CP/M, and it is not a property of the controller. It is a decision the author of a **BIOS** made, and the BIOS is precisely the part of CP/M that every machine's owner rewrote for himself. Period Altair BIOSes went both ways:

- **Track-buffered.** Writes pile up in memory and reach the disk on a console read. The CP/M that ships in this package is one of these.
- **Write-through.** Every `BIOS WRITE` puts a sector on the disk before it returns, and when you get your prompt back your file is already there. Some of these hook `CONIN` too — but only to *unload the head*, which is a courtesy to the drive and touches no data.

You cannot tell which one you have by looking at the `A>` prompt, and a disk image somebody handed you arrives with whatever BIOS its author wrote on it. So treat the rule below as the rule regardless: on a write-through BIOS it costs you nothing, and on a buffered one it is the difference between having your file and not.

> **Never `UNMOUNT`, copy, or kill the program immediately after a file operation. Get back to the `A>` prompt first.**

The `A>` prompt is CP/M reading the console; the console read flushes the buffer if there is one, and the directory update lands on the disk. Once you are looking at `A>`, the disk on your host is what you think it is — under either kind of BIOS. Save the file, watch the prompt come back, *then* do whatever you were going to do.

`^E` back to the monitor from the `A>` prompt is safe. `^E` back to the monitor one instant after `ED` says it has written your source file is **not**, and on the CP/M in this package the file will not be there.

The disk is real, and there is no undo

A mounted disk is mounted **read/write**, and every write goes through to the file on your host as it happens. There is no journal and no way back — and `SNAPSHOT` will not save you either: it captures the machine's *state*, not your host files, so the disk image is not in it. A CP/M cannot save your work onto a disk it is not allowed to write, so read/write is the only default that lets the machine be a machine — and the price of it is that you can destroy the example disk with one mistyped `ERA`.

Copy the directory before you experiment. It is self-contained and boots from anywhere.

Use `RO` when you want the guest to look and not touch.

Read-only is not an error message

`RO` is a promise to **you**, about your file. It is not a message to the guest, and this is worth being exact about, because it is easy to assume otherwise.

The controller has no way to say "write protected." The 88-DCDD's status byte is seven bits — write-circuit-wants-a-byte, head-movement-OK, head-loaded, drive-enabled, interrupts-enabled, on-track-0, new-read-data — and none of them means *the notch is covered*. A program reading that byte cannot distinguish a protected disk from an ordinary one.

And CP/M has nowhere to put the answer even if it had it. A CP/M 2.2 BIOS write returns `0` for success and `1` for a non-recoverable error. That is the entire vocabulary. There is no "protected" code, which is why a genuine hardware write failure surfaces as the blunt `Bdos Err On A: Bad Sector` — CP/M is telling you the only thing the interface let it hear.

The message people remember — `Bdos Err On A: R/O` — is a **different mechanism entirely**. That is CP/M's own software read-only flag, the one `STAT A:=R/O` sets and a warm boot clears. It lives in CP/M's head, not on the disk and not in the controller, and a write-protected image will never produce it.

So: a guest that tries to write to an `RO` disk is asking for something it cannot be refused politely. **Do not mount `RO` and then expect CP/M to cope gracefully** — mount `RO` for a disk you are going to read. If you want a disk the guest can write and you can throw away, copy the directory; that is what the copy is for.

Making a scratch disk

It is two steps, not two ways, and the second one is the guest's — which is exactly how it was in 1976. **`CREATE` gets you the blank medium; a formatter makes it a disk.**

Step one, from the monitor. `CREATE` makes the file and puts it in the drive:

```
altairsim> MOUNT dsk0:drive1 my-scratch.dsk CREATE
dsk0:drive1: created my-scratch.dsk (empty)
dsk0:drive1: mounted my-scratch.dsk
```

Zero bytes, and mounted — an unformatted disk, which is a thing you can hold in your hand and a thing CP/M will refuse to read. That is the correct state for a disk nobody has formatted yet.

Step two, from inside CP/M. Boot, and format it the way the period did. The CP/M disk in this package carries **`AFORMAT`**, Eberhard's Altair Disk Format Utility, and formatting an image is formatting a disk:

```
A>AFORMAT B:

Put disk in B
Ready (Y/N)?Y
Formatting an 8" Disk.............................................................................
```

The dots are tracks. When they stop, the file on your host has grown from nothing to a formatted 8″ floppy, and `B:` is a disk CP/M will `DIR`, `PIP` to, and put files on.

> **`DIR` on the CP/M disk in this package shows one file, and the disk is nearly full.** Almost everything on it is marked `$SYS`, which is precisely the attribute that hides a file from `DIR` — `STAT *.*` is the command that lists them, and it is a long list. `AFORMAT` is one of the hidden ones, and so is `DDT`.

And if what you want is a blank **8″ floppy** rather than a blank disk as wide as the controller can step, say `media = "8in"` on the drive — see "The geometry is probed" above. `AFORMAT` will format either, but a period program that assumes 77 tracks is happier with the 77-track one.

`CREATE` only works on a controller that can grow its disk. The `dcdd` and `mds` mount a blank at the widest format they reach and let the guest's formatter fill it in — that give is what makes the empty file a disk. A **fixed-geometry** controller has none: the 88-HDSK "Datakeeper" hard disk knows exactly how big a platter is and nothing else, so it cannot make a blank one for you. `CREATE` there is refused with

```
h0: 88-HDSK has a fixed geometry and cannot make a blank platter -- supply an existing
    4988928-byte image (406 cyl x 2 sides x 24 sectors x 256)
```

and the empty file is removed, not left behind. For that disk you supply a full-size image; there is no scratch-disk step.

Either way, remember which chapter you are in: get back to `A>` before you go looking at the file.

Looking at what is in the machine

`SHOW` on the controller lists its drives and what is in each one:

```
altairsim> SHOW dsk0
```

`BOARDS` shows the backplane — every board, where it decodes, and who is fighting whom:

```
altairsim> BOARDS
```

Between them they answer nearly every "why is it not booting" question there is, and the first one is almost always *the disk is in the wrong drive*.

Tapes

Before the floppy, there was a **cassette recorder**. Not a special one — a home audio cassette deck from a department store, with a microphone jack and an earphone jack, and MITS sold you a card that turned bytes into a noise it could record and turned the noise back into bytes. That card is the **88-ACR**, and it is in this simulator.

The **Sol-20** has a cassette interface too, but a different one — built into its motherboard, with two decks, its own modulation, and motor control the guest can work. It is described with the rest of the Sol's onboard hardware in the boards chapter, under `sol`. This chapter is about the 88-ACR, and everything in it — mounting, rewinding, audio files, recording — works the same way on the Sol's decks; only the unit name changes.

The chapter ends with the thing this is all for: loading Altair 4K BASIC 3.1 the way it was actually loaded in 1976, with nothing in ROM, nothing on a disk, a bootstrap you enter by hand, and a tape.

The 88-ACR

`acr` is the **MITS 88-ACR** — an Audio Cassette Record/playback interface.

Underneath, it is **an 88-SIO channel B with an FSK modem soldered to it**. That is not a metaphor; that is what MITS built. The guest talks to a perfectly ordinary serial port, and the modem on the other side of it turns the bits into tones. Default port **06** (`0x06` status/control, `0x07` data). **300 baud**, which is the tape's speed, not a setting you picked.

It has one unit: **`tape`**. There is one slot in a cassette recorder.

You cannot `CONNECT` it to anything

The ACR takes no endpoint. It cannot be wired to your terminal, to a socket, or to a real serial port, and the reason is the same one: **the line is soldered to the modem.** There is no connector on the back of an 88-ACR because there is nothing to connect — the signal goes to a cassette deck and nowhere else. **It is a cassette interface, not a serial port**, and the fact that it is built out of a serial port does not make it one. The serial chapter is about the boards that *do* take endpoints.

It cannot work the motor

An 88-ACR cannot start the tape, stop it, or rewind it. It can only listen to whatever is going past the head. The machine genuinely does not know a tape is there, so the simulator does not pretend that it does — which is why mounting is something you type, below.

Working a tape

Everything in this section is written for the 88-ACR's one `tape` unit. It is the same on the Sol's decks — only the unit name changes (`sol0:tape1`, `sol0:tape2`), and where the two boards genuinely differ the boards chapter's `sol` section says so.

The tape is not hardware

The tape is NOT in the machine file, deliberately.

A machine file describes the **hardware** — what boards are in the backplane, what they decode, how much memory is on the bus. Which cassette is sitting in the recorder is not hardware. It is a thing you did with your hands, this morning, and you can do a different thing with your hands this afternoon without unscrewing the lid.

You put the tape in, and you press PLAY. That is `MOUNT`, and it is a thing you type.

Putting a tape in

```
altairsim> MOUNT acr0:tape "examples/basic/4K BASIC Ver 3-1.tap"
```

The ACR has one unit, so the unit carries no information, so you may drop it:

```
altairsim> MOUNT ACR tape.bin
```

A Sol has two, so you may not: name the deck.

Names are case-blind. The rules are the ones in the disks chapter, and they are the same rules.

`mode = play | record`

```
altairsim> SET acr0:tape mode=record
```

`play` loads from the file; `record` saves to it. Which way the bytes are going is the whole of the setting.

The two are **mutually exclusive** — one tape, one head, one direction at a time — so `mode` is one setting with two values, and there is no third.

Where the head is — the tape counter

`SHOW MOUNTS`, and `SHOW <id>`, report where the head is sitting as a position on the tape:

```
altairsim> SHOW MOUNTS
  acr0:tape  tape  BASIC.WAV  00:15 / 01:28 (17%)  301/2048 bytes
```

The counter is a **time and a percentage** — minutes and seconds into the tape, and how far along it you are. For a `.WAV` that time is the recording's *own*: it counts the leader and any silent gaps between programs, exactly as a real cassette counter would, so a program a manual indexes by "seconds from the start of the tape" is at the second the manual says. For a byte `.TAP`, which carries no audio, the time is estimated from the baud. The byte count is still there beside it.

`WIND` — move the head to a time

A tape unit brings a verb of its own:

```
WIND <id>:<unit> <mm:ss | START | END>
```

`WI` will do. It winds the head to a time on the tape — so a cassette holding several programs one after another is **reachable**: read the counter (or a manual) for where the next one begins, and wind there.

```
altairsim> WIND acr0:tape 2:05
acr0:tape: wound to 02:05 / 08:40 (24%) -- BASIC.WAV (...)
```

The position is a time — `mm:ss`, or a bare number of seconds — or the words `START` and `END`. A time past the end lands at the end. On the Sol you must name the deck, because there are two.

`REWIND` — wind to the start

`REWIND <id>:<unit>` (`REW`) is `WIND … START`: the common case, kept as its own verb.

You need it to load the same tape twice. A tape that has been read is a tape whose head is at the end of the tape, and playing it again gets you silence. This surprises people exactly once.

```
altairsim> REW acr0:tape
```

Watching a load — the live counter

When a tape plays in real time (`rate = real`, below) the counter **ticks up on the console while it loads**, so you can watch a long tape's progress. It is on by default; turn it off for a machine whose guest is writing to the same terminal:

```
altairsim> MOUNT acr0:tape "tape.wav" counter=off
```

or `SET acr0:tape counter=off` at any time. Turning it off changes nothing about `SHOW` — the position is always there to ask for.

`stop` — halt the tape at a time

A multi-program tape runs one program straight into the next. The `stop` mark is your finger on the recorder's **STOP button at a counter mark**: set it to a time and the tape goes quiet there instead of running on. Set it at `MOUNT`, or with `SET` any time after:

```
altairsim> MOUNT acr0:tape "tape.wav" stop=2:05
altairsim> SET  acr0:tape stop=2:05
```

Load program 1 and the line falls silent at 2:05 — the same quiet the end of the tape gives — so the loader stops there rather than reading into program 2. To carry on, move the mark forward (or clear it) and go again:

```
altairsim> SET acr0:tape stop=5:30      (the next boundary)
altairsim> SET acr0:tape stop=off       (play to the physical end)
```

It is `off` (play to the end) unless you set it. `SHOW` shows an armed mark as `stop @ 02:05`. It halts **playback only** — a recording writes straight through it — and it is independent of `WIND`: winding the head past an armed stop leaves the tape parked there (SHOW says why), so move or clear the mark to continue.

`rate = full | real`

The cassette carries its own clock, and by default it runs flat out. `rate = full` — the default — hands the guest each byte the moment it is ready to read the next one, so a tape loads in about a second whatever speed the CPU is set to. This is almost always what you want: a program you are trying to run should not make you wait for a recorder that has been gone for forty years.

```
altairsim> SET acr0:tape rate=real
```

`rate = real` paces playback in **real wall-clock time** at the tape's baud — 1200 or 300 — so a load takes as long as it took on the day. Set it when the *wait itself* is the point: a demo, a screen recording, the sound of the thing. It is playback only; recording takes as long as the guest drives it either way.

What this is *not* is the CPU clock. The tape's clock and the processor's are separate crystals on the real hardware, and they are separate here — see *Speed* below. A faster or slower CPU no longer drags the tape with it; `rate` is the only thing that governs how fast a tape plays.

Audio tapes — `.WAV`

Most surviving Altair and Sol cassettes are not files of bytes. They are **audio**: somebody put a cassette in a deck, played it into a sound card, and saved a `.WAV`. You can mount one, on either board.

There are two in the package, in the BASIC example under `examples/basic/`: `4K BASIC Ver 3-1.wav` and `BASIC Ver 1-0.wav`, the two cassette BASICs of example 2 as 88-ACR audio rather than decoded bytes. Each has a machine file beside it — `basic4k-wav.toml` and `basic1-wav.toml` — that mounts it and boots. `4K BASIC Ver 3-1.wav` is a 300-baud FSK cassette carrying 4K BASIC, so from the `basic` example this is a line you can actually type:

```
altairsim> MOUNT acr0:tape "4K BASIC Ver 3-1.wav"
acr0:tape: mounted 4K BASIC Ver 3-1.wav
4K BASIC Ver 3-1.wav: fsk300, 4439 bytes, 0 framing errors (100.0% of frames intact)
```

Nothing else changes. The recording is demodulated **once, when you mount it** — never while the machine is running — and from that moment everything above it, including `SHOW`'s byte count and `WIND`/`REWIND`, means exactly what it meant for a `.TAP`. The one thing the audio adds is a *real* clock: the tape counter reads the recording's own minutes and seconds, gaps and leader included, where a byte tape can only estimate them. The guest cannot tell.

Read that first line. A mount always says what it found, and the framing-error count is the number that matters: a tape that decoded at 60% is noise, not a program, and you want to know that now rather than after the loader has crashed. A decode below 90% is refused outright.

But it is a framing rate, not a clean bill of health. The percentage counts frames whose start and stop bits landed where they belonged; it cannot tell you the eight bits between them are the ones that were recorded. The two can come apart: a worn or poorly dubbed recording can keep its framing and still hand the loader wrong bytes. A high percentage means the demodulator kept its footing, and no more. If a tape mounts well and the program still will not run, a worn recording is the likely answer.

What decides is the file's magic, never its name. A `.TAP` somebody renamed `.WAV` is still read as bytes, and a recording renamed `.TAP` is still demodulated.

`extract` — split a WAV into per-program `.TAP` files

A cassette WAV often holds several programs one after another, separated by a few seconds of silence. **`extract` writes each program out as its own `.TAP`** — so you can keep, mount or load them one at a time instead of winding through the whole tape. Ask for it at `MOUNT`:

```
altairsim> MOUNT acr0:tape "games.wav" extract
acr0:tape: mounted games.wav
  games-1.tap  2048 bytes
  games-2.tap  3120 bytes
2 programs extracted
```

The files land **beside the WAV**, named from it: `games.wav` becomes `games-1.tap`, `games-2.tap`, … with a 1-to-N index (a single-program tape is just `games.tap`, no index). Each line prints the file's name and size. `extract=<base>` names them yourself (`extract=disk1` → `disk1-1.tap`, …). It only **reads** the tape and **writes** the files — nothing in the machine changes.

The same thing has a verb, so you can split a WAV that is already in the deck without re-mounting:

```
altairsim> EXTRACT acr0:tape          (on the Sol, name the deck: EXTRACT sol0:tape1)
```

Only a `.WAV` can be extracted — a `.TAP` is already the bytes, and has no gaps left to split on. The boundary is a second or more of silence, which is far longer than the gaps *inside* a program, so programs come apart cleanly and none is cut in half.

A board will refuse audio it could not really have heard

The package ships that same 4K BASIC in a modulation the 88-ACR *cannot* read — `4K BASIC (Kansas City).wav`, lying right beside the FSK one — precisely so you can watch the board refuse a tape it could not really have heard:

```
altairsim> MOUNT acr0:tape "4K BASIC (Kansas City).wav"
acr0: 4K BASIC (Kansas City).wav: this board's modem cannot hear that tape -- it carries 2400 Hz / 1200 Hz,
and this board reads fsk300
```

This is deliberate, and it is not the simulator being fussy.

Not all published Altair cassette audio is in the 88-ACR's modulation. The ACR uses **2400/1850 Hz FSK**; plenty of archive tapes are **Kansas City** (2400/1200 Hz) — the two share the 2400 Hz mark tone but part company on the space, 1850 against 1200 — and the Sol's own CUTS tapes are an octave lower still, **1200/600 Hz** at 1200 baud. The demodulator here measures the tones actually on the tape, so it *could* read any of them perfectly well — but a real 88-ACR could not. Its demodulator is a PLL centred at 2125

Hz with about ±100 Hz of range, and a 1200 Hz space tone is nowhere near that. A real card fed that tape does not read it badly; it reads **nothing**.

Decoding it anyway would hand your guest program data that no 88-ACR on earth could have produced. So the board says what the tape is instead, and you go find a machine that reads it.

The frequencies in that message are a measurement, not the tape's specification. All the demodulator has to go on is where the signal crosses zero; on a cleanly dubbed tape the two readings land right on the nominal tones — here 2400/1200 — but on a worn or oddly shaped recording a crossing interval can read as a whole cycle or half of one, and the figure can come out an octave off. Read the message as "*not mine, and here is roughly what is there*".

Recording back out to a `.WAV`

Put the recorder in `record`, and when the transport stops the tape is re-modulated and written back over the file — in the format and at the sample rate it was mounted with:

```
altairsim> SET acr0:tape mode=record
altairsim> RUN 0                        (the guest records)
altairsim> SET acr0:tape mode=play      (...and the WAV is written here)
```

You can also make a blank tape from scratch — `MOUNT … CREATE`. A blank file has no recording to sniff, so its *name* decides what it becomes: a `.wav` name makes a blank **audio** tape that records a real, playable WAV; any other name makes a blank **byte** tape.

```
altairsim> MOUNT acr0:tape new.wav CREATE mode=record    (a fresh FSK-300 audio cassette)
altairsim> MOUNT acr0:tape new.tap CREATE mode=record    (a fresh byte cassette)
```

The mount line tells you which you got — `new.wav: blank fsk300 tape, ready to record`, or `new.tap: blank raw byte tape` — so there is no guessing. `mode=record` is there because the deck comes up in PLAY, one head and one direction: it is what lets the tape *record*, not what makes it audio — the name does that. The guest can then write to it (a `CSAVE` from BASIC, say), and when the transport stops the recording is written; `UNMOUNT` and `QUIT` are stops. Without `CREATE`, `MOUNT` needs a file that already exists, so a mistyped name is caught as a mistake rather than turned into a blank tape.

Recording over a WAV you already mounted works too. The format and the sample rate to re-modulate into are then the ones the decode found at mount time — so they come from a recording that was already in the file. The recording overwrites the file in place, so work on a *copy* unless you mean to lose the original.

For a file that already exists, the magic decides, never the name. A `.wav` that is not really one — anything that is not RIFF/WAVE — is read as bytes, and recording then puts *bytes* in it, not audio. It will look like it worked; nothing will play it. (This is the existing-file case only: a blank file from `CREATE` carries no magic, so there its name decides, as above.) If you meant audio and the mount line says `raw`, that is what happened.

The stop is what writes it. `UNMOUNT` and any `WIND` (`REWIND` included) are stops too. (The Sol's decks add one the ACR has not got — the guest dropping the motor line; the boards chapter's `sol` section covers it.)

The whole file is rewritten each time, not patched: the audio for a byte starts at an offset that depends on every byte before it, so there is no cheaper splice.

Time is the one thing that does not survive the round trip. A byte image holds no durations, so the leader a real transport needs has to be put back by whoever writes the audio. Two properties do that, in seconds:

Property	88-ACR	Where the number comes from
`leader`	`15`	the MITS manual's *at least ~15 s of steady tone*
`trailer`	`5`	§8's *at least 5 s between batches*

(The Sol's decks default to shorter leader and trailer, measured off a real dub; the boards chapter's `sol` section gives the numbers.)

Set either to `0` to trim the file to its data — which is what the published archive `.wav` files are, and why they will not load on real hardware. Even then a floor of sixteen bit times goes on each end, because a start bit is found by its **edge**: a tape that opened on the start bit itself would lose its first byte.

The carrier can be shaped like the real hardware, or smoothed. A third property, `waveform`, chooses how the tone is drawn when audio is written back:

Property	Default	Choices	What it does
`waveform`	`square`	`square` | `sine`	`square` is what the real modem lays down — a fuller, louder tone that sounds like a genuine cassette dub. `sine` is a smoother, quieter tone.

It is audible, not structural: a tape written either way decodes back to the same bytes, so this changes how the recording **sounds**, never what it holds. `square` is the default because it is the closer match to a real recorder.

The recording level is structural too — it decides whether the tape reads back cleanly.

Property	88-ACR	What it does
`level`	`36`	Recording level, percent of full scale. The old default ran at 80% — more than twice a real dub — which overdrove a real deck's input AGC so the tape read its header and then failed.

Unlike `waveform`, `level` is not merely audible: a tape written at a real dub's level is built to load on the hardware, not only to read back here. The Sol's CUTS modem needs one more property, `rc`, to shape the edges of its lower tone; the boards chapter's `sol` section covers it and the fidelity work behind these defaults.

A multi-file tape comes back as one continuous run. The decoded bytes carry no file boundaries, so the gaps a real operator left between programs are not reproduced.

Recording to a `.TAP` works as it always has, and is unaffected by any of this.

`format`, when you need to overrule the sniff

Each tape unit has a `format` property. `auto` is the default and is almost always right.

```
altairsim> SET acr0:tape format=raw
altairsim> SHOW acr0
```

(`SET` addresses the unit; `SHOW` addresses the **board**, and lists every unit on it with its properties underneath. There is no `SHOW <id>:<unit>`.)

Value	What it does
`auto`	Sniff for RIFF magic; demodulate a recording, read anything else as bytes
`raw`	Read the file's own bytes **even if it is a WAV** — how you inspect a tape that decodes badly
a modulation	Force one: `fsk300` on the ACR, `cuts1200` or `kcs300` on the Sol

It selects a *reading*; it never widens the hardware. Telling an 88-ACR to demodulate `cuts1200` is refused just as firmly as letting it sniff one — the board has the modem it has. A companion read-only `detected` property reports what the mounted tape turned out to be.

The same `format` is also what picks the modulation for a **blank** tape from `MOUNT … CREATE`: there is no separate flag for it. A `.wav` name records in the card's own modem (`fsk300` on the ACR), and `format=fsk300` — or, on the Sol, `format=cuts1200` / `kcs300` — forces one by name. An empty file has no magic to sniff, so `format` (or the `.wav` name) is the whole of the choice.

`format` takes effect at the **next** `MOUNT`, because a tape is decoded once, when you put it in.

Loading Altair 4K BASIC 3.1 — the three-step ritual

This is the whole point of the chapter. It is three commands, and it is what an Altair owner did every single time they wanted to use BASIC, because there was nowhere to keep it.

1. Put the cassette in.

```
altairsim> MOUNT acr0:tape "examples/basic/4K BASIC Ver 3-1.tap"
```

2. Toggle in the bootstrap.

```
altairsim> LOAD "examples/basic/LDR4K31.HEX"
```

About twenty bytes. **On a real Altair you entered this by hand on the front-panel switches** — eight switches, one byte, DEPOSIT, again, twenty times — and if you got a bit wrong you found out by the tape not loading. MITS printed the listing in the manual and expected you to type it in with your fingers. `LOAD` is doing exactly that job and no more: it is not a loader, it is a pair of hands.

3. Run it from address zero.

```
altairsim> RUN 0
```

The bootstrap starts the ACR reading, the tones come off the tape, and BASIC lands in memory and starts itself.

Then BASIC asks you the three questions:

```
MEMORY SIZE?
TERMINAL WIDTH?
WANT SIN? Y

742 BYTES FREE

ALTAIR BASIC VERSION 3.1
[FOUR-K VERSION]

OK
```

Empty answers to the first two mean "all of it" and "the default". `WANT SIN?` is asking whether you would like to spend some of your 4K on trigonometry. Say `Y`; you have 742 bytes left and a working `SIN`.

You are in Altair BASIC. It is 1976, and this is the first product Microsoft ever sold.

To do the whole thing in one command, the package ships the machine that does it for you:

```
$ altairsim examples/basic/basic4k.toml
```

That machine file types those three lines on your behalf. It gets no special powers — every line in it is a line you could have typed.

The sense switches matter

The machine file sets **`sense = 0x80`** on the front panel, and it is not decoration.

The bootstrap reads the sense switches to find out **what device to load from**. Its own printed header says: "*Set A15 on (cassette load), all other switches off.*" A15 on, and nothing else, is `0x80`. Get it wrong and BASIC dutifully loads from **the wrong device**, and then sits there forever waiting for bytes that are never coming, because you told it to listen to a teletype that isn't there.

If a tape load hangs and you are sure the tape is mounted, **check the sense switches first.** The front-panel chapter says where they live.

Speed: the tape and the CPU are on separate clocks

The machine runs flat out by default, and so does the tape — a cassette that took a real Altair about **110 seconds** comes off in **about one**. The two are on separate clocks, as they were in the hardware: the processor's speed is `clock_hz` on the CPU card, the tape's is `rate` on the deck (above), and neither drags the other.

So `SET cpu0 clock_hz=2000000` buys back the period feel of the *processor* — a game plays at its real speed — while `SET acr0:tape rate=real` is the separate switch that makes the *load* take its real wall-clock time. In `rate = full` the guest cannot tell the difference at any CPU speed, because a polled loader only ever asks for the next byte and the byte is always ready.

Altair Disk Extended BASIC 4.1

The other BASIC in the package does not use tape at all. Altair Disk Extended BASIC 4.1 lives on an 8" floppy, and it has what a cassette cannot give you: files, a directory, and a `SAVE` that takes a name.

```
$ cd examples/diskbasic
$ altairsim diskbasic.toml
```

It asks five questions before it will talk to you, and **the second one is the one that stops people**:

```
MEMORY SIZE?
LINEPRINTER? C
HIGHEST DISK NUMBER? 0
HOW MANY FILES?
HOW MANY RANDOM FILES?

37033 BYTES FREE
ALTAIR BASIC REV. 4.1
[DISK EXTENDED VERSION]
COPYRIGHT 1977 BY MITS INC.
OK
```

`MEMORY SIZE?` takes a bare Return and uses all of it. `HIGHEST DISK NUMBER?` is `0` — one drive, numbered from zero. The last two take Return.

`LINEPRINTER?` accepts `C`, `O` or `Q`, and nothing else. Give it a blank line, or an `N`, and it asks again — no error, no complaint, no hint that it wants something particular. A machine sitting at a prompt that reappears every time you answer it looks exactly like a machine that has hung, and it is not one; it is a 1977 program that expects you to have the manual open. `C` is the 88-C700 line printer, and it is a legal answer here even though this machine has no printer board fitted — the answer only tells BASIC where `LPRINT` should go, and nothing is sent anywhere until you use it.

Past that, it is BASIC:

```
OK
PRINT 2+2
 4
```

The disk is mounted read/write, because that is what a real machine was. `SAVE` will write to it.

The cassette BASIC in the package is the real thing, and it is the one that shows you what an Altair actually was: a box with no operating system, no storage, and no software in it, which you talked into existence one toggled byte at a time.

Serial ports, sockets and telnet

The Altair had no screen. What it had was a serial card, and whatever you chose to hang off it — a Teletype, a glass terminal, a modem, a paper tape reader. The card did not know or care. It moved characters.

`altairsim` keeps that arrangement exactly. **Any board that moves characters has one or more UNITS, and every unit can be CONNECTed to an ENDPOINT.** The unit is the socket on the back of the board. The endpoint is what you plugged into it.

```
altairsim> CONNECT sio0:b socket:2323
altairsim> DISCONNECT sio0:b
```

That is the whole of the interface. The interesting part is the endpoint grammar, and it is short enough to print in full.

The endpoints

This table is exhaustive. There are no others.

Endpoint	Is
`console`	the host terminal — your keyboard and your screen.
`null`	nowhere. Writes vanish. Reads never come.
`loopback`	itself. What the guest writes comes straight back as a read.
`socket:PORT`	**LISTENS** on that TCP port. This is the telnet-in case.
`socket:HOST:PORT`	**CALLS OUT** to that host and that port.
`serial:DEVICE`	a real serial port on this host.
`in:PATH`	a host file, read-only — a **paper-tape reader**. The file's bytes feed the board.
`out:PATH`	a host file — a **paper-tape punch**. Whatever the board sends is written to it.
`in:PATH,out:PATH`	both at once on one line: a reader and a punch, two files, two positions.
`terminal`	a terminal in a window the simulator draws itself — a built-in VT100 (or ADM-3A, VT52, H19). Present only in a build with a display. See *A terminal in its own window*, below.
`printer:QUEUE`	a real print queue on this host, write-only. Buffers the bytes into a job and prints it. Present only where the build found a host print system.
`scripted`	a terminal with a caller in place of a human. No tty need exist. It is what the MCP tools and the test suite type into; you are unlikely to type it yourself.

Any of these can be **tapped**: append `|FILE` to log the line to a hex file as it runs, or `|socket:PORT` to **mirror** it live — a second person `telnet`s in to watch the session and can type back onto the line to take over. Both are modifiers on an endpoint, not endpoints of their own — see *Tapping a line to a log file* and *Mirroring a line so a person can watch and take over*, below.

`null` is not an error

An unconnected unit is `null`, and **that is a legitimate state, not a fault.** A 6850 with no cable in it sits there with its transmit register permanently empty, forever ready, and a program that writes to it runs perfectly and talks to nobody. Reads never complete because nothing is sending.

Which is precisely what the real card does with no cable in it. A machine with a second serial port nobody plugged anything into is not a broken machine. `null` models the missing cable, and the guest is entitled to be fooled by it exactly as it would have been in 1977.

The colon is the whole distinction

`socket:2323` **listens.** `socket:localhost:2323` **calls out.** One colon, and it is the same convention every terminal program has used for forty years: a bare port is a port you own, a host and a port is a place you go. Nothing else about the endpoint changes.

Telnetting into the guest

Wire a unit to a listening socket, and the guest has a serial port with a terminal on the end of it. That the terminal is your telnet client, several processes away, is not something the guest can discover.

```
altairsim> CONNECT sio0:b socket:2323
altairsim> RUN
```

Then, from another terminal on your machine:

```
$ telnet localhost 2323
```

The guest is now talking to that window. Your first terminal still has the monitor and `^E` in it. This is how you give a machine two terminals, and it is how you drive a program that wants a console that is not the one you are sitting at.

A terminal in its own window

The telnet route above works, but it has moving parts you do not own: a telnet client must be installed, it must be pointed at the right port, and it mangles any byte it thinks is a telnet command — which breaks cursor sequences and file transfers, because to a serial line every byte is just data. And a modern host terminal emulates a modern terminal, not the ADM-3A or VT52 the 1970s software on the disk was written for.

`terminal` removes all of that. It is a terminal the simulator draws itself, in its own window:

```
altairsim> CONNECT sio0:a terminal
altairsim> RUN
```

The guest's console is now that window; the monitor and `^E` stay in the one you launched from. There is nothing to install and nothing to connect — the window is up the moment the line is, on every platform. Because the terminal is the simulator's, the emulation is exactly the one you ask for, and it answers the reports (like `ESC[6n`) a period program expects.

Bare `terminal` is a VT100 in an 80×24 window. Options ride the connect string after a `?`, joined by `&`, the same as any other endpoint:

```
altairsim> CONNECT sio0:a terminal?emulation=adm3a
altairsim> CONNECT sio0:a terminal?emulation=vt52&size=80x24
altairsim> CONNECT sio0:a "terminal?emulation=h19&size=132x24"
```

`emulation` is one of `vt100` (the default; `ansi` is the same engine), `adm3a` (the Lear Siegler ADM-3A, the classic CP/M terminal), `vt52`, or `h19` (the Heath/Zenith H19, a VT52 superset with an ANSI mode). `size` is *columns*×*rows* and defaults to `80x24`. `phosphor` is the tube colour — `green` (the default) or `amber`. `width` is the opening window width in pixels (a bare number, e.g. `width=1100`); left off, the window auto-sizes to about half the screen. The text is drawn in the real **DEC VT220** character set, and pairs with the period tube look — `[display] crt = true` (see Configuring), under which a `width` opens the window at exactly that size.

```
altairsim> CONNECT sio0:a terminal?emulation=vt100&phosphor=amber&width=1100
```

A `terminal` needs a window, so it is available only in a build with a display. Ask for one in a build without, and it is refused cleanly at `CONNECT`, with the reason named; use `console` or a `socket:` there instead.

The built-in terminal has the same fold-the-bytes settings the console does — `strip7out`, `upper`, a CR/LF option and the rest — under `[terminal]`. They matter for a period monitor that sets bit 7; see *The* *built-in terminal has these too*, later in this chapter.

Calling out

```
altairsim> CONNECT sio0:b socket:bbs.example.com:23
```

The guest dials. As far as the software inside the machine is concerned it has a modem and the modem is connected; it will happily run a period terminal program over it.

A real serial port

```
altairsim> CONNECT sio0:b serial:/dev/tty.usbserial-A600K1XY
altairsim> CONNECT sio0:b serial:COM3
```

The second form is Windows. The bytes go out of a real UART, down a real wire, into whatever you have on the other end.

If you get the device name wrong, it lists the ports that actually exist on your machine. It does not merely say "cannot open". A cable that appeared under a name one character off from the one you expected is ten minutes of somebody quietly deciding the simulator is broken, and the fix is to print the answer instead of the complaint.

What the board does to the wire

The host port is opened at 9600 8N1, and then **immediately re-programmed by the board** — because the board is the only thing in the system that knows what frame it is carrying. What it re-programs the port *from* depends on the board, and the difference is real hardware, not a house style:

- On an **88-SIO** or an **88-ACR** the word format is a set of **jumpers**, so it is the board's `baud`, `data_bits`, `stop_bits` and `parity` properties that become the frame on the wire.
- On an **88-2SIO** — the board in the example above — there are **no such properties, and there must not be**: the 6850's word format is a *register the guest writes*. A property would be a second place to say one thing, and the two would disagree the moment software touched the chip. So the frame on the wire is whatever the **guest** last programmed, and a guest that selects 7E1 reconfigures the cable to 7E1.

So if you want 300 baud, 7 bits, even parity going out of that connector, you do not configure it on the connector. You configure it on the **board**, where a 1975 operator would have set it, with a jumper. Modem control lines — DCD, CTS, RTS — are wired through.

And give the machine its real crystal before you transfer a file

```
altairsim> SET cpu0 clock_hz=2000000
```

The moment the wire leaves the machine, the guest is talking to something that keeps time the way *you* do — and it does not: it counts instructions, so flat out it retires a "three-second" timeout in milliseconds and decides your sender is dead. Flat out is right for a machine talking to itself; **a machine talking to you wants the crystal.** The troubleshooting chapter has the full story.

A paper-tape reader and punch: `in:` and `out:`

A serial or parallel line is where a **paper-tape station** lived, and `altairsim` gives you one out of two host files. The direction is the keyword, not a flag:

```
altairsim> CONNECT lpt0:prn out:printout.txt          # a punch: capture what the board sends
altairsim> CONNECT 4pio0:ja in:reader.tap             # a reader: feed a file to the board
altairsim> CONNECT 4pio0:ja in:reader.tap,out:punch.tap   # both, on one bidirectional line
```

`in:` is a **reader** — a byte *source*. It reads the file from the start, hands the bytes to the board one at a time, and when the file runs out the line simply goes **quiet**: no error, no end-of-file byte, exactly as a reader with no more tape sits idle. A file that is not there is a clean refusal at `CONNECT`, with the path named.

`out:` is a **punch** — a byte *sink*. Whatever the board sends is written to the file. It does **not** truncate: it overwrites forward from the start and extends past the old end, so a short run into a longer old file leaves the old tail behind — the same as spooling fresh tape over a reel that still had some on it. An absent file is created.

`in:` and `out:` are **separate files with separate positions**, so the combined form is two independent heads on one line — reading the tape cannot disturb what the punch has written, and vice versa. Both are **8-bit clean**: the bytes on the wire are the bytes in the file, control codes and all. Nothing reformats them. A relative path follows the usual rule: relative to the file when it is written in a machine configuration, relative to your shell when you type it.

The 88-HSR — a reader with a speed

A real paper-tape reader has a rate, and a program that times its input cares. Pace an `in:` reader with `?cps=N` (characters per second) or `?baud=N` (a line rate at 10 bits per character):

```
altairsim> CONNECT 4pio0:ja in:tape.tap?cps=300      # the 88-HSR high-speed reader
altairsim> CONNECT 4pio0:ja in:tape.tap?cps=30       # the slow reader
```

`in:tape.tap?cps=300` **is** the MITS 88-HSR. Give exactly one of `cps` or `baud`, and a positive rate. With neither, the reader runs flat out — the generic default. The punch takes no options; it writes at the line's own speed.

Printing to a real printer

Where your build was made with host printing, a line can go to a real print queue instead of a host file:

```
altairsim> CONNECT lpt0:prn printer:linewriter
```

Host printing is built on **macOS and Linux** today; the **Windows** builds do not have it yet, so on Windows `printer:` is absent and printing goes to a host file (`out:`) or a socket (`socket:`) instead. To see which your build has, connect to `printer:` with no name — it either lists the queues it can reach or tells you host printing is not in this build.

`printer:` is a **write-only** sink like `out:`, and just as un-printer-specific — any line can use it, not only the 88-C700. The difference is what happens to the bytes: they are held in a buffer and then handed to the host print system as one **job**.

When does a job end? A printer has no "done" signal — a program prints and then simply stops. So `altairsim` decides the boundary for you, and you can tune it in the endpoint itself:

Option	Means	Default
`?idle=N`	end the job after **N seconds** with nothing more printed. `0` = never.	`5`
`?onff`	also end the job on a **form feed** (the page-eject character).	off
`?max=N`	end the job at **N bytes**, so a runaway program cannot fill memory.	a large number

```
altairsim> CONNECT lpt0:prn printer:linewriter?idle=15
altairsim> CONNECT lpt0:prn printer:linewriter?onff
altairsim> CONNECT lpt0:prn "printer:Generic / Text Only?idle=0&onff"
```

Write a bare option (`?onff`) to turn it on; the common case never types `=1`. The boundaries combine — the first to fire ends the job — and an **empty buffer never prints**, so a form feed followed by silence does not leave a blank page behind. A job also goes out when you `DISCONNECT` the line, load another machine, or quit, so nothing you printed is ever left un-sent.

The queue must be one the host passes through **untouched** (a *raw* queue): a printer control language is not text, and a normal queue would try to reformat it. Creating that queue is a one-time step in your operating system's printer setup, outside `altairsim`. If a queue name contains spaces, quote the whole endpoint as shown above. Connect to `printer:` with no name and `altairsim` lists the queues it can see.

Like `out:`, a printer line is **8-bit clean** — the bytes the program sent are the bytes the printer gets.

Tapping a line to a log file (a poor man's protocol analyzer)

When you are trying to work out what a guest and the far end are actually saying to each other, you want to *see the bytes*. Append **`|FILE`** to any endpoint and `altairsim` writes every byte that crosses the line — both directions — to a text file as it runs, in hex and ASCII, with timestamps. The guest cannot tell the tap is there, and it never changes a byte, so it is safe on a binary transfer as well as on a terminal.

```
altairsim> CONNECT sio0:b socket:2323|bbs.hex
```

Telnet in, drive the guest, and `bbs.hex` fills with the conversation:

```
# altairsim capture  socket:2323  2026-07-29 14:03:11  fmt=dump
+0.001000  TX 0000  41 54 5A 0D                                       ATZ.
+0.048213  RX 0000  0D 0A 4F 4B 0D 0A                                 ..OK..
+9.100000  [DCD^]
+45.30000  [DTR_]
```

`TX` is what the guest sent, `RX` what came back; the offset counts bytes within a burst, and the right-hand column is the ASCII (a `.` for anything unprintable). Lines in `[...]` are **modem control-line** edges — carrier, DTR, RTS and the rest — logged as they change, with `^` for a rising edge and `_` for a falling one, so you can see the far end pick up and hang up.

The file is truncated each time you connect: a capture is a fresh trace, not an append. The whole tap is remembered — `SHOW` prints it and `CONFIG SAVE` writes it back — so a machine file can carry a line that is permanently traced.

Three layouts, and a few knobs

The tap's options ride the same `?key=value` grammar the other endpoints use, after the file name:

```
altairsim> CONNECT sio0:b socket:2323|bbs.hex?fmt=cols
altairsim> CONNECT sio0:b in:reader.tap?cps=300|trace.log?fmt=jsonl
```

- **`fmt=dump`** (the default) is the layout above: one hex row per line, strictly in time order — the easy one to read, and the easy one to `grep`.
- **`fmt=cols`** puts what the guest sent on the **left** and what it received on the **right**, so a request and its reply read down the page like a transcript.
- **`fmt=jsonl`** writes one JSON record per transfer — for feeding another program, diffing two runs, or importing into a spreadsheet.

The rest, all optional: `ts=elapsed` (the default — seconds since the first byte), `ts=wall` (the host clock), or `ts=none`; `width=N` bytes per hex row (default 16); `gap=MS` for how long a quiet line waits before a partial row is flushed (default 200 ms); and `pins=off` to leave the modem control-line edges out. Note that the second example taps a **paced paper-tape reader** — the tap composes with any endpoint, options and all.

Mirroring a line so a person can watch and take over

A tap writes to a file. A **mirror** writes to a *socket*, and it is a two-way wire: append `|socket:PORT` to any endpoint and a second person can `telnet localhost PORT` to watch the very session the machine is having — every character the guest prints — and **type back onto the line**, sharing it.

```
altairsim> CONNECT sio0:a console|socket:2323
```

Now the guest talks to your terminal as before, and anyone who telnets to port 2323 sees the same output and can join in at the keyboard. It is the same idea as the tap — a modifier that composes with any endpoint — but where the tap only listens, the mirror also speaks. The everyday use is a console being driven by a program: a person telnets in, watches it work, and takes the keyboard when they want to.

There is no echo added by the mirror. What the watcher sees is exactly what the guest sends, and what the watcher types is input the guest reads — so if the guest echoes (a monitor, CP/M), the typed characters come back the ordinary way, and a password the guest does *not* echo stays unseen at the mirror too. One watcher at a time, the same as a serial line is one wire.

Add `?ro` to make it **watch-only** — a spectator who cannot touch the keyboard:

```
altairsim> CONNECT sio0:a console|socket:2323?ro
```

The watcher never sets the pace. If a watcher's connection is slow, or they pause their terminal, the guest runs on regardless — a laggy watcher loses a little scrollback, never a byte of the guest's. Like the tap, the mirror is remembered: `SHOW` prints it and `CONFIG SAVE` writes it back.

A `CONNECT` it does not understand is an error

If `altairsim` cannot make sense of your endpoint, it **refuses, and tells you what it could have meant.** It never quietly falls back to `null`.

This matters more than it sounds like it does. A silent fallback gives you a machine that boots, runs, prints nothing, and hands you a dead terminal to debug — and you will debug the guest, and the board, and the disk, before you think to doubt the thing you typed. A refusal is a worse morning for exactly two seconds. A dead terminal is a worse afternoon.

Exactly one unit may hold the console

The console is **your keyboard**, and there is one of it.

```
altairsim> CONNECT sio1:a console
console taken from sio0:a
sio1:a: connected to console
```

Connecting a second unit to `console` **steals it, and says who it took it from.** It is not an error and it is not shared. Two boards reading one keyboard would each get roughly half the characters, in an order neither of them could predict, and the resulting machine would appear to be haunted.

To see who has it:

```
altairsim> SHOW CONSOLE
```

That also shows the transforms, which is the rest of this chapter.

The transform chain belongs to the console

This is the most important rule in the chapter, and it is worth stating twice before explaining it.

The `[console]` settings are the only thing in the simulator that alters a byte. Every serial LINE is 8-bit clean. There is no knob anywhere on any board that masks a bit.

They belong to the console, so they stop where the console does. Send a board's console unit down a `socket:` or a real `serial:` port and the far end gets the bytes as the guest wrote them — see *Where the transforms stop*, below, because it is the same rule and it surprises people.

Setting	Does
`upper`	folds what you type to upper case
`strip7in`	clears bit 7 of every character you type
`strip7out`	clears bit 7 of every character the guest prints
`crlf`	translates line endings
`echo`	echoes your keystrokes locally
`bell`	rings the terminal bell on `^G`
`bsdel`	folds backspace and delete together: `off` (default), `bs` (send BS for both), or `del` (send DEL for both)
`attn`	which control character is ATTN (default `^E`)
`base`	`hex` or `octal` — the base the **monitor** prints numbers in. Not a transform: it changes nothing about a byte crossing the console, only how a number is spelled back to you. The *Monitor* document has it.

Set them with `CONSOLE k=v`. (`SET CONSOLE k=v` is the same thing said longer.)

```
altairsim> CONSOLE strip7out=on
altairsim> CONSOLE upper=on crlf=off
altairsim> CONSOLE attn=1D
```

`attn=1D` moves the escape key from `^E` to `^]`. **It must be a control character** — an ATTN you can type by accident in the middle of a sentence is not an escape key, it is a trap.

Why it works this way: `MEMORY SIZ?`

Boot MITS BASIC with the transforms off and it asks you:

```
MEMORY SIZ?
```

The `E` is missing, and there is a garbage character where it should be. BASIC is not broken. **MITS BASIC sets bit 7 of the last character of every message**, as a string terminator — that is how it knows where a string ends — and it sends it. `E` is `45`; with bit 7 set it is `C5`, and your terminal prints whatever `C5` happens to mean to it.

The fix is `CONSOLE strip7out=on`, and the reason the fix lives *there* is the whole argument:

On the real machine, the card sent all eight bits. Nothing masked anything. The card put `C5` on the wire because that is the byte BASIC handed it. The Teletype on the other end simply **did not look at bit 7** — on a Model 33, that is the parity position, and the printing mechanism does not decode it. It printed `E` and threw the eighth bit on the floor.

Nothing was masked. Something on the far end did not look. **`strip7out` is the terminal not looking.** It is a property of the thing you are sitting at, and it belongs on the thing you are sitting at.

Where the transforms stop

Follow that argument one step further and it tells you something the first time it happens is alarming. If `strip7out` is your terminal not looking at bit 7, then the moment your terminal is **not** the thing on the far end, there is nothing there to do the not-looking:

```
altairsim> CONSOLE strip7out=on
altairsim> CONNECT sio0:a socket:2323
sio0:a: connected to socket:2323
```

Telnet in, and `MEMORY SIZ?` is back — garbage character and all, exactly as though you had never set `strip7out`. The same is true of `upper`, `crlf`, `echo`, `bell` and `bsdel`, and it is true of a `serial:` port as well as a socket.

Nothing has been undone. The console settings are still on and still doing their job; the byte simply no longer goes through the console. It goes 2SIO → socket → your telnet client, and every hop on that path is 8-bit clean — which is the rule at the top of this section, not an exception to it. A transform that *did* travel down the cable would be a filter on a line, and the previous two sections are about why that is the one thing this simulator will not do.

So set the equivalent where it now belongs — on the terminal that is actually displaying the text. Every terminal emulator and telnet client has these, under its own names: strip parity or 7-bit display for `strip7out`, local echo for `echo`, newline or CR/LF handling for `crlf`. That is not a workaround; it is the same fix in the same place, one machine further out.

The built-in terminal has these too — in `[terminal]`

There is one terminal one machine further out that *is* the simulator's: the built-in `terminal` window from *A terminal in its own window*, above. It draws the text itself, so it is the simulator that must do the not-looking — and it gives you the same knobs the console has, under `[terminal]` instead of `[console]`:

Setting	Does
`upper`	folds what you type to upper case
`strip7in`	clears bit 7 of every character you type
`strip7out`	clears bit 7 of every character the guest prints
`cr`	`cr` (default, pass the guest's CR through) or `crlf` (add an LF after every CR)
`echo`	echoes your keystrokes locally, for half-duplex software
`bell`	passes `^G` through to the terminal (default on)
`bsdel`	folds backspace and delete together: `off` (default), `bs`, or `del`

Set them with `SET TERMINAL k=v`, read them back with `SHOW TERMINAL`, or put a `[terminal]` block in a machine file. Like `[console]`, it is one section for the machine — the window it applies to is whichever `terminal` line is open.

```
altairsim> CONNECT sio0:a terminal
altairsim> SET TERMINAL strip7out=on
```

`strip7out` earns its keep here on an **even-parity monitor** — the MITS Programming System II computes parity *into* bit 7 of every character, so it sends a carriage return as `8D`. To a console that is masked away; to the raw terminal window `8D` is not `0D`, so it prints as a glyph and the cursor never returns to the left — every line feeds without a carriage return. `strip7out=on` is the fix, exactly as it is for BASIC's prompt. `cr=crlf` is the neighbouring tool, for the rarer guest that sends a bare CR and expects the terminal to supply the LF.

ATTN is the other half of the same fact: it is intercepted at **your keyboard**, before any board is offered the byte, and it is never looked for on a socket or a serial line. A `05` arriving down a cable is a byte of somebody's protocol, and scanning a modem line for a key that exists only on the operator's terminal would corrupt data rather than help.

What *does* travel is the board's own line coding — baud, data bits, parity, stop bits — because that belongs to the board and not to you. That is the section after next.

Why not just strap the board to 7 bits

Because it would work, and then it would silently destroy your data.

Set a 7-bit mask on the board — or a filter on the line — and BASIC's prompt comes out clean. It also **silently corrupts every XMODEM transfer through that port**, because XMODEM sends binary, every byte of it matters, and bit 7 is a real bit in half of them. The file arrives. The checksums even pass on a bad packet often enough to be maddening. And the corruption is in the *plumbing*, which is the last place anyone looks.

A line may carry binary. A terminal is not a line. The transform belongs to the terminal because only the terminal knows it is displaying text.

`data_bits` and `parity` are real hardware, and are not this

A board genuinely does have `data_bits`, `stop_bits` and `parity`, and they are genuinely configurable, because a 6850 genuinely has those straps. They are **a FRAME** — they describe what physically travels down the wire, bit by bit, and on a real serial port they are what the far end must agree to or it will read garbage.

They are never a mask. `data_bits=7` is not "and the byte with `7F`". It is "put seven data bits in the frame", which is a statement about the wire, not about the byte the guest wrote. Do not reach for it to fix a prompt.

Moving files in and out

You will want to get a file into CP/M, and you will want to get one back out. There is a board for it.

The Host Bridge is ours, and it is not a period card

Say it plainly: **MITS never made this.** No S-100 vendor made this. The `hostbridge` board is **our own**, invented for this simulator, and it exists because the alternative — a modem protocol over a simulated serial port at 300 baud — is a slow, fiddly answer to a question that only exists because the machine is simulated in the first place.

Everything else in this program is a board somebody could buy in 1977. This one is not, and you should know which is which. It sits in the manual next to the real cards, and it is the only one wearing a different badge.

It is in the **`default` machine**, so unless you have written a machine file that leaves it out, **you already have it.**

Default port **B0**: `BASE+0` is command/status, `BASE+1` is data.

The three utilities, and where they live

The utilities are **on the disk**, not in the board. They are ordinary CP/M `.COM` files, and they were assembled inside the machine, by the machine's own assembler, from source that ships with it.

Which means a disk can be missing them, and some are — a minidisk image you supplied is one, and so is a disk you formatted yourself. *Getting the utilities onto a disk that hasn't got them*, below, is how you fix that.

`HDIR [pattern]`	List what is on the host.
`R <hostfile> [cpmfile]`	Read host → CP/M.
`W <cpmfile> [hostfile] [B|T]`	Write CP/M → host.

`HDIR`

```
A>HDIR
    1792  07/14/26  R.COM
     640  07/14/26  HDIR.COM
   <DIR>  07/20/26  SRC/

3 files.
A>HDIR *.ASM
```

`HDIR` lists one file per line — size, date and name — much as `ls -l` or `DIR` would. The size is a byte count, or `<DIR>` for a directory (which also keeps its trailing `/`, the one mark that says whether `R` can read it). The list is sorted, and the count at the end is the host directory's, which can run to far more than a CP/M disk holds.

`HDIR` always prints the TRUE host names — the real ones, with their real case and their real length, not the 8.3 names CP/M is going to see them as. That is the point of it. When you cannot work out what to call a file, ask `HDIR` what it is actually called.

`R` — host to CP/M

```
A>R README.TXT
A>R *.ASM
A>R SRC/FOO.ASM
A>R SRC/FOO.ASM WORK.ASM
```

Wildcards work. Subdirectories work — and **both `/` and `\` are accepted on every host**, because you should not have to remember which machine you are sitting at. A second argument names the file on the CP/M side and overrides the automatic mapping.

`W` — CP/M to host

```
A>W GAME.COM
A>W GAME.COM game.com
A>W NOTES.TXT notes.txt T
```

`W` defaults to B, and that is the important sentence

Binary is the default. `W` writes **every byte of every record, exactly** — nothing is inspected, nothing is stripped, nothing is guessed. A `.COM` file survives the trip and runs.

The cost is visible and it is honest: **a text file arrives with up to 127 trailing `^Z` bytes** on the end, because CP/M stores files in 128-byte records and pads the last one, and `W` in binary mode does not presume to know which of those bytes you meant. Your editor will show them. Trim them, or ask for `T`.

`T` (text) stops at the first `^Z`. You get clean text, exactly as long as it should be.

And now the trap, which is the reason `T` is not the default: **a binary file containing byte `1Ah` comes back TRUNCATED.** Byte `1Ah` is `^Z`. It is a perfectly ordinary byte in a `.COM` file — it is `LDAX D` — and in text mode the transfer stops dead at the first one, silently, and hands you a file that is the right name and the wrong length.

Some emulators default to text and keep a list of extensions to exempt. **That silently truncates files** the day you transfer something whose extension is not on the list, and you find out weeks later. We would rather hand you a text file with some padding on it than hand you half a program and call it done.

Mode	What it does	Costs you
`B` (default)	Every byte, exactly.	Up to 127 `^Z` bytes on a text file.
`T`	Stops at the first `^Z`.	**Truncates any binary containing `1Ah`.**

The sandbox

The board has a **`hostdir`** property. It is the host directory the guest can see, and it is the **only** host directory the guest can see.

```
altairsim> SET hb0 hostdir=/tmp/xfer
```

Empty — which is the default — means **the directory you ran `altairsim` from.**

Where a relative `hostdir` points

`hostdir` is a path, so it obeys the path rule from *Machines* — **both halves of it**, and this is the one place the difference has teeth, because this path is a fence and the other end of it is a CP/M program.

```
altairsim> SET hb0 hostdir=xfer          # you typed it -> ./xfer, from YOUR shell
```

```
[[board]]
id      = "hb0"
hostdir = "xfer"                        # the file wrote it -> the xfer beside THIS FILE
```

Both say `xfer`. They are **different directories**, and each is the one its author could see — which is the same rule that decides where `mount = "cpm.dsk"` looks. The machine file's `xfer` travels with the machine file, so an example directory you copy to your desktop still has its own transfer folder.

You never have to work it out. The written value and the resolved one are both printed, and the resolved one is the fence:

```
altairsim> SHOW hb0
  property        value           legal
  port            0xB0            0x0..0xFE
  hostdir         xfer
  hostdir_root    /home/you/altair/disks/cpm22/xfer (read-only)
  readonly        false           true|false
```

`hostdir` is what was **written**. `hostdir_root` is where the fence **actually is** — read-only, because it is a fact about this run rather than a setting, and it is never written back into a machine file.

`SHOW PATHS` prints the same root beside the other two bases. If `R` cannot find a file you are certain you put in `xfer`, that is the line to read: there is very likely a second `xfer`, and the guest is standing in the other one.

…and what that means for `R` and `W`

The path rule stops at the board. **It does not reach the guest, and `R` and `W` never see it.**

At the `A>` prompt you are not typing a host path at all — you are typing a **name**, and the board resolves it inside `hostdir_root` and nowhere else:

```
A>R FOO.ASM         <- hostdir_root/FOO.ASM. Never your cwd, never the machine file's
A>W RESULT.TXT      <- hostdir_root/RESULT.TXT
```

So the path rule reaches `R` and `W` **exactly once**: it decides which directory `hostdir` named, back when someone wrote it. After that the guest has one directory and no way to say anything about any other — `..`, an absolute path and a drive letter are all refused at the board, as below.

This is worth stating plainly because the two mechanisms look alike and are not:

	decides	**confines**
the path rule	where a path *written by a human* points	nothing at all
`hostdir`	nothing you type	**everything the guest can reach**

This is the *only* fence in the simulator, and it is worth not confusing with the path rule in *Machines*. That rule decides where a path written in a machine file points, and it confines nothing at all. This one decides how far the **guest** can reach, and confines everything. A machine file may mount a disk from anywhere on your system; the CP/M program running off that disk still sees only `hostdir`.

The guest cannot escape it. All of the following are refused, at the board, before anything touches your filesystem:

- an absolute path
- a drive letter
- any `..` component
- a symlink that resolves outside the root

This is a **deliberate, hard boundary**, not a best-effort filter. The guest is running software from 1977 that you found on the internet, and the answer to "what if it is hostile" should not be "well, it probably isn't."

`readonly` makes it a one-way street:

```
altairsim> SET hb0 readonly=on
```

Files come **out of** the host. Nothing goes back in. `W` fails.

And it means the guest can write your working directory

Here is the other half of that, and it does not get buried:

By default, guest software can read and write the directory you ran `altairsim` from.

That is a real capability and it is on unless you turn it off. It is a **deliberate trade**, not an oversight: `R FOO.ASM` working the instant you unzip the package, with no setup, is worth a great deal, and the directory you are standing in is the directory you almost always meant. But it is your directory, with your files in it, and a CP/M program you did not write can reach them.

If that is not what you want, point `hostdir` somewhere else, or set `readonly=on`. Both are one line.

Names: 8.3, and how a host name becomes one

CP/M has eight characters and an extension of three, and your host does not. So the board maps:

1. **uppercase** the host name,
2. truncate the base to **8**,
3. truncate the extension to **3**,
4. **drop illegal characters.**

```
my-notes(2).txt   →   MYNOTES2.TXT
```

A **second argument overrides the whole business**, and when the mapping produces something you do not like, that is what it is for:

```
A>R my-notes(2).txt NOTES.TXT
```

And again: `HDIR` prints the **true** host names, so you can always find out what you are actually asking for.

Case, and why the board does not guess

The CP/M command line folds everything to upper case. This is not something we do; it is what the CCP does, before your program ever sees the line. By the time `R` runs, `R readme.txt` and `R README.TXT` are **the same command**. The information is gone.

So the board compensates, in this order:

1. **Exact match wins.** If there is a file called exactly what you asked for, that is the file.
2. **Otherwise, fold case and look again.** `README.TXT` will find `readme.txt`.
3. **If several files match, it REFUSES.**

Step 3 is the one that matters. If your directory holds `readme.txt` and `README.TXT` and `ReadMe.Txt`, the board does not pick one, does not pick the newest, and does not pick the first. **It tells you it cannot tell**, and stops. A file transfer that guesses wrong and tells you it succeeded is worse than one that fails.

Getting the utilities onto a disk that hasn't got them

There is an obvious hole in everything above: **`R` is how you get a file onto a disk, and `R` is a file on the disk.** Boot something that has not got it — a minidisk image you supplied, or a fresh disk you made yourself — and you cannot use the board to fetch the program that drives the board.

You break the loop through **the console**, which is the one channel that exists before any file-transfer utility does. You type the program in. Not literally: you paste it.

You cannot paste `R.COM`. It is binary, and a console is a text device — the first `1Ah` in it would end the paste, and the bytes above `7Fh` would not survive the trip at all. So you paste the **`R.HEX`** that `LOAD` was going to turn into `R.COM` anyway. It is Intel HEX: colons, hex digits and line ends, about 4.8 KB of them, and every byte of it is printable.

It ships with the package. The utilities are in `cpm/hostbridge/` — the 8080 sources, the `.HEX`, and the assembled `.COM`, all together. The CP/M disk that ships in `examples/cpm` already has `R.COM`, `W.COM` and

`HDIR.COM` on it, which is why this section is about the other kind of disk — a fresh one, or a minidisk image you brought yourself. For the paste below, open `cpm/hostbridge/R.HEX`.

1. Tell PIP to write a file from the console.

```
A>PIP R.HEX=CON:
```

PIP is now copying what you type into `R.HEX`, and it will keep doing so until you tell it to stop.

2. Paste the whole of `R.HEX` into the terminal.

Open the `R.HEX` you fetched on your host, select all of it, and paste it into the window where `altairsim` is running. It echoes as it goes — 108 lines of it — because PIP echoes console input, and that echo is your confirmation the guest is really receiving it.

3. End it with `^Z`.

```
^Z
A>
```

Ctrl-Z is CP/M's end-of-file on a console, and it is what closes the file. You are back at the `A>` prompt and `R.HEX` is on the disk.

4. Turn the HEX into a program.

```
A>LOAD R

FIRST ADDRESS 0100
LAST  ADDRESS 07A0
BYTES READ    06A1
RECORDS WRITTEN 0E
```

`LOAD` reads `R.HEX` and writes `R.COM`. It assumes the `.HEX`, so `LOAD R` is the whole command. `FIRST ADDRESS 0100` is the number to glance at: a CP/M program starts at `100h`, and if that line says anything else the paste lost something.

5. Use it, and never paste again.

```
A>R W.COM
A>R HDIR.COM
```

That is the payoff. `R.COM` exists now, so the other two utilities come across the board as ordinary binaries — no HEX, no `LOAD`, no pasting — provided `hostdir` points at the directory they are in:

```
altairsim> SET hb0 hostdir=cpm/hostbridge
```

Two things that would bite you on real hardware and do not bite you here

Nothing is dropped, however fast you paste. On a real Altair, shoving 4.8 KB at a 300-baud serial card with no flow control is exactly how you overrun the UART and lose characters in the middle of a HEX record. Here you cannot: the simulated 6850 and 1602 both pace arrivals at the line rate but **only take a byte when the receive register is free**, so the byte waits on the host side rather than being lost. A paste of any size arrives intact. (This is a deliberate departure from the hardware, and the serial chapter says so — an overrun is data loss the host transport genuinely does not have.)

And a bad paste tells you. Every Intel HEX record carries a checksum, and `LOAD` checks it. A corrupted line is an error, not a program that runs almost correctly.

The same route builds them from source

Pasting `R.HEX` gets you the program. Pasting `R.ASM` gets you the program *and* the ability to change it — the disk's own `ASM.COM` assembles it, and `LOAD` finishes the job exactly as above:

```
A>PIP R.ASM=CON:
   ...paste R.ASM, ^Z...
A>ASM R
A>LOAD R
```

That is not a different technique, only a longer paste — about 28 KB instead of 4.8 — and it is how the `.COM` files that ship on the `hostbridge` disk were built in the first place. If you only want to *run* the utility, paste the HEX; it is six times shorter and needs no assembler on the disk.

Why one `R.COM` works on every disk you will ever build

Every file operation goes through CP/M's BDOS. Not the BIOS. Not a disk parameter block. Not an assumption about how many sectors are on a track or where the directory lives.

`R` and `W` open, read, write and close through the same BDOS calls any CP/M program uses, and the BDOS is the thing that knows about your disk. Which means the same `R.COM`, unmodified, runs on:

- the 8″ floppy controller,
- the 8 MB disk,

- the minidisk,
- and **any BIOS anybody writes later**, for a controller that does not exist yet.

That was the design goal. The board is not a period card, but the software that drives it is a perfectly well-behaved CP/M program, and it will still work on the machine you have not built yet.

Worked examples

Complete sessions. **Every transcript below was captured from the program**, not typed out from memory — if it says the machine printed something, the machine printed it.

These are a few of the machines in `examples/`, gone through at length because each one teaches something about how the machine works rather than merely how to start it. **They are not the whole of what is there.** Every folder under `examples/` carries its own README — as Markdown and as a PDF beside it — saying what that machine is and what to type, so the place to see what you have is the directory itself, not a list in here.

---

1. CP/M from a floppy

```
$ altairsim examples/cpm/cpm22-buffered.toml
```

```
startup> RUN FF00
[console -- ^E returns to the monitor]

56K CP/M 2.2b v2.3
For Altair 8" Floppy

A>
```

What is on the disk

Ask the obvious way and the answer is alarming:

```
A>DIR
A: L80      COM

A>
```

One file, on a disk with 18K free. The arithmetic does not work, and it is not supposed to: almost everything on this disk is marked `$SYS`, and `$SYS` is precisely the attribute that hides a file from `DIR`. It is how a period system disk kept its utilities out of the way of your own filenames. `STAT` sees through it:

```
A>STAT *.*

 Recs  Bytes  Ext Acc
   31     4k    1 R/W A:ACOPY.COM
   23     4k    1 R/W A:AFORMAT.COM
   64     8k    1 R/W A:ASM.COM
   17     4k    1 R/W A:CRC.COM
   38     6k    1 R/W A:DDT.COM
   ...            ...  (thirty-eight lines in all)
  190    24k    2 R/W A:MBASIC.COM
   58     8k    1 R/W A:PIP.COM
  149    20k    2 R/W A:STARTRK.BAS
   83    12k    1 R/W A:WM.COM
Bytes Remaining On A: 18k

A>
```

Thirty-eight files: a macro assembler, a linker, Microsoft BASIC, `DDT`, `PIP`, a text editor, a disk formatter, and Star Trek. It is a working 1977 development system, and `STAT *.*` is how you look at it.

Out, and back in

`^E` stops the machine and gives you the monitor. Nothing is lost — a bare `RUN` resumes at the instruction it was about to execute.

```
A>
ATTN -- the machine is still at CA9C. RUN resumes.
C0Z1M0E1I0 A=00 BC=007F DE=CA01 HL=BC0E SP=BC37 IE=1 PC=CA9C  CALL CA78
altairsim>
```

Look at the machine while it sits there:

```
altairsim> BOARDS
altairsim> SHOW dsk0
altairsim> DUMP 100
```

…and hand the keyboard back:

```
altairsim> RUN
```

Your `A>` prompt is exactly where you left it.

Before you write anything

The disk is mounted **read/write** and there is no undo. Copy the folder first — it is self-contained and boots from anywhere:

```
$ cp -R examples/cpm my-cpm
$ altairsim my-cpm/cpm22-buffered.toml
```

And when you are done writing files, **get back to the `A>` prompt before you quit.** The BIOS holds a track in memory and only flushes it when it next reads the console. The disks chapter explains why.

---

2. Altair BASIC from a cassette

```
$ altairsim examples/basic/basic4k.toml
```

```
startup> MOUNT acr0:tape "4K BASIC Ver 3-1.tap"
acr0:tape: mounted 4K BASIC Ver 3-1.tap
startup> LOAD "LDR4K31.HEX"
loaded 20 bytes (1 page) from LDR4K31.HEX (0000-0013)
startup> RUN 0
[console -- ^E returns to the monitor]

MEMORY SIZE?
TERMINAL WIDTH?
WANT SIN? Y

742 BYTES FREE

ALTAIR BASIC VERSION 3.1
[FOUR-K VERSION]

OK
PRINT 6*7
 42

OK
```

Seven hundred and forty-two bytes free. That is the machine Microsoft was founded on.

What those three startup lines actually are

They are not magic, and they are not a boot command — **there is no boot command.** They are the three things an operator did in 1975, written down:

`MOUNT acr0:tape "…"`	**Put the cassette in the recorder and press PLAY.**
`LOAD "LDR4K31.HEX"`	**Toggle in the bootstrap.** Twenty bytes, entered by hand on the front-panel switches. MITS printed them in the manual.
`RUN 0`	**Set the switches to zero, press EXAMINE, then press RUN.** EXAMINE is what puts the switches into the program counter; without it RUN carries on from wherever the machine already was.

Anything you can type at the prompt, a machine file can do. It gets no special powers, and that is why you can do this yourself:

```
altairsim> REWIND acr0:tape
altairsim> RUN 0
```

`REWIND` is a verb the **cassette board brings with it** — it exists only because there is an ACR in the machine. You need it to load the same tape twice, for exactly the reason you would have needed it in 1975.

The tape is not in the machine file

Look again at what the machine file declares: a front panel, a processor, a serial board, a cassette interface, and some memory. **It does not declare the tape.**

A machine file describes **hardware**. Which cassette is in the recorder is not hardware — and there is no motor control on the board to make it one. You put the tape in, and you press PLAY. That is what `MOUNT` is.

One number in that file you could not have guessed

The front panel's `sense` switches are set to `80`. That is `A15` up, and nothing else.

The bootstrap's own printed header says why:

```
** Set A15 on (cassette load) **
** All other switches off **
```

`A15` up means *load from the cassette*. Get it wrong and BASIC comes up talking to the wrong device, or to nothing at all. Period software reads those switches, so they are not decoration.

It loaded in a second, and a real one took two minutes

The machine runs **flat out** by default. A 300-baud cassette that took a real Altair 110 seconds comes off in about one.

If you want the real thing:

```
altairsim> SET cpu0 clock_hz=2000000
altairsim> REWIND acr0:tape
altairsim> RUN 0
```

…and now it takes 110 seconds, because the tape costs the same number of T-states either way. **What the guest sees is identical.** The crystal buys period *feel*, not period *behaviour*.

The one before this one: BASIC 1.0

Beside the 4K is `basic1.toml`, and it boots "**8080 BASIC VER 1.0**" — the oldest Altair BASIC there is, the one Bill Gates and Paul Allen wrote in 1975. Same idea, one difference that is worth seeing:

```
$ altairsim examples/basic/basic1.toml
```

```
tape: 00:15 / 02:30 (100%)

[console -- ^E returns to the monitor]
RUN 0

MEMSIZ?
WANT SIN-COS-ATN?

2000 BYTES FREE

8080 BASIC VER 1.0

READY
```

It takes two moves, not one. The 4K's bootstrap jumps into BASIC on its own when the tape runs out. BASIC 1.0's does not — it copies the tape into memory from `0000` and loops forever, because it has no idea how long the tape is. That is not a shortcoming of the simulator; it is what the 1975 operator faced. So you watch the tape load, press **`^E`** when it is done, and type **`RUN 0`** to start BASIC by hand — the STOP/RESET/RUN a real operator did at the front panel.

Two small things that look like bugs and are not: the prompt really is `MEMSIZ?` — Microsoft set the end-of-message bit on the `Z` rather than spend a byte on a trailing `E` — and at the flat-out default the tape loads in an instant, so `tape: 02:30 / 02:30 (100%)` is the counter's *finished* frame. `SET acr0:tape rate=real` before you run plays it at the true 300-baud speed and the counter climbs through the two and a half minutes it took in 1975.

---

3. CP/M 2.2 off an 88-HDSK hard disk

```
$ altairsim examples/hdsk/hdsk.toml
```

Example 1 boots CP/M off an 8" floppy. This one boots the *same operating system* off a **hard disk** — a five-megabyte platter where the floppy holds a third of a megabyte — and it is here because the disk is the least floppy-like thing in the package: the 88-HDSK is not a disk *card* at all, but an outboard controller with its own processor that hands the Altair whole sectors over a handshake protocol.

```
startup> RUN FC00
[console -- ^E returns to the monitor]

HDBL 2.00
LOADING FROM 0

48K CP/M 2.2b v1.6
For MITS 88-HDSK

A0>
```

`RUN FC00` is the machine file's whole `startup`, run before you arrive — on a real machine it was EXAMINE `FC00` and RUN. `FC00` is **HDBL**, the hard-disk boot PROM: it reads the platter's Pack Descriptor Page, loads the boot pages it names, and jumps into CP/M. `LOADING FROM 0` is HDBL telling you which platter it took (sense switch A11 picks it; the default is drive 0).

The prompt is `A0>`, not `A>`. This CP/M numbers the platter as well as the drive: `A0` is drive A, platter 0. Otherwise it is CP/M as example 1 leaves it — `DIR`, `TYPE`, `STAT`, and the rest all behave, and this build carries its own source on the disk:

```
A0>DIR
A: BOOT     ASM
```

`BOOT.ASM` is the 88-HDSK bootstrap, read straight off the platter through the controller — proof the drive is not just spinning but being *read*, sector by sector, the way the real Datakeeper was. `^E` (ATTN) takes the keyboard back to the monitor at any point and `RUN` resumes; `^C` is CP/M's warm boot and CP/M gets it.

There is no undo. Drive 0 is mounted read/write, because that is what a real machine is, and CP/M writes to `A:` for anything you create. The package you were handed has no rope back to the original image — copy the folder before you test writes in anger, or add `readonly = true` to the drive in the machine file.

---

4. Altair Disk Extended BASIC 4.1 from a floppy

```
$ altairsim examples/diskbasic/diskbasic.toml
```

The BASIC in example 2 comes off a cassette and forgets everything when you turn it off. This one lives on an 8" floppy and has files, a directory, and a `SAVE` that takes a name. It boots from the same DBL PROM at `FF00` that CP/M does — `startup` runs it for you.

It interviews you first, and this is the example where knowing the answers matters:

```
MEMORY SIZE?
LINEPRINTER? C
HIGHEST DISK NUMBER? 0
HOW MANY FILES?
HOW MANY RANDOM FILES?

37033 BYTES FREE
ALTAIR BASIC REV. 4.1
[DISK EXTENDED VERSION]
COPYRIGHT 1977 BY MITS INC.
OK
```

Three of the five take a bare Return. `HIGHEST DISK NUMBER?` is `0`, because there is one drive and it is numbered from zero.

`LINEPRINTER?` takes `C`, `O` or `Q` — and re-asks in silence on anything else. No error, no hint. Answer it with Return or `N` and you get the prompt back, forever, which looks like a hang and is not one. `C` is the 88-C700 line printer; it is legal here even with no printer board fitted, since the answer only decides where `LPRINT` would go.

That is the whole trick. Past it, it is BASIC, and the tapes chapter has the longer version.

Where to go next

- **The examples this chapter did not walk through** — `examples/`, and the README in each folder.
- **Move a file between CP/M and your own machine** — the file-transfer chapter (`R`, `W`, `HDIR`).
- **Telnet into the guest, or wire it to a real serial port** — the serial chapter.
- **Look at the bus while it runs** — the *Debugger* document.

The MCP server

```
$ altairsim --mcp
```

That runs `altairsim` as an **MCP (Model Context Protocol) server** on stdin and stdout, so that an AI assistant — Claude, or anything else that speaks MCP — can drive the machine through **typed, structured tools** instead of screen-scraping a text terminal.

It is not a wrapper, and it is not a second model of the world. **The MCP server runs on the same machine object as the monitor.** What the tools see is exactly what `SHOW` sees, because it is the same machine, answering the same questions, through a different door.

What the tools do

Enough to operate the machine:

- List the board types available, and every property each one has — **with its type, its default and its legal range.**
- List the boards actually in the machine.
- Get and set any property on any board.
- Add a board; mount a disk or a tape into it; wire a serial line to an endpoint.
- Examine, deposit, fill, search, save and disassemble memory.
- Run the machine, step it, set breakpoints, and read the bus flight recorder.
- Snapshot the whole machine's state and restore it.

The five you will see named are `board_types`, `board_list`, `board_get`, `board_set` and `board_add`. The rest follow the monitor's own vocabulary.

Driving a running guest

Building a machine is half of it; the other half is **operating one that is running** — typing at its console and reading what it prints. Four tools do that, and they are what let an assistant boot CP/M, run `ASM`, and talk to a program over a serial port entirely through MCP:

- **`run`** — advance the guest a bounded slice and return what it printed. It is the expect loop in one call: pass `input` to type a line, `until` to stop when a string (a prompt like `A0>`) appears, `from` to set the PC first (booting is `from` the boot PROM). It **also stops on its own when the guest reaches a prompt** — spinning on the console with nothing to say — so you get control back without guessing a timeout. Every stop says why in `stopped`: `match`, `idle`, `timeout`, `steps`, `halt`, `breakpoint`.

- `send` — type at the console without running (then `run` to let it be read).
- `recv` — drain what the guest has printed since you last looked, without running.
- `regs` — the CPU registers right now.

The shape of a session is therefore: `run {from: 0xFF00, until: "A0>"}` to boot, then `run {input: "ASM FOO\r", until: "A0>"}` per command, reading the reply each time. A `run` **never blocks** — it advances the guest for at most `timeout_ms` (default 2000) and returns — so a `tools/call` always comes back, unlike a bare `RUN` through the `monitor` tool, which under a pipe waits on a stdin that is the JSON-RPC channel itself.

By default the guest runs flat out, which is what you want for booting and for driving a prompt. But when a real device is on a serial line and you have set a clock speed with `SET cpu0 clock_hz=…`, `run` paces the guest to that clock, so a reply the device sends a fraction of a second later lands while the guest is still waiting for it. And with such a device on the line `run` will not cut a transfer off when `timeout_ms` runs out: as long as bytes are still arriving off the wire it keeps going, and returns only once the line has genuinely gone quiet. So a boot loader that reads its whole system image in over a serial disk finishes in one call.

Under `--mcp` the console line is quietly re-seated onto an in-memory terminal the server owns (there is no host keyboard behind a pipe), which is what `send`/`run`/`recv` read and write. Everything else on the machine — a second serial board wired to a real port, a socket — keeps running and is serviced on every `run` slice, so a program shuttling bytes between the console and a modem port works exactly as it would at a real terminal.

Watching over your shoulder — `--mirror`

Add `--mirror socket:PORT` alongside `--mcp` and a person can `telnet localhost PORT` to **watch the very session the assistant is driving** — every character the guest prints — and **type back onto the line to take over**, sharing the console:

```
$ altairsim cpm --mcp --mirror socket:2323
```

It wraps the assistant's console in the same mirror the monitor offers (`<endpoint>|socket: PORT`, see the *Serial lines* chapter). The assistant keeps driving through `run`/`send`/ `recv` exactly as before — the mirror is transparent to it — while whatever it types and whatever the guest prints also crosses the socket to the watcher. Add `?ro` to make it watch-only — **quote it on the command line**, because `?` is a shell wildcard:

```
$ altairsim cpm --mcp --mirror 'socket:2323?ro'
```

Without the quotes the shell tries to match `socket:2323?ro` as a filename and fails (zsh reports `no matches found`) before `altairsim` ever sees it. One watcher at a time.

The watcher never sets the pace, and one thing follows from that: the guest only advances **during a `run`**, so a character the watcher types between runs waits on the line and is read on the next `run` — the same as `send` staging input for the next `run`. While a `run` is in flight the two share the console live.

Debugging and inspecting

The monitor's debugger is here too, structured. `step` advances a set number of instructions and hands back the register file and where the CPU came to rest; `breakpoints` lists, adds and removes the same breakpoints `BREAK` sets — and because they are the machine's breakpoints, a `run` or a `step` stops when one fires. `disasm` decodes memory through a non-invasive read, so it works on a ROM and even with no processor running. `bus_trace` returns the always-on flight recorder — the last cycles every board saw, with who drove and who answered — and `bus_irq` reports the interrupt lines the way `bus_map` reports the decode. `snapshot` and `restore` save the whole machine's state and read it back into a machine built the same way.

None of these block, and none of them need the console: they are questions about the machine, answered the same way `SHOW` answers them. When a typed tool does not reach a corner you need — a conditional breakpoint, an octal listing — the `monitor` tool runs any monitor command and returns its text.

The schemas describe themselves

Every tool's schema comes off the same reflection layer as the TOML keys and the `SET` / `SHOW` commands. There is one description of what a board is and what it can be asked, and the machine file parser, the monitor, and the MCP server all read it.

The consequence is the point: **a board added tomorrow is drivable by an assistant the day it lands, with no new code.** Nobody writes an MCP tool for the new board. The board declares its properties, as it must anyway to be configurable at all, and the tool schema is that declaration.

So there is no tool reference in this manual. Start the server and ask it what it has — `tools/list` returns every tool the server exposes, and `board_types` every board type; their answers are authoritative in a way a printed list could never be.

Configuring an assistant to use it

MCP clients differ, but they all want the same two things: a command to run, and the fact that it speaks over stdio. The command is `altairsim <machine> --mcp`, and it does. Register it **once** and from then on you talk to the assistant, not to the server.

Claude Code (the command line) takes it as one command — everything after `--` is what it will run, and it is best run from the directory you want the machine's files to resolve against:

```
claude mcp add altairsim -- altairsim <machine> --mcp
claude mcp list                       # confirm it registered and is reachable
```

Claude Desktop, or any client that reads a JSON config, wants an `mcpServers` entry naming the command and its arguments (a `cwd` sets the working directory, since a desktop app has no shell to inherit one from):

```
{
  "mcpServers": {
    "altairsim": { "command": "altairsim", "args": ["<machine>", "--mcp"] }
  }
}
```

The `<machine>` is a built-in name or a machine file, exactly as on the command line. If `altairsim` is not on your `PATH`, give its full path as the command.

You are not limited to one. Register several machines under different names (`altair-cpm`, `altair-basic`) and an assistant sees them all at once; `claude mcp add`'s *scope* decides whether a server is tied to the one project directory you added it from (the default), travels with a folder as a `.mcp.json` (`--scope project`), or is available everywhere (`--scope user`). One thing to know for file exchange: the host-bridge sandbox is the server's working directory. The `claude` command line sets that to wherever you started it, so a relative machine path and the sandbox line up with the folder you are in; the desktop app currently starts the server in your home directory instead, so there give the machine an absolute path.

`DRIVING-WITH-AI.md`, in this package, is the briefing written for the assistant itself — drop it in a working directory and the assistant has the recipes for booting, building and debugging over these tools. The `examples/ai-mcp/` folder is a ready-made such directory, with a walkthrough that has an assistant find and fix a bug entirely over MCP.

Starting a project of your own

The `examples/ai-mcp/` folder is a guided round trip; once you want to write your own software the same shape is how to begin. **Make a folder of your own, put a machine in it, and point the assistant at that folder — not at the copy that came in the package.**

Put the machine and its images at the root of the folder. Copy a machine that boots the system you want to build on — the whole `examples/ai-mcp/` folder is a good starting point, disk and machine file together — into a new folder, and work there. Register the server from inside that folder (as in the section above), so the machine file and the host-bridge sandbox both resolve there. The guest writes to the disk image as it runs and the assistant leaves build files beside it, so keeping the shipped copy untouched means you always have a clean one to start over from. Make that a real baseline: keep a spare copy of just the machine and its fresh disk, so "start over" restores something known rather than whatever the guest last wrote.

Give it the local truth to read. Drop `DRIVING-WITH-AI.md` in the folder, and keep a second folder — call it `Reference` — for anything else you want the assistant to lean on: the parts of this manual that matter to your project, and any source material of your own, converted to plain Markdown, which is the form an assistant ingests most reliably. These copies go stale the moment altairsim updates, and the assistant cannot tell an old copy from a current one, so refresh them when you update altairsim.

Keep a hand on it — it will reach for what it already knows. An assistant does not read your machine's files and then do as they say. It predicts its next move from everything in front of it, weighed against everything it learned in training — and what it learned is vast, while a line in a local file is a single faint signal it saw a moment ago. So when the job looks like a common one — boot CP/M, assemble a file, copy something onto a disk — the move it reaches for first is the one it has seen a thousand times, even when the files in the folder say otherwise. A plain example: asked to get a program onto a disk, it will reach for a host tool like `cpmtools`, which does not understand the Altair's hard-sectored disks at all — when the way that works is to build inside the machine over the host bridge, exactly as `DRIVING-WITH-AI.md` lays out. The pull toward the familiar answer gets stronger the longer a session runs, and once it starts down that road it tends to talk itself further along rather than stop and re-read.

So watch what it actually sends and reads back — the `--mirror` socket above is there for exactly this — rather than trusting its own summary of what it did. Correct it at the first wrong step, before it builds on that step, and point it at the file you mean and tell it to use that and only that. Be most wary when its answer looks the most standard: that is the moment a habit has most likely overridden something particular about your machine.

Make starting and stopping a ritual. A short instruction you give every session pays for itself. At the end — "clean up" — have it close any processes still running, write down what you learned, and save where you are in a note in the folder. At the start, have it read that note and the `Reference` folder, boot the machine, and tell you where you left off. The note in the folder is what makes a session resumable; without it the assistant reconstructs the state from scratch each time, and reconstructs it wrong.

Troubleshooting

Most of what goes wrong here is not a fault. It is the machine doing exactly what a real one did, in a way nobody warned you about. This chapter is the warning, after the fact.

`^C` does not stop the machine

It is not supposed to.

`^C` belongs to the guest. CP/M reads it — it is how you warm-boot, and how you break out of a BASIC program. A stop key the guest also wants is a stop key the guest will eat, and then you are trapped inside your own simulator.

`^E` is the stop key. It is ATTN, the host intercepts it before the guest is ever offered the byte, and **no program running inside the machine can disable it, ignore it, or take it from you.**

If a guest genuinely needs `^E` for something of its own, move ATTN out of its way:

```
altairsim> CONSOLE attn=1D
```

That makes it `^]`. It must be a control character.

I pressed ATTN and now the machine seems stuck

It is not stuck. It is **stopped**, which is what ATTN is for, and nothing has been lost.

ATTN halts the processor and hands you the monitor. It is not RESET and not POWER: the registers, the memory and the disk are precisely as the guest left them, and the monitor told you where to pick it up when it printed "*still at …*".

```
altairsim> RUN
```

A bare `RUN` — no address — resumes at that exact instruction, and your `A>` is where you left it. (`RUN <addr>` is a different thing: it loads the PC first, so it *restarts* rather than resumes.)

And while you are stopped, you can look: `REGS`, `EXAMINE`, `DUMP`, `DISASM` and `STEP` all work at this prompt, on a machine that is not moving under you. See the *Debugger* document. `BREAK` is for stopping at a place you cannot reach by hand.

My file was not written / the disk lost my changes

You quit too early.

The CP/M in this package has a **track-buffering BIOS**: it keeps a whole track in memory and only flushes it to the image when it next reads the console. This is not a shortcut in the simulator; it is what that BIOS does, and it is why the machine was fast. (Not every CP/M is built this way — some write each sector as it goes — but you cannot tell from the prompt which one you are sitting at, so assume you are on a buffered one.)

Get back to the `A>` prompt before you quit or copy the image. The moment CP/M asks you for a keystroke, the buffer has landed. Kill the program the instant a file operation appears to have finished and the last track never reached the disk. The disks chapter has the detail.

The disk image changed and I wanted it not to

There is no undo. **The image is mounted read/write, like a real machine** — a CP/M that cannot write its disk cannot save your work.

Two answers, and take one *before* you experiment:

```
altairsim> MOUNT dsk0:drive0 file.dsk RO
```

`RO` refuses every write at the controller, so your file is safe whatever the guest does. It is for a disk you mean to **read**: the guest is never told the disk is protected — the controller has no bit that says so and CP/M has no error that means it — so a program that sets out to write to an `RO` disk will not fail gracefully. The disks chapter explains why.

or copy the folder. It is a directory with a machine file and an image in it, and it boots from anywhere:

```
$ cp -R examples/cpm my-cpm
```

`MOUNT` says "no such file" and the file is right there

Look at *where you typed it from*.

A path you TYPE is relative to YOUR SHELL. A path inside a machine file is relative to THAT MACHINE FILE. Those are two different directories, and one of them is not the one you are thinking about.

This is deliberate, and it is the reason an example folder boots from anywhere: the machine file says `cpm22.dsk` and means *the image next to me*, so you can move the folder, or run it from three levels up, and it still finds its disk. If a typed path resolved the same way, you could not mount a file from your own working directory without knowing where the machine file happened to live.

Every prompt ends in a garbage character

```
MEMORY SIZ?
```

That is **MITS BASIC setting bit 7 of the last character of every message** as a string terminator, and your terminal printing the result. The `E` is there; it arrived as `C5` instead of `45`.

```
altairsim> CONSOLE strip7out=on
```

The real Teletype ignored bit 7 and printed the `E`. `strip7out` is your terminal doing the same. The serial chapter explains why the fix belongs on the console and **never** on the board.

Nothing appears when I type / the guest does not see my keys

Ask who has the keyboard.

```
altairsim> SHOW CONSOLE
```

Exactly one unit may hold the console. If the console is on a unit the guest is not reading, you are typing into a board nobody is listening to. Connect the right one — and note that connecting a second unit to `console` *steals* it from the first, and says so.

The machine runs impossibly fast — a cassette loads in one second

It is running **flat out**, which is the default. `clock_hz = 0` means "no divisor, go".

```
altairsim> SET cpu0 clock_hz=2000000
```

That is the real 2 MHz Altair, and it will give you the real 110-second cassette load. Both behaviours are correct; one of them is just a much longer lunch.

A file transfer keeps timing out — `PCGET`, `PCPUT`, XMODEM

Slow the machine down. Transfers with the outside world want the real crystal.

```
altairsim> SET cpu0 clock_hz=2000000
```

Here is why, because it is worth understanding once and it explains a whole class of symptom.

A guest program has no clock. It counts instructions. `PCGET` times a second the way every program of the period timed a second — by spinning in a loop and counting the trips. Its own source says so:

```
MSEC    lxi  d,(159 shl 8)   ;49 cycle loop, 6.272ms/wrap * 159 = 1 second
```

That arithmetic is *only* a second if the crystal is 2 MHz. `PCGET` waits three of them for a block header. Run the machine flat out and those three seconds — three seconds of **T-states** — are retired by your host in a few tens of **milliseconds**, while the sending program on the other end of the wire is still living in seconds of the ordinary kind. `PCGET` concludes the sender is dead, NAKs, purges the line, and gives up. Nothing is broken. The two ends are simply no longer using the same clock.

And that is the boundary, exactly: timing inside the machine is consistent at any speed, because everything in there counts the same T-states — which is why a cassette loads correctly flat out, the ACR and the tape agreeing with each other at whatever speed the pair of them run. A transfer to your host has **one end inside the machine and one end outside it**, and only the crystal makes those two agree.

So: flat out for everything the machine does to itself. The real 2 MHz for anything it does with you. Set it back afterwards if you like the speed — and note the built-in file-transfer board is not affected by any of this, having no timeouts to expire.

Two boards are fighting over a port

```
altairsim> SHOW BUS CONTENTION
```

names them. To see the whole map of who decodes what:

```
altairsim> SHOW BUS IO
```

On a real S-100 machine this was two cards strapped to the same address and a bus you could not trust. Here it is a list.

An `IN` from a port returns `FF` and I expected something

Nothing decodes that port. The bus floats high when no board is driving it, and `FF` is what a floating bus reads. It is not an error, and the machine will not tell you — a real one did not either.

To find out who *would* have answered, without running a cycle:

```
altairsim> WHO IO 10
```

And if the symptom is a **hang** — a machine you assembled yourself starts, prints nothing, and never comes back — the cause is often exactly this: it is polling a port for a board that is not there, reading `FF` forever. Arm the bus to say so:

```
altairsim> SET BUS UNCLAIMED=WARN
altairsim> RUN
warning: IN 10 -> FF at PC=0043: no board decodes port 0x10. it floated to 0xFF.
```

`WARN` prints the port and the address that reached for it and keeps running; `HALT` stops the machine right there, so `SHOW REG` and a `DUMP` show it exactly as it wedged. It watches I/O only — memory is normal to scan — and names each absent port once per run. The default is `SILENT`; you turn it on when you need it.

`RESET` did not clear memory

It is not supposed to.

`RESET` is the bus's RESET* line. It does what pulling that line did: the processor goes to zero, boards return to their power-on state. RAM is RAM; it was not cleared on the real machine and it is not cleared here.

`POWER` is the only thing that loses memory. That is the difference between pressing a button and pulling a plug, and the manual keeps it.

macOS refuses to run it

"*cannot be opened because the developer cannot be verified*".

```
$ xattr -dr com.apple.quarantine ./altairsim
```

Once. It is not a comment on the program; it is what macOS does to every unsigned binary that arrives in a zip.

If that prints nothing, it worked — and if the flag was never there (a zip fetched with `curl` or `scp` is not marked; only one a browser or a mail client downloaded is), it also prints nothing and succeeds. That is the whole reason for `-dr` over a plain `-d`, which announces an error when there is nothing to remove.

Glossary

ACIA — Asynchronous Communications Interface Adapter. The Motorola 6850 chip at the heart of the 88-2SIO serial board. It has two registers a program can see: a status register and a data register. Everything a program does with a serial port on this machine, it does through one of those.

ACR — Audio Cassette Recorder interface. The MITS 88-ACR: a card that wrote bits to an ordinary audio cassette and read them back. It is how you loaded BASIC if you could not afford a floppy, which in 1976 was most people.

ATTN — Attention. The key that stops a running guest and returns you to the monitor. It stops the machine but does not disturb it — not RESET, not POWER — so a bare `RUN` resumes at the instruction it was about to execute. It is `^E` by default, the host intercepts it before the guest sees the byte, and no guest program can take it from you. Move it with `CONSOLE attn=`.

backplane — The PCB with the connectors on it that every board plugs into. It is not itself one of them: nothing in a machine file fits a backplane, because the backplane is what a machine file describes the contents of. On an Altair it is the S-100 bus itself: eighteen slots, one hundred pins each, all wired in parallel. There is no chipset. The backplane *is* the machine.

bank switching — Making more memory than the processor can address by putting several banks at the same addresses and switching between them. An 8080 can address 64K and no more. A machine with 128K of RAM in it lies to the processor about which half it is looking at.

BDOS — Basic Disk Operating System. The middle layer of CP/M: files, directories, the console. A CP/M program asks the BDOS for things and does not know what hardware answers.

BIOS — Basic Input/Output System. The bottom layer of CP/M, and the only part that knows what machine it is on. Move CP/M to a new computer and the BIOS is what you rewrite; the BDOS and the CCP are untouched. It is also where a track buffer lives *if the author put one there* — which is why, on some CP/Ms and not others, a file can be "written" and not yet be on the disk. See the disks chapter.

board — Anything that plugs into the backplane: memory, a serial port, a disk controller, the front panel, the processor itself. It is the word this program uses everywhere — `BOARDS` lists them, `[[board]]` fits them in a machine file — and a board is the thing you add, remove, `SHOW` and `SET`. See also **card**, which means the same thing.

card — The same object as a **board**. The period hardware and its manuals said "card", and this manual keeps the word where the sentence is about the physical thing somebody bought, socketed chips into and set jumpers on. Everywhere else it says board.

CCP — Console Command Processor. The top layer of CP/M — the part that prints `A>` and runs what you type. It is deliberately expendable: a big program is allowed to overwrite it, and CP/M reloads it from disk afterwards. That reload is the warm boot.

contention — Two boards answering the same address. On a real S-100 machine both would drive the data bus at once and you would get a byte that is neither of theirs, intermittently, in a way that would take you a week. `SHOW BUS CONTENTION` names them instead.

CP/M — Control Program for Microcomputers. Digital Research's disk operating system, and the reason the 8080 mattered. Write a program for CP/M and it ran on every 8080 machine ever built, whoever built it — the first time that was true of anything.

CUTS — Computer Users Tape System. The Sol-20's cassette scheme, and a faster cousin of Kansas City: at its default 1200 baud it drops an octave to **1200/600 Hz** tones and packs a bit into one cycle (a "1") or a half cycle (a "0"); its slower 300-baud mode is Kansas City proper, 2400/1200 Hz. The Sol's UART does both, and the guest picks which at `OUT 0FAh` D5.

DBL — Disk Boot Loader. The boot PROM on the MITS floppy controller, at `FF00`. It reads one sector off track 0 and jumps into it. **There is no `BOOT` command on an Altair** — you set the address switches to `FF00`, press EXAMINE to load them into the program counter, then press RUN, and this is the thing you are running.

decode — What a board does when it recognises an address as its own and answers. A board that does not decode an address stays silent and lets somebody else have it. Which board decodes which address is the entire question of how an S-100 machine is put together.

DMA — Direct Memory Access. A board taking the bus away from the processor and reading or writing memory itself, without the processor's help. Faster, and the origin of some spectacular bugs.

endpoint — In this program, the thing on the far end of a serial unit's cable: `console`, `null`, `loopback`, a TCP socket, or a real serial port. The serial chapter has the complete list; there are no others.

floating bus — What the data bus reads when nothing is driving it. On the S-100 it floats high, so an `IN` from a port no board decodes returns `FF`. That is not an error. That is the absence of a board, and it looks like `FF`.

front panel — The switches and lamps on the front of the Altair. The address switches, the data lamps, DEPOSIT, EXAMINE, RUN, STOP, RESET. Before there was a terminal, this *was* the user interface, and you toggled your bootstrap in through it one byte at a time. In this program it is a board like any other.

FSK — Frequency Shift Keying. Encoding bits as two audible tones — one for a zero, one for a one. It is how the ACR got data onto a cassette, and it is why a loading tape sounds the way it does.

The tones were **not** standard across machines, which matters more than it sounds: the 88-ACR uses 2400/1850 Hz and holds a tone for the whole bit; Kansas City counts *whole cycles* of 2400/1200 Hz per bit, and CUTS at its 1200-baud default does the same an octave down (1200/600 Hz, a half cycle for a

"0"). A board can only read the modulation its own modem was built for, so mounting a recording in the wrong one is refused rather than decoded. See the tapes chapter.

hard-sector — A floppy where the sector boundaries are marked by physical holes punched in the disk, and the controller counts them going past. The MITS floppies are hard-sectored. A soft-sectored disk has one hole and finds its sectors by reading marks written in the data, which is the arrangement that won.

IntAck — Interrupt Acknowledge. The bus cycle the 8080 runs when it accepts an interrupt. The interrupting board puts one instruction on the data bus during that cycle, and the processor executes it. Almost always an `RST`.

Kansas City standard — The 1975 agreement on how microcomputers should record data on ordinary audio cassettes, named for the meeting that settled it. A one is eight cycles of 2400 Hz, a zero is four cycles of 1200 Hz, and both take the same time — which is the whole point: a receiver counts cycles instead of trusting a clock, so a tape that plays 5% slow still reads. 300 baud. See also **CUTS**, its faster variant, and **FSK**.

MCP — Model Context Protocol. The protocol an AI assistant uses to call structured tools. `altairsim --mcp` speaks it, so an assistant can drive the machine directly. See the MCP chapter.

monitor — The `altairsim>` prompt. Not a program running inside the machine — it is you, standing in front of it, with a much better front panel than MITS shipped.

PHANTOM* — An S-100 line that tells memory boards to shut up. Pull it, and a ROM can answer at an address a RAM board also decodes, without contention. It is how a boot PROM can sit on top of RAM and then get out of the way. The asterisk means the line is active low, and on this bus most of them are.

pINT — The S-100 interrupt request line. Any board may pull it. The processor notices, if interrupts are enabled, and runs an IntAck cycle to find out who and why.

PROM — Programmable Read-Only Memory. A chip with a program burned into it that survives power-off. The Altair's boot loader lives in one, which is the only reason a disk machine can start at all: something has to already be there to read the disk.

RST — Restart. A one-byte 8080 instruction that calls a fixed low address — `RST 0` through `RST 7`, at `0000`, `0008`, and so on to `0038`. One byte, so it fits in an IntAck cycle, which is exactly what it is for.

S-100 — The bus. One hundred pins, eighteen slots, designed by MITS for the Altair and then adopted by everyone. It was the first open standard in personal computing, mostly by accident, and it is the reason a card from one company worked in another company's machine.

sector — The smallest chunk of a disk you can read or write. On these floppies, 137 bytes of which 128 are yours.

SENSE switches — The top eight address switches (A8–A15) on the front panel, readable by a program as an input port. Software used them for configuration before there was anywhere else to put it: which port is the console, how much memory to use, whether to load from tape. Half the boot procedures in this manual begin with a sense switch setting.

T-state — One clock cycle. Every 8080 instruction costs a known number of them, and that is how this program knows what time it is. At 2 MHz a T-state is 500 nanoseconds, and a cassette takes 110 seconds because it takes 220 million of them.

TDRE — Transmit Data Register Empty. The bit in the 6850's status register that means "the board has room for another character". A program that wants to print polls it until it sets. A serial port connected to `null` sets it forever, which is why writing to nothing works fine.

track — One concentric ring of sectors on a disk. The head steps in and out to reach a track and does not move again to reach the sectors on it, which is why a BIOS that buffers buffers a whole track at a time: having paid for the seek, it may as well have the lot.

UART — Universal Asynchronous Receiver/Transmitter. The chip that turns a byte into a sequence of bits on a wire and back again. The ACIA is one.

unit — One channel on a board that moves characters — the socket on the back. A 2SIO has two of them, `a` and `b`, and they are connected independently. `CONNECT sio0:b` names the board and then the unit.

VI0–VI7 — The eight vectored interrupt lines on the S-100 bus, served by the 88-VI/RTC board. **VI0 is the highest priority.** A board pulling VI*n* gets `RST` *n*, and the interrupt board sorts out who wins when two pull at once.

warm boot — CP/M reloading its own top layer (the CCP) from the disk, because the program that just finished was allowed to overwrite it. `^C` at the `A>` prompt does one deliberately. It is not a reset; the machine never stopped.

Boards and their parameters

Every key below is a key you may write in a machine file, and the *same* key you may `SET` at the monitor prompt. That is not a coincidence and it is not a convention: a board's properties **are** its TOML schema, so there is nothing here that could disagree with the program.

Numbers follow the one rule: **on the wire → hex, never on the wire → decimal.** A port is hex; a baud rate and a drive count are decimal. The defaults below are printed in each property's own base.

The catalogue is **grouped by function** — CPU, memory, disk, serial, and so on — and within a group the boards are in **alphabetical order**.

CPU

Type	What it is
`6800`	Altair 680b CPU board: a Motorola 6800 at 500 KHz. Decodes nothing -- it drives the bus. The 88-CPU's twin, one core down, with memory-mapped I/O
`8080`	MITS 88-CPU: an 8080A at 2 MHz. Decodes nothing -- it drives the bus
`8085`	Generic 8085 CPU board. Decodes nothing -- it drives the bus. The 88-CPU's twin, with an 8085 core (RIM/SIM + TRAP/RST 5.5/6.5/7.5)
`z80`	Generic Z80 CPU board. Decodes nothing -- it drives the bus. The 88-CPU's twin, with a Z80 core

Memory

Type	What it is
`bankmem`	S-100 bank-switched RAM. One card, four decoders (card=vector|cromemco64kz|northstar|expandoram2): a write-only select port swaps which RAM plane(s) drive the bus. Each card owns its own decode -- one-hot select (Vector 40), 8-bit bank mask (Cromemco 40), on/off+one-hot toggle (North Star C0), or PROM page-select (ExpandoRAM II FF, approximated)
`memory`	RAM/ROM board: a list of regions and PHANTOM* -- plain, unbanked memory (bank switching is its own board, `bankmem`)
`v2z80rom`	S100Computers V2 Z80 CPU board -- its onboard paged monitor EEPROM (the Z80 itself is board 'z80cpu'). An 8K 28C64 at F000-FFFF holding two 4K pages, builtin:master0 (low) / master1 (high), selected by OUT D3H bit1 (bit0=1 inactivates the EEPROM so RAM shows through). Shadows RAM in its window while enabled. Cold-start the MASTER monitor with startup=["RUN F000"]; the 'I' command boots CP/M 3 off a dualsd card

Disk

Type	What it is
`16fdc`	Cromemco 16FDC: WD FD1793 soft-sector floppy (single + double density), up to 4 drives. Disk registers at 30-34, a TMS 5501 console UART at 00-09 (unit 'tty'), and a 4K RDOS 2.52 boot PROM at C000 (OUT 40H banks it out, RESET restores it). Boots CDOS
`64fdc`	Cromemco 64FDC: the 16FDC's 1983 successor -- same FD1793 + TMS 5501, carrying an 8K RDOS 3.12 boot PROM at C000-DFFF (OUT 40H banks it out, RESET restores it). Boots CDOS
`dcdd`	MITS 88-DCDD: 8" hard-sector floppy, up to 16 drives. Three ports at BASE+0..2. INVERTED status bits
`dualide`	S100Computers IDE-AB (CF): the IDE/CompactFlash half of the IDE+ESP32 combination board -- two CF sockets (drives 0/1 = A:/B:) for CP/M 3. Five 8255 ports at BASE+0..4 (default 30): A/B data, C control lines, mode config, drive select. Programmed-I/O ATA register engine (LBA read/write, 512-byte sectors). No boot PROM -- the CPU board's monitor boots CP/M from the CF. Mounts the SAME card image as dualsd (a .img with a .geo geometry sidecar); pair with dualsd for the full A:/B:+C:/D: system
`dualsd`	S100Computers Dual SD: two microSD sockets (drives 0/1) presented as raw 512-byte-sector CF/SD cards, for CP/M 3. Two ports at BASE+0..1 (default 80): status/command + data. Programmed-I/O command/handshake engine (33H-lead + 8 commands). No boot PROM -- the CPU board's monitor loads CP/M from track 0. Mount a card image (a .img with a .geo geometry sidecar)
`hdsk`	MITS 88-HDSK Datakeeper: Pertec hard disk, 256-byte sectors from a linear .DSK. Eight ports at BASE+0..7 (default A0). Command/handshake protocol, four page buffers
`icom`	iCOM FD3712/FD3812 8" floppy: a programmed-I/O command/handshake controller on the S-100 Interface board. Two ports at BASE+0..1 (default C0) plus a boot PROM and 6810 scratch RAM in high memory (rom=builtin:icom-fd3712-cpm | icom-fd3712-fdos | icom-fd3812-cpm). Boots CP/M 2.2 (single and double density) and FDOS. Up to 4 drives
`mds`	MITS 88-MDS: 5.25" minidisk, 4 drives. Same three ports as the dcdd -- but 300 RPM, 64 us/byte, and a motor that stops after 6.4 s
`tarbell`	Tarbell #1011: single-density WD FD1771 floppy, up to 4 drives. Eight ports at BASE+0..7 (default F8). Carries a 32-byte boot PROM that shadows 0000 over PHANTOM* -- boots CP/M automatically at reset (bootstrap=on)
`tarbelldd`	Tarbell #2022: double-density WD FD1791 floppy (mixed-density media, SD track 0), up to 4 drives. The single-density card's twin with a bitmap OUT-FC latch and a port-FD DMA/ext-addr register. Same 32-byte boot PROM
`versafloppy`	SD Systems VersaFloppy I/II: WD FD177x soft-sector floppy, up to 4 drives. Eight ports at BASE+0..7 (default 60). variant=vfi (FD1771, single density) | vfii (FD1791, single+double). Boots SDOS with the SBC-200 + DDBIOS

Serial

Type	What it is
`2sio`	MITS 88-2SIO: two 6850 ACIAs, units 'a' and 'b'. Four ports at BASE+0..3
`680io`	Altair 680b onboard I/O: a 6850 ACIA console ('tty') at F000/F001 and the config-strap read port at F002. Memory-mapped
`gsio`	Generic SIO: a strap-configurable serial board with TWO independent channels, units 'a' and 'b' (configure each under [board.unit.a] / [board.unit.b]). Basic transmit/receive only -- no programmable word length, parity, stop bits or framing/overrun status; a specific card that needs those is a separate emulated board. Per channel you strap: a status/control port (status_port -- read synthesizes DAV/TBMT at bit positions you pick, write is discarded) and a data port (data_port); one inverter_gate knob inverts both status bits together. Built-in profiles preset the straps: profile=sior0 (MITS SIO Rev 0, the default) | tuart (Cromemco TU-ART) | imsai-sio2 | compupro-if2 (CompuPro Interfacer II) | compupro-ss1 (CompuPro System Support 1). Channel a defaults to ports 0/1, b to 2/3. Polled, no interrupts. CONNECT each channel to a file, socket, serial port, in:/out:
`io4`	SSM IO-4 (2P+2S): the real Solid State Music board -- two full-duplex serial channels AND a four-port parallel section. Serial units 'a'/'b' are real 1602-family UARTs with programmable word length (data_bits 5-8), parity and stop bits (unlike the generic gsio), plus the full status-word strap-up (stat_* map, invert_status, port_reversal) and named profiles (default altair-rev1). A 4-port block set by switch S3 (default 0-3): Serial A status/data at BASE+0/+1, Serial B at BASE+2/+3. Parallel units 'pa'/'pb' are 8212 latched ports (input latch + service request, output latch) on their own 2-port block set by switch S4 (par_port, default 4-5): Parallel A at PAR+0, B at PAR+1; a byte the far end sends is strobed in, a write latches out, and the §3.2.2 status/data console flag is strappable (dav_bit/dav_source/dav_active_low). If the two blocks overlap, neither section answers the shared ports. Each serial channel's receive (DAV) and transmit (TBMT) and each parallel input (service request) can raise an interrupt, strapped on header W4 to a VI line, pin 73 (int), or none (rx_int/tx_int per serial channel, int per parallel port) -- there is no software enable, the strap is the enable, and a parallel input interrupt rises even when the port is not addressed. Configure each unit under [board.unit.a] / [board.unit.pa] etc.; CONNECT each to a file, socket, serial port, in:/out:
`pmmi`	PMMI MM-103: Bell 103 modem on an S-100 card, unit 'line'. Four ports at BASE+0..3 (default C0), read/write different registers. Transmit/receive over a ByteStream; CONNECT it to in:/out: files. No dialer; modem status is a fixed stub
`propio`	S100Computers Console IO Board (Parallax-Propeller console), unit 'serial'. A strap-configurable serial subtype preset to the board's documented convention: status/data at 00/01, RX-ready = status bit 1, TX-ready = status bit 2, both active high. Every strap (status_port/data_port/dav/tbmt/inverter_gate) is still overridable -- the real board is jumpered. Polled, no interrupts. CONNECT it to a file, socket, serial port, in:/out:
`sbc`	SD Systems SBC-100/200: Z80 single-board computer. One 8-port block (78-7F): Intel 8251 console (unit 'tty', data 7C / status 7D, RxD->/DSR auto-baud for MSMONR21), Z80-CTC (78-7B) whose ch1 raises a mode-2 keyboard interrupt (vector 0x82) off the 8251 RxRDY, and a parallel port (7E/7F) whose OUT 7F bit 1 switches the onboard PROM out. Optional onboard boot PROM via [[board.socket]] (at+mount). variant=sbc100|sbc200
`sio`	MITS 88-SIO: one COM2502 UART, unit 'tty'. Two ports at BASE+0..1. INVERTED status bits

`turnkey`	MITS 8800b Turnkey Module: phantom boot PROM (FC00-FFFF), integrated 6850 SIO (unit 'tty', default 0x10), sense switches at FF, and the Auto-Start JMP jam. Sockets via [[board.socket]]

Tape

Type	What it is
`680kcacr`	Altair 680b KCACR audio-cassette interface: a 1602-family UART recording Kansas City Standard FSK, memory-mapped at F010 (status/control) and F011 (data), active-LOW. Adds software motor control (control D7=on, D6=off) and interrupt-driven transfer (D0/D1 enables pull the 6800 IRQ). Reuses the 88-ACR tape machinery -- MOUNT a tape, WIND/REWIND it
`acr`	MITS 88-ACR: cassette. An 88-SIO B + an FSK modem, unit 'tape'. Brings the WIND/REWIND/EXTRACT verbs and a tape counter
`uio`	MITS 88-UIO: serial + cassette on one board. A 6850 (unit 'serial', default 0x10) and an 88-ACR cassette section (unit 'tape', default 0x06) with motor control and a SW-1 MITS/Kansas-City modulation switch. Defaults reproduce the standard 0x10 + 0x06 layout

Parallel and printer

Type	What it is
`4pio`	MITS 88-4PIO: up to four 6820 PIAs, sections ja/jb.. per port. 16 ports from BASE (default 20). Software-set direction; CONNECT each section
`680uio`	Altair 680b Universal I/O: a second 6850 ACIA serial port ('serial') and a 6820 PIA parallel port (sections 'p1a/p1b', 'p2a/p2b' with pias=2) in an S9-relocatable window (default base F000: serial F006/F007, PIA F008-F00F), plus fixed switch inputs at F003 and a non-latched output at F010-F013. Memory-mapped, active-high
`c700`	MITS 88-C700: Centronics line-printer controller, unit 'prn'. Two ports at BASE+0..1 (default 02). Output-only; CONNECT it to a file, a socket, or a real printer queue
`d7a`	Cromemco D+7A: analog + parallel I/O. Eight ports from BASE (default 18): one parallel port + seven two's-complement A/D-in/D/A-out channels. Reads 1-2 JS-1 joysticks from the host
`lpc`	MITS 88-LPC: 88-LP line-printer controller, unit 'prn'. Two ports at BASE+0..1 (default 02). Line-buffered: 6-bit codes + PRINT/LINE FEED/CLEAR. CONNECT it to a file, a socket, or a real printer queue
`pio`	MITS 88-PIO: 8-bit parallel port, units 'out'/'in'. Two ports at BASE+0..1 (default 04). CONNECT a printer, a keyboard, a socket

Video

Type	What it is
`dazzler`	Cromemco Dazzler: color graphics from a framebuffer in main RAM. Two ports at BASE+0..1 (default 0E): control/status and format. 32x32 to 128x128, 16 colors/greys. Needs a Display
`vdb8024`	SD Systems VDB-8024: an 80x24 video terminal on one board -- the video console for an SBC-100/200 (the alternative to the 8251). Two I/O ports at BASE+0..1 (default 00): status/keyboard/display. Unit 'keyboard' (CONNECT). Optional keyboard-strobe interrupt strap (interrupt=vi0..vi7) for the SBC-200's CTC to vector -- what the SD video CBIOS needs; polled by default. Boots sdmonv21. Needs a Display
`vdm1`	Processor Technology VDM-1: memory-mapped 16x64 video, screen RAM at BASE (default CC00), scroll/status port (default CC). Needs a Display

Systems

Type	What it is
`sol`	Processor Technology Sol-PC I/O: serial, keyboard, parallel, CUTS tape as one board. Seven ports F8..FE. Units serial/printer/keyboard (CONNECT) and tape1/tape2 (MOUNT). Brings the WIND/REWIND/EXTRACT verbs and a tape counter

PROM programmer

Type	What it is
`pb1`	SSM PB1: 2708/2716 EPROM programmer + on-board EPROM board. A 4K programming-socket window (default D000, sockets U22=2708/U23=2716) and one control port (default 10): OUT arms the board and picks the chip (D0=2708, D1=2716), then a window write burns a byte and a window read disarms it. Save the burn to a host hex file with `SAVE file window`. Optional read-only on-board EPROM area via [[board.prom]] (at + mount). The board ships no firmware -- run any 2708/2716 burner; SSM's own from the PB1 manual is in examples/pb1

Other

Type	What it is
`fp`	Altair front panel: the SENSE switches a guest reads at IN 0FFH -- a configured byte (SET fp0 sense= or TOML), not toggled here. No OUT
`hostbridge`	Host Bridge: guest <-> host file transfer, sandboxed. OUR OWN BOARD, not a period one. Two ports at BASE+0..1. R.COM/W.COM/HDIR.COM
`ss1`	CompuPro System Support 1: multifunction S-100 board. Dual 8259A interrupt controllers in a master/slave cascade (master/slave at base+0..+3; master watches VI0-6 and drives pin 73, slave takes the timer OUTs and the UART's Rx/TxRDY), an 8253 interval timer (three counters + control at base+4..+7, 2 MHz clock), the OKI MSM5832 battery-backed real-time clock/calendar (command/data at base+10/+11) and a 2651 UART serial channel (base+12..+15); base default 50H. The 9511/9512 math socket is unpopulated
`virtc`	MITS 88-VI/RTC: vectored interrupts (VI0-VI7 -> RST n) and a real-time clock. One port at FE

CPU

`6800`

Altair 680b CPU board: a Motorola 6800 at 500 KHz. Decodes nothing -- it drives the bus. The 88-CPU's twin, one core down, with memory-mapped I/O

Units: `6800` (cpu)

Board properties

Key	Kind	Default	Legal	Meaning
`clock_hz`	int	`0`	`0` .. `100000000`	Crystal on the board. 0 runs flat out -- as fast as the host can.
`idle`	bool	`true`	`on` | `off`	Stand down when the guest is only polling an empty keyboard. On by default -- the guest cannot tell, and a prompt stops burning a core.
`achieved_hz`	int	—	—	LIVE: T-states per real second the run loop last reached -- the crystal you got, beside the one you asked for. Read-only; 0 until it has run. **(read-only — not a key you may set)**

`8080`

MITS 88-CPU: an 8080A at 2 MHz. Decodes nothing -- it drives the bus

Units: `8080` (cpu)

Board properties

Key	Kind	Default	Legal	Meaning
`clock_hz`	int	`0`	`0` .. `100000000`	Crystal on the board. 0 runs flat out -- as fast as the host can.
`idle`	bool	`true`	`on` | `off`	Stand down when the guest is only polling an empty keyboard. On by default -- the guest cannot tell, and a prompt stops burning a core.
`achieved_hz`	int	—	—	LIVE: T-states per real second the run loop last reached -- the crystal you got, beside the one you asked for. Read-only; 0 until it has run. **(read-only — not a key you may set)**

`8085`

Generic 8085 CPU board. Decodes nothing -- it drives the bus. The 88-CPU's twin, with an 8085 core (RIM/SIM + TRAP/RST 5.5/6.5/7.5)

Units: `8085` (cpu)

Board properties

Key	Kind	Default	Legal	Meaning
`clock_hz`	int	`0`	`0` .. `100000000`	Crystal on the board. 0 runs flat out -- as fast as the host can.
`idle`	bool	`true`	`on` | `off`	Stand down when the guest is only polling an empty keyboard. On by default -- the guest cannot tell, and a prompt stops burning a core.
`achieved_hz`	int	—	—	LIVE: T-states per real second the run loop last reached -- the crystal you got, beside the one you asked for. Read-only; 0 until it has run. **(read-only — not a key you may set)**

`z80`

Generic Z80 CPU board. Decodes nothing -- it drives the bus. The 88-CPU's twin, with a Z80 core

Units: `z80` (cpu)

Board properties

Key	Kind	Default	Legal	Meaning
`clock_hz`	int	`0`	`0` .. `100000000`	Crystal on the board. 0 runs flat out -- as fast as the host can.
`idle`	bool	`true`	`on` | `off`	Stand down when the guest is only polling an empty keyboard. On by default -- the guest cannot tell, and a prompt stops burning a core.
`achieved_hz`	int	—	—	LIVE: T-states per real second the run loop last reached -- the crystal you got, beside the one you asked for. Read-only; 0 until it has run. **(read-only — not a key you may set)**

Memory

`bankmem`

S-100 bank-switched RAM. One card, four decoders (card=vector|cromemco64kz|northstar|expandoram2): a write-only select port swaps which RAM plane(s) drive the bus. Each card owns its own decode -- one-hot select (Vector 40), 8-bit bank mask (Cromemco 40), on/off+one-hot toggle (North Star C0), or PROM page-select (ExpandoRAM II FF, approximated)

Board properties

Key	Kind	Default	Legal	Meaning
`card`	enum	`vector`	`vector` | `cromemco64kz` | `northstar` | `expandoram2`	which banked card this is -- each owns its own decode: vector | cromemco64kz | northstar | expandoram2
`port`	int	`0x40`	`0x0` .. `0xFF`	the write-only bank-select port. Card default (vector/cromemco 40, northstar C0, expandoram2 FF); overridable, as the real boards relocate it
`banks`	int	`8`	`1` .. `10`	how many switchable banks/planes/pages this subsystem carries (one per board). Card-capped: vector/cromemco 8, northstar 6, expandoram2 10. The common region (partition) is not a bank. Setting `ram` derives this
`partition`	enum	`none`	`none` | `ex48` | `ex32`	ExpandoRAM II common-memory partition (the EX-48/EX-32 PROM): none | ex48 | ex32. ex48 = 48K banked (0000-BFFF) + 16K common (C000-FFFF); ex32 = 32K banked + 32K common (8000-FFFF). The common region is identical in every bank -- what a banked CP/M's resident OS and bank-switch routine live in. `none` (default) is a whole 64K plane
`ram`	int	`512`	`1` .. `640`	total board RAM in KB. Sets the bank count from the current partition: with ex48, 256 -> 5 banks; with none (whole plane), 256 -> 4 banks of 64K. The real ExpandoRAM II is 64K (16K chips) or 256K (64K chips)
`honors_phantom`	enum	`none`	`none` | `read` | `all`	A JUMPER. Another board pulls PHANTOM* (a boot PROM shadowing the RAM this card sits under) -- do I switch off? none | read | all. `read` keeps answering writes so a cold-boot loader falls through to this RAM while the PROM shadows reads (an ExpandoRAM II under the SBC-200 boot PROM). Default none -- a plain banked-RAM machine has no PROM overlay
`active`	string	—	—	the live bank(s) right now -- the guest sets this by writing the select port. Read-only here **(read-only — not a key you may set)**
`fill`	enum	`random`	`zero` | `random`	RAM contents at power-on: zero | random (real RAM is not zeroed)
`seed`	int	`1`	any	seed for fill=random -- the same seed fills RAM the same way at every POWER, so a run is repeatable

`memory`

RAM/ROM board: a list of regions and PHANTOM* -- plain, unbanked memory (bank switching is its own board, `bankmem`)

`[[board.region]]` — a list you may add

Key	Kind	Legal	Meaning
`type`	enum	`ram` | `rom`	RAM, or ROM (which needs a `mount`, unless you want an empty socket)
`at`	int	`0x0` .. `0xFFFF`	Where it starts. An address: 0000, F800
`size`	int	`1` .. `65536`	How much. Decimal, and it takes a suffix: 48K, 1024, 2M
`mount`	string	text	The ROM image. A file (relative to THIS FILE), or builtin:

Board properties

Key	Kind	Default	Legal	Meaning
`honors_phantom`	enum	`all`	`none` | `read` | `all`	A JUMPER. Another board pulls PHANTOM* -- do I switch off? none | read | all
`phantom`	enum	`all`	`none` | `read` | `all`	What I ASSERT over my rom regions: none | read | all
`fill`	enum	`random`	`zero` | `random`	RAM contents at power-on: zero | random (real RAM is not zeroed)
`seed`	int	`1`	any	Seed for fill=random. The same seed fills RAM the same way at every POWER, so a run is repeatable; change it for a different junk pattern
`pages`	string	—	—	the composite page map -- which pages this board answers for. Derived from the regions you declared **(read-only — not a key you may set)**

`v2z80rom`

S100Computers V2 Z80 CPU board -- its onboard paged monitor EEPROM (the Z80 itself is board 'z80cpu'). An 8K 28C64 at F000-FFFF holding two 4K pages, builtin:master0 (low) / master1 (high), selected by OUT D3H bit1 (bit0=1 inactivates the EEPROM so RAM shows through). Shadows RAM in its window while enabled. Cold-start the MASTER monitor with startup=["RUN F000"]; the 'I' command boots CP/M 3 off a dualsd card

Board properties

Key	Kind	Default	Legal	Meaning
`port`	int	`0xD3`	`0x0` .. `0xFF`	Page/ROM-control latch (D3H): bit1 = EEPROM A12 page select, bit0 = ROM inactivate

Disk

`16fdc`

Cromemco 16FDC: WD FD1793 soft-sector floppy (single + double density), up to 4 drives. Disk registers at 30-34, a TMS 5501 console UART at 00-09 (unit 'tty'), and a 4K RDOS 2.52 boot PROM at C000 (OUT 40H banks it out, RESET restores it). Boots CDOS

Units: `tty` (serial, CONNECT), `drive0` (disk, MOUNT), `drive1` (disk, MOUNT), `drive2` (disk, MOUNT), `drive3` (disk, MOUNT)

`[[board.drive]]` — a list you may add

Key	Kind	Legal	Meaning
`unit`	int	`0` .. `3`	Which drive (0..3)
`mount`	string	text	The disk image to put in it. Relative to THIS FILE.
`readonly`	bool	`on` | `off`	Write-protect the disk. The drive senses it, so the guest is never told *(also `writeprotect`)*

Board properties

Key	Kind	Default	Legal	Meaning
`bootstrap`	bool	`true`	`on` | `off`	The BOOT/MON strap. On (default): the RDOS ROM is mapped at C000 and ¬BOOT reads low, so RDOS boots the disk. Off: the ROM still answers but ¬BOOT reads high (the monitor prompt instead of an auto-boot)
`drives`	int	`4`	`1` .. `4`	Drives on the controller (A-D, one-hot select DS4-DS1)

Unit `tty` — `[board.unit.tty]`

Key	Kind	Default	Legal	Meaning
`baud`	int	`9600`	`0` .. `76800`	Line rate. The guest sets it by writing the baud register; this seeds it and shows the effective rate (0 = the register selected no rate)
`rate`	string	`full`	text	Console speed: full (as fast as the guest reads) | real (wall-clock baud)
`dcd`	enum	`ground`	`ground` | `wired`	/DCD pin: grounded on the card, or wired to the connector
`cts`	enum	`ground`	`ground` | `wired`	/CTS pin: grounded on the card, or wired -- and then it gates the transmitter
`lines`	string	—	—	Live pin state (read-only). CAPITALS = asserted. in: DCD CTS **(read-only — not a key you may set)**
`connect`	string	`null`	text	The endpoint on the other end of the line (CONNECT sets this)

`64fdc`

Cromemco 64FDC: the 16FDC's 1983 successor -- same FD1793 + TMS 5501, carrying an 8K RDOS 3.12 boot PROM at C000-DFFF (OUT 40H banks it out, RESET restores it). Boots CDOS

Units: `tty` (serial, CONNECT), `drive0` (disk, MOUNT), `drive1` (disk, MOUNT), `drive2` (disk, MOUNT), `drive3` (disk, MOUNT)

`[[board.drive]]` — a list you may add

Key	Kind	Legal	Meaning
`unit`	int	`0` .. `3`	Which drive (0..3)
`mount`	string	text	The disk image to put in it. Relative to THIS FILE.
`readonly`	bool	`on` | `off`	Write-protect the disk. The drive senses it, so the guest is never told *(also `writeprotect`)*

Board properties

Key	Kind	Default	Legal	Meaning
`bootstrap`	bool	`true`	`on` | `off`	The BOOT/MON strap. On (default): the RDOS ROM is mapped at C000 and ¬BOOT reads low, so RDOS boots the disk. Off: the ROM still answers but ¬BOOT reads high (the monitor prompt instead of an auto-boot)
`drives`	int	`4`	`1` .. `4`	Drives on the controller (A-D, one-hot select DS4-DS1)

Unit `tty` — `[board.unit.tty]`

Key	Kind	Default	Legal	Meaning
`baud`	int	`9600`	`0` .. `76800`	Line rate. The guest sets it by writing the baud register; this seeds it and shows the effective rate (0 = the register selected no rate)
`rate`	string	`full`	text	Console speed: full (as fast as the guest reads) | real (wall-clock baud)
`dcd`	enum	`ground`	`ground` | `wired`	/DCD pin: grounded on the card, or wired to the connector
`cts`	enum	`ground`	`ground` | `wired`	/CTS pin: grounded on the card, or wired -- and then it gates the transmitter
`lines`	string	—	—	Live pin state (read-only). CAPITALS = asserted. in: DCD CTS **(read-only — not a key you may set)**
`connect`	string	`null`	text	The endpoint on the other end of the line (CONNECT sets this)

`dcdd`

MITS 88-DCDD: 8" hard-sector floppy, up to 16 drives. Three ports at BASE+0..2. INVERTED status bits

Units: `drive0` (disk, MOUNT), `drive1` (disk, MOUNT), `drive2` (disk, MOUNT), `drive3` (disk, MOUNT)

`[[board.drive]]` — a list you may add

Key	Kind	Legal	Meaning
`unit`	int	`0` .. `3`	Which drive on the daisy chain
`mount`	string	text	The disk image to put in it. Relative to THIS FILE.
`readonly`	bool	`on` | `off`	Write-protect the disk. The DRIVE senses it, so the guest is never told: it writes, the head is inhibited, the bytes go nowhere *(also `writeprotect`)*
`media`	enum	`8in` | `fdc8mb`	Force the format instead of probing the image's size
`create`	bool	`on` | `off`	Make the disk file (empty) if it is not there, then mount it -- a fresh disk to FORMAT. Pair with `media` to pick its geometry

Board properties

Key	Kind	Default	Legal	Meaning
`port`	int	`0x8`	`0x0` .. `0xFD`	Base address. The board decodes three ports: BASE+0 .. BASE+2
`drives`	int	`4`	`1` .. `16`	Drives on the daisy chain
`interrupt`	enum	`none`	`none` | `int` | `vi0` | `vi1` | `vi2` | `vi3` | `vi4` | `vi5` | `vi6` | `vi7`	Where the card's interrupt is soldered *(interrupt strap)*

`dualide`

S100Computers IDE-AB (CF): the IDE/CompactFlash half of the IDE+ESP32 combination board -- two CF sockets (drives 0/1 = A:/B:) for CP/M 3. Five 8255 ports at BASE+0..4 (default 30): A/B data, C control lines, mode config, drive select. Programmed-I/O ATA register engine (LBA read/write, 512-byte sectors). No boot PROM -- the CPU board's monitor boots CP/M from the CF. Mounts the SAME card image as dualsd (a .img with a .geo geometry sidecar); pair with dualsd for the full A:/B:+C:/D: system

Units: `drive0` (disk, MOUNT), `drive1` (disk, MOUNT)

`[[board.drive]]` — a list you may add

Key	Kind	Legal	Meaning
`unit`	int	`0` .. `1`	Which CF socket (0 = drive A:, 1 = drive B:)
`mount`	string	text	The card image (a .img with a sibling .geo geometry) to put in it. Relative to THIS FILE.
`readonly`	bool	`on` | `off`	Write-protect the card *(also `writeprotect`)*

Board properties

Key	Kind	Default	Legal	Meaning
`port`	int	`0x30`	`0x0` .. `0xFB`	Base address. The board decodes five 8255 ports: BASE..BASE+4 (A,B,C,cfg,drive)

`dualsd`

S100Computers Dual SD: two microSD sockets (drives 0/1) presented as raw 512-byte-sector CF/SD cards, for CP/M 3. Two ports at BASE+0..1 (default 80): status/command + data. Programmed-I/O command/handshake engine (33H-lead + 8 commands). No boot PROM -- the CPU board's monitor loads CP/M from track 0. Mount a card image (a .img with a .geo geometry sidecar)

Units: `drive0` (disk, MOUNT), `drive1` (disk, MOUNT)

`[[board.drive]]` — a list you may add

Key	Kind	Legal	Meaning
`unit`	int	`0` .. `1`	Which SD socket (0 = drive 1 / C:, 1 = drive 2 / D:)
`mount`	string	text	The card image (a .img with a sibling .geo geometry) to put in it. Relative to THIS FILE.
`readonly`	bool	`on` | `off`	Write-protect the card *(also `writeprotect`)*

Board properties

Key	Kind	Default	Legal	Meaning
`port`	int	`0x80`	`0x0` .. `0xFE`	Base address. The board decodes two ports: BASE (status/command) and BASE+1 (data)

`hdsk`

MITS 88-HDSK Datakeeper: Pertec hard disk, 256-byte sectors from a linear .DSK. Eight ports at BASE+0..7 (default A0). Command/handshake protocol, four page buffers

Units: `drive0` (disk, MOUNT)

`[[board.drive]]` — a list you may add

Key	Kind	Legal	Meaning
`unit`	int	any	Which logical drive (slot = unit*2 + platter)
`mount`	string	text	The disk image to put in it. Relative to THIS FILE.
`readonly`	bool	`on` | `off`	Write-protect the disk *(also `writeprotect`)*

Board properties

Key	Kind	Default	Legal	Meaning
`port`	int	`0xA0`	`0x0` .. `0xF8`	Base address. The board decodes eight ports: BASE+0 .. BASE+7
`drives`	int	`1`	`1` .. `8`	Logical drives (one platter each): slot = unit*2 + platter
`interrupt`	enum	`none`	`none` | `int` | `vi0` | `vi1` | `vi2` | `vi3` | `vi4` | `vi5` | `vi6` | `vi7`	Where the card's interrupt is soldered *(interrupt strap)*

`icom`

iCOM FD3712/FD3812 8" floppy: a programmed-I/O command/handshake controller on the S-100 Interface board. Two ports at BASE+0..1 (default C0) plus a boot PROM and 6810 scratch RAM in high memory (rom=builtin:icom-fd3712-cpm | icom-fd3712-fdos | icom-fd3812-cpm). Boots CP/M 2.2 (single and double density) and FDOS. Up to 4 drives

Units: `drive0` (disk, MOUNT), `drive1` (disk, MOUNT)

`[[board.drive]]` — a list you may add

Key	Kind	Legal	Meaning
`unit`	int	`0` .. `1`	Which drive (0..3)
`mount`	string	text	The disk image to put in it. Relative to THIS FILE.
`readonly`	bool	`on` | `off`	Write-protect the disk *(also `writeprotect`)*

Board properties

Key	Kind	Default	Legal	Meaning
`port`	int	`0xC0`	`0x0` .. `0xFE`	Base address. The board decodes two ports: BASE (command/status) and BASE+1 (data)
`rom`	string	`builtin:icom-fd3712-cpm`	text	The boot PROM this interface board carries: builtin:icom-fd3712-cpm (CP/M 2.2, single density, F000), builtin:icom-fd3712-fdos (FDOS, C000), or builtin:icom-fd3812-cpm (CP/M 2.2, double density, F000). The window base and the 6810 scratch RAM follow the image
`drives`	int	`2`	`1` .. `4`	Drives on the controller (unit 0..3, selected by cDRVSEC bits 7:6)

`mds`

MITS 88-MDS: 5.25" minidisk, 4 drives. Same three ports as the dcdd -- but 300 RPM, 64 us/byte, and a motor that stops after 6.4 s

Units: `drive0` (disk, MOUNT), `drive1` (disk, MOUNT), `drive2` (disk, MOUNT), `drive3` (disk, MOUNT)

`[[board.drive]]` — a list you may add

Key	Kind	Legal	Meaning
`unit`	int	`0` .. `3`	Which drive on the daisy chain
`mount`	string	text	The disk image to put in it. Relative to THIS FILE.
`readonly`	bool	`on` | `off`	Write-protect the disk. The DRIVE senses it, so the guest is never told: it writes, the head is inhibited, the bytes go nowhere *(also `writeprotect`)*
`media`	enum	`minidisk`	Force the format instead of probing the image's size
`create`	bool	`on` | `off`	Make the disk file (empty) if it is not there, then mount it -- a fresh disk to FORMAT. Pair with `media` to pick its geometry

Board properties

Key	Kind	Default	Legal	Meaning
`port`	int	`0x8`	`0x0` .. `0xFD`	Base address. The board decodes three ports: BASE+0 .. BASE+2
`drives`	int	`4`	`1` .. `4`	Drives on the daisy chain
`interrupt`	enum	`none`	`none` | `int` | `vi0` | `vi1` | `vi2` | `vi3` | `vi4` | `vi5` | `vi6` | `vi7`	Where the card's interrupt is soldered *(interrupt strap)*
`motor`	enum	`free`	`free` | `real`	free: always at speed (default). real: 1 s spin-up, and it stops after 6.4 s

`tarbell`

Tarbell #1011: single-density WD FD1771 floppy, up to 4 drives. Eight ports at BASE+0..7 (default F8). Carries a 32-byte boot PROM that shadows 0000 over PHANTOM* -- boots CP/M automatically at reset (bootstrap=on)

Units: `drive0` (disk, MOUNT), `drive1` (disk, MOUNT), `drive2` (disk, MOUNT), `drive3` (disk, MOUNT)

`[[board.drive]]` — a list you may add

Key	Kind	Legal	Meaning
`unit`	int	`0` .. `3`	Which drive (0..3)
`mount`	string	text	The disk image to put in it. Relative to THIS FILE.
`readonly`	bool	`on` | `off`	Write-protect the disk. The drive senses it, so the guest is never told *(also `writeprotect`)*

Board properties

Key	Kind	Default	Legal	Meaning
`bootstrap`	bool	`true`	`on` | `off`	The boot-PROM enable DIP. On (default): the 32-byte PROM shadows 0000 over PHANTOM* at reset and boots the disk. Off: a plain disk controller, no PROM
`port`	int	`0xF8`	`0x0` .. `0xF8`	Base address. The board decodes eight ports: BASE+0 .. BASE+7 (default F8)
`drives`	int	`4`	`1` .. `4`	Drives on the controller (binary select 0-3)
`interrupt`	enum	`none`	`none` | `int` | `vi0` | `vi1` | `vi2` | `vi3` | `vi4` | `vi5` | `vi6` | `vi7`	Where the card's interrupt is soldered *(interrupt strap)*

`tarbelldd`

Tarbell #2022: double-density WD FD1791 floppy (mixed-density media, SD track 0), up to 4 drives. The single-density card's twin with a bitmap OUT-FC latch and a port-FD DMA/ext-addr register. Same 32-byte boot PROM

Units: `drive0` (disk, MOUNT), `drive1` (disk, MOUNT), `drive2` (disk, MOUNT), `drive3` (disk, MOUNT)

`[[board.drive]]` — a list you may add

Key	Kind	Legal	Meaning
`unit`	int	`0` .. `3`	Which drive (0..3)
`mount`	string	text	The disk image to put in it. Relative to THIS FILE.
`readonly`	bool	`on` | `off`	Write-protect the disk. The drive senses it, so the guest is never told *(also `writeprotect`)*

Board properties

Key	Kind	Default	Legal	Meaning
`bootstrap`	bool	`true`	`on` | `off`	The boot-PROM enable DIP. On (default): the 32-byte PROM shadows 0000 over PHANTOM* at reset and boots the disk. Off: a plain disk controller, no PROM
`port`	int	`0xF8`	`0x0` .. `0xF8`	Base address. The board decodes eight ports: BASE+0 .. BASE+7 (default F8)
`drives`	int	`4`	`1` .. `4`	Drives on the controller (binary select 0-3)
`interrupt`	enum	`none`	`none` | `int` | `vi0` | `vi1` | `vi2` | `vi3` | `vi4` | `vi5` | `vi6` | `vi7`	Where the card's interrupt is soldered *(interrupt strap)*
`dmaport`	int	`0xE0`	`0x0` .. `0xF0`	Base of the on-card 8257 DMA controller's 16-port register block (default E0). The DMA-mode CBIOS programs ADR/WCT here; the SD2DD strap is E0

`versafloppy`

SD Systems VersaFloppy I/II: WD FD177x soft-sector floppy, up to 4 drives. Eight ports at BASE+0..7 (default 60). variant=vfi (FD1771, single density) | vfii (FD1791, single+double). Boots SDOS with the SBC-200 + DDBIOS

Units: `drive0` (disk, MOUNT), `drive1` (disk, MOUNT), `drive2` (disk, MOUNT), `drive3` (disk, MOUNT)

`[[board.drive]]` — a list you may add

Key	Kind	Legal	Meaning
`unit`	int	`0` .. `3`	Which drive (0..3)
`mount`	string	text	The disk image to put in it. Relative to THIS FILE.
`readonly`	bool	`on` | `off`	Write-protect the disk. The drive senses it, so the guest is never told *(also `writeprotect`)*
`media`	enum	`8sd` | `8dd` | `8dd256` | `8sd-ds` | `8dd-ds` | `8dd256-ds` | `5sd` | `5sd-ds` | `5dd` | `5dd-ds`	Force the format instead of probing the image's size

Board properties

Key	Kind	Default	Legal	Meaning
`variant`	enum	`vfii`	`vfi` | `vfii`	Which board: vfi (FD1771, single density) or vfii (FD1791, single and double density). vfii is the default -- it boots SDOS's DD-256 disks
`port`	int	`0x60`	`0x0` .. `0xF8`	Base address. The board decodes eight ports: BASE+0 .. BASE+7 (60H)
`drives`	int	`4`	`1` .. `4`	Drives on the controller (one-hot select D0-D3)
`interrupt`	enum	`none`	`none` | `int` | `vi0` | `vi1` | `vi2` | `vi3` | `vi4` | `vi5` | `vi6` | `vi7`	Where the card's interrupt is soldered *(interrupt strap)*

Serial

`2sio`

MITS 88-2SIO: two 6850 ACIAs, units 'a' and 'b'. Four ports at BASE+0..3

Units: `a` (serial, CONNECT), `b` (serial, CONNECT)

Board properties

Key	Kind	Default	Legal	Meaning
`port`	int	`0x10`	`0x0` .. `0xFC`	Base address. The board decodes four ports: BASE+0 .. BASE+3

Unit `a` — `[board.unit.a]`

Key	Kind	Default	Legal	Meaning
`baud`	int	`9600`	`50` .. `76800`	Line rate. A JUMPER on the real card -- software cannot change it, and there is no free-running setting: the rate paces the line
`interrupt`	enum	`none`	`none` | `int` | `vi0` | `vi1` | `vi2` | `vi3` | `vi4` | `vi5` | `vi6` | `vi7`	Where this channel's IRQ is jumpered: none | int | vi0..vi7 *(interrupt strap)*
`dcd`	enum	`ground`	`ground` | `wired`	/DCD pin: grounded on the card, or wired to the connector
`cts`	enum	`ground`	`ground` | `wired`	/CTS pin: grounded on the card, or wired -- and then it gates the transmitter
`lines`	string	—	—	Live pin state (read-only). CAPITALS = asserted. in: DCD CTS, out: RTS BRK **(read-only — not a key you may set)**
`connect`	string	`null`	text	The endpoint on the other end of the line (CONNECT sets this)

Unit `b` — `[board.unit.b]`

Key	Kind	Default	Legal	Meaning
`baud`	int	`9600`	`50` .. `76800`	Line rate. A JUMPER on the real card -- software cannot change it, and there is no free-running setting: the rate paces the line
`interrupt`	enum	`none`	`none` | `int` | `vi0` | `vi1` | `vi2` | `vi3` | `vi4` | `vi5` | `vi6` | `vi7`	Where this channel's IRQ is jumpered: none | int | vi0..vi7 *(interrupt strap)*
`dcd`	enum	`ground`	`ground` | `wired`	/DCD pin: grounded on the card, or wired to the connector
`cts`	enum	`ground`	`ground` | `wired`	/CTS pin: grounded on the card, or wired -- and then it gates the transmitter
`lines`	string	—	—	Live pin state (read-only). CAPITALS = asserted. in: DCD CTS, out: RTS BRK **(read-only — not a key you may set)**
`connect`	string	`null`	text	The endpoint on the other end of the line (CONNECT sets this)

`680io`

Altair 680b onboard I/O: a 6850 ACIA console ('tty') at F000/F001 and the config-strap read port at F002. Memory-mapped

Units: `tty` (serial, CONNECT)

Board properties

Key	Kind	Default	Legal	Meaning
`straps`	int	`0x0`	`0x0` .. `0xFF`	Config straps read at F002: bit7 No-Terminal (0=terminal), bit2 stop bits

Unit `tty` — `[board.unit.tty]`

Key	Kind	Default	Legal	Meaning
`baud`	int	`9600`	`50` .. `76800`	Line rate. A JUMPER on the real card -- software cannot change it, and there is no free-running setting: the rate paces the line
`interrupt`	enum	`none`	`none` | `int` | `vi0` | `vi1` | `vi2` | `vi3` | `vi4` | `vi5` | `vi6` | `vi7`	Where this channel's IRQ is jumpered: none | int | vi0..vi7 *(interrupt strap)*
`dcd`	enum	`ground`	`ground` | `wired`	/DCD pin: grounded on the card, or wired to the connector
`cts`	enum	`ground`	`ground` | `wired`	/CTS pin: grounded on the card, or wired -- and then it gates the transmitter
`lines`	string	—	—	Live pin state (read-only). CAPITALS = asserted. in: DCD CTS, out: RTS BRK **(read-only — not a key you may set)**
`connect`	string	`null`	text	The endpoint on the other end of the line (CONNECT sets this)

`gsio`

Generic SIO: a strap-configurable serial board with TWO independent channels, units 'a' and 'b' (configure each under [board.unit.a] / [board.unit.b]). Basic transmit/receive only -- no programmable word length, parity, stop bits or framing/overrun status; a specific card that needs those is a separate emulated board. Per channel you strap: a status/control port (status_port -- read synthesizes DAV/TBMT at bit positions you pick, write is discarded) and a data port (data_port); one inverter_gate knob inverts both status bits together. Built-in profiles preset the straps: profile=sior0 (MITS SIO Rev 0, the default) | tuart

(Cromemco TU-ART) | imsai-sio2 | compupro-if2 (CompuPro Interfacer II) | compupro-ss1 (CompuPro System Support 1). Channel a defaults to ports 0/1, b to 2/3. Polled, no interrupts. CONNECT each channel to a file, socket, serial port, in:/out:

Units: `a` (serial, CONNECT), `b` (serial, CONNECT)

Board properties

No settable properties.

Unit `a` — `[board.unit.a]`

Key	Kind	Default	Legal	Meaning
`profile`	enum	`sior0`	`custom` | `sior0` | `tuart` | `imsai-sio2` | `compupro-if2` | `compupro-ss1`	Built-in card to preset the straps from: custom, or a named board. Selecting one sets status_port/data_port/bits/inverter_gate (still overridable)
`status_port`	int	`0x0`	`0x0` .. `0xFF`	Status(read)/control(write) port. Control writes are discarded
`data_port`	int	`0x1`	`0x0` .. `0xFF`	Data port: receive(read)/transmit(write)
`dav`	int	`0`	`0` .. `7`	Status bit (0-7) carrying DAV, data available (a byte to receive)
`tbmt`	int	`7`	`0` .. `7`	Status bit (0-7) carrying TBMT, transmit buffer empty (ready to send)
`inverter_gate`	bool	`true`	`on` | `off`	Route both status bits through the inverter gate -- asserted DAV/TBMT read 0 (active low)
`baud`	int	`9600`	any	Line rate programmed onto a CONNECTed real serial port (8N1). Inert on a socket/file; does not pace the emulated line
`connect`	string	`null`	text	The endpoint on the serial line (CONNECT sets this): a file, socket, serial port, in:/out: file, null, loopback

Unit `b` — `[board.unit.b]`

Key	Kind	Default	Legal	Meaning
`profile`	enum	`sior0`	`custom` | `sior0` | `tuart` | `imsai-sio2` | `compupro-if2` | `compupro-ss1`	Built-in card to preset the straps from: custom, or a named board. Selecting one sets status_port/data_port/bits/inverter_gate (still overridable)
`status_port`	int	`0x2`	`0x0` .. `0xFF`	Status(read)/control(write) port. Control writes are discarded
`data_port`	int	`0x3`	`0x0` .. `0xFF`	Data port: receive(read)/transmit(write)
`dav`	int	`0`	`0` .. `7`	Status bit (0-7) carrying DAV, data available (a byte to receive)
`tbmt`	int	`7`	`0` .. `7`	Status bit (0-7) carrying TBMT, transmit buffer empty (ready to send)
`inverter_gate`	bool	`true`	`on` | `off`	Route both status bits through the inverter gate -- asserted DAV/TBMT read 0 (active low)
`baud`	int	`9600`	any	Line rate programmed onto a CONNECTed real serial port (8N1). Inert on a socket/file; does not pace the emulated line
`connect`	string	`null`	text	The endpoint on the serial line (CONNECT sets this): a file, socket, serial port, in:/out: file, null, loopback

`io4`

SSM IO-4 (2P+2S): the real Solid State Music board -- two full-duplex serial channels AND a four-port parallel section. Serial units 'a'/'b' are real 1602-family UARTs with programmable word length (data_bits 5-8), parity and stop bits (unlike the generic gsio), plus the full status-word strap-up (stat_* map, invert_status, port_reversal) and named profiles (default altair-rev1). A 4-port block set by switch S3 (default 0-3): Serial A status/data at BASE+0/+1, Serial B at BASE+2/+3. Parallel units 'pa'/'pb' are 8212 latched ports (input latch + service request, output latch) on their own 2-port block set by switch S4 (par_port, default 4-5): Parallel A at PAR+0, B at PAR+1; a byte the far end sends is strobed in, a write latches out, and the §3.2.2 status/data console flag is strappable (dav_bit/dav_source/dav_active_low). If the two blocks overlap, neither section answers the shared ports. Each serial channel's receive (DAV) and transmit (TBMT) and each parallel input (service request) can raise an interrupt, strapped on header W4 to a VI line, pin 73 (int), or none (rx_int/tx_int per serial channel, int per parallel port) -- there is no software enable, the strap is the enable, and a parallel input interrupt rises even when the port is not addressed. Configure each unit under [board.unit.a] / [board.unit.pa] etc.; CONNECT each to a file, socket, serial port, in:/out:

Units: `a` (serial, CONNECT), `b` (serial, CONNECT), `pa` (serial, CONNECT), `pb` (serial, CONNECT)

Board properties

Key	Kind	Default	Legal	Meaning
`port`	int	`0x0`	`0x0` .. `0xFC`	Serial base address (switch S3) -- a 4-PORT BLOCK, so a multiple of 4. Serial A at BASE+0/+1, Serial B at BASE+2/+3
`par_port`	int	`0x4`	`0x0` .. `0xFE`	Parallel base address (switch S4) -- a 2-PORT BLOCK, so a multiple of 2. Parallel A (J6-in/J5-out) at BASE, Parallel B (J4-in/J3-out) at BASE+1. If it overlaps the serial block, neither section answers the shared ports

Unit `a` — `[board.unit.a]`

Key	Kind	Default	Legal	Meaning
`profile`	enum	`altair-rev1`	`custom` | `altair-rev1` | `altair-rev0` | `i8251` | `proctech` | `imsai`	Preset the status straps from a host personality: custom, altair-rev1 (the default and the SSM 8080 monitor console), altair-rev0, i8251, proctech, imsai. Sets stat_*, invert_status and port_reversal (overridable)
`stat_dav`	enum	`0`	`none` | `0` | `1` | `2` | `3` | `4` | `5` | `6` | `7`	Data-bus bit carrying DAV, data available (0-7 | none)
`stat_tbmt`	enum	`7`	`none` | `0` | `1` | `2` | `3` | `4` | `5` | `6` | `7`	Data-bus bit carrying TBMT, transmit buffer empty (0-7 | none)
`stat_teoc`	enum	`none`	`none` | `0` | `1` | `2` | `3` | `4` | `5` | `6` | `7`	Data-bus bit carrying TEOC, transmitter end of character (0-7 | none)
`stat_ror`	enum	`none`	`none` | `0` | `1` | `2` | `3` | `4` | `5` | `6` | `7`	Data-bus bit carrying ROR, receiver over-run (0-7 | none; always inactive)
`stat_rpe`	enum	`none`	`none` | `0` | `1` | `2` | `3` | `4` | `5` | `6` | `7`	Data-bus bit carrying RPE, receiver parity error (0-7 | none; always inactive)
`stat_rfe`	enum	`none`	`none` | `0` | `1` | `2` | `3` | `4` | `5` | `6` | `7`	Data-bus bit carrying RFE, receiver framing error (0-7 | none; always inactive)
`invert_status`	bool	`true`	`on` | `off`	Invert every status bit -- a 74368 buffer (negative sense): an asserted signal reads 0, an inactive one reads 1. Off = 74367 (positive sense)
`port_reversal`	bool	`false`	`on` | `off`	Reverse the channel's two ports (S1/S2-PR): off = status first, data last (MITS, Proc Tech); on = data first, status last (IMSAI)
`rx_int`	enum	`none`	`none` | `int` | `vi0` | `vi1` | `vi2` | `vi3` | `vi4` | `vi5` | `vi6` | `vi7`	Where the receive interrupt (a new character, DAV) is strapped on header W4: none | int | vi0..vi7. Canonical: vi1 (Serial A) / vi0 (Serial B) *(interrupt strap)*
`tx_int`	enum	`none`	`none` | `int` | `vi0` | `vi1` | `vi2` | `vi3` | `vi4` | `vi5` | `vi6` | `vi7`	Where the transmit interrupt (buffer empty, TBMT) is strapped on header W4: none | int | vi0..vi7. It is a LEVEL -- held while the transmitter is idle. Canonical: vi2 (Serial A) / vi3 (Serial B) *(interrupt strap)*
`baud`	int	`9600`	`50` .. `25000`	Line rate (header W3). RX and TX share one rate here -- a real host serial port cannot be split. Canonical IO-4 rates: 55-9600
`data_bits`	int	`8`	`5` .. `8`	Data bits per character (S1/S2 NDB1+NDB2)
`stop_bits`	int	`1`	`1` .. `2`	Stop bits (S1/S2 NSB): 1 or 2

`parity`	enum	`none`	`none` | `odd` | `even`	Parity (S1/S2 NPB/POE): none | odd | even
`connect`	string	`null`	text	The endpoint on this channel's line (CONNECT sets this): a file, socket, serial port, in:/out: file, null, loopback

Unit `b` — `[board.unit.b]`

Key	Kind	Default	Legal	Meaning
`profile`	enum	`altair-rev1`	`custom` | `altair-rev1` | `altair-rev0` | `i8251` | `proctech` | `imsai`	Preset the status straps from a host personality: custom, altair-rev1 (the default and the SSM 8080 monitor console), altair-rev0, i8251, proctech, imsai. Sets stat_*, invert_status and port_reversal (overridable)
`stat_dav`	enum	`0`	`none` | `0` | `1` | `2` | `3` | `4` | `5` | `6` | `7`	Data-bus bit carrying DAV, data available (0-7 | none)
`stat_tbmt`	enum	`7`	`none` | `0` | `1` | `2` | `3` | `4` | `5` | `6` | `7`	Data-bus bit carrying TBMT, transmit buffer empty (0-7 | none)
`stat_teoc`	enum	`none`	`none` | `0` | `1` | `2` | `3` | `4` | `5` | `6` | `7`	Data-bus bit carrying TEOC, transmitter end of character (0-7 | none)
`stat_ror`	enum	`none`	`none` | `0` | `1` | `2` | `3` | `4` | `5` | `6` | `7`	Data-bus bit carrying ROR, receiver over-run (0-7 | none; always inactive)
`stat_rpe`	enum	`none`	`none` | `0` | `1` | `2` | `3` | `4` | `5` | `6` | `7`	Data-bus bit carrying RPE, receiver parity error (0-7 | none; always inactive)
`stat_rfe`	enum	`none`	`none` | `0` | `1` | `2` | `3` | `4` | `5` | `6` | `7`	Data-bus bit carrying RFE, receiver framing error (0-7 | none; always inactive)
`invert_status`	bool	`true`	`on` | `off`	Invert every status bit -- a 74368 buffer (negative sense): an asserted signal reads 0, an inactive one reads 1. Off = 74367 (positive sense)
`port_reversal`	bool	`false`	`on` | `off`	Reverse the channel's two ports (S1/S2-PR): off = status first, data last (MITS, Proc Tech); on = data first, status last (IMSAI)
`rx_int`	enum	`none`	`none` | `int` | `vi0` | `vi1` | `vi2` | `vi3` | `vi4` | `vi5` | `vi6` | `vi7`	Where the receive interrupt (a new character, DAV) is strapped on header W4: none | int | vi0..vi7. Canonical: vi1 (Serial A) / vi0 (Serial B) *(interrupt strap)*
`tx_int`	enum	`none`	`none` | `int` | `vi0` | `vi1` | `vi2` | `vi3` | `vi4` | `vi5` | `vi6` | `vi7`	Where the transmit interrupt (buffer empty, TBMT) is strapped on header W4: none | int | vi0..vi7. It is a LEVEL -- held while the transmitter is idle. Canonical: vi2 (Serial A) / vi3 (Serial B) *(interrupt strap)*
`baud`	int	`9600`	`50` .. `25000`	Line rate (header W3). RX and TX share one rate here -- a real host serial port cannot be split. Canonical IO-4 rates: 55-9600
`data_bits`	int	`8`	`5` .. `8`	Data bits per character (S1/S2 NDB1+NDB2)
`stop_bits`	int	`1`	`1` .. `2`	Stop bits (S1/S2 NSB): 1 or 2

`parity`	enum	`none`	`none` | `odd` | `even`	Parity (S1/S2 NPB/POE): none | odd | even
`connect`	string	`null`	text	The endpoint on this channel's line (CONNECT sets this): a file, socket, serial port, in:/out: file, null, loopback

Unit `pa` — `[board.unit.pa]`

Key	Kind	Default	Legal	Meaning
`connect`	string	`null`	text	The endpoint on this parallel port's line (CONNECT sets this): a file, socket, printer, in:/out: file, null. A WRITE latches out; a byte the far end sends is strobed into the input latch
`dav_bit`	enum	`none`	`none` | `0` | `1` | `2` | `3` | `4` | `5` | `6` | `7`	Data-bus bit a data-available flag is jumpered onto when this port is read (§3.2.2 status/data console): 0-7, or none
`dav_source`	enum	`self`	`self` | `sibling`	Whose service request the dav_bit shows: self (this port's) or sibling (the other parallel port's -- the status/data console reads the data port's flag at the status port)
`dav_active_low`	bool	`false`	`on` | `off`	Present the data-available flag active-low -- the dav_bit reads 0 when a byte is waiting (§3.2.2 "D0 going low"). Off = active-high
`int`	enum	`none`	`none` | `int` | `vi0` | `vi1` | `vi2` | `vi3` | `vi4` | `vi5` | `vi6` | `vi7`	Where this parallel input's interrupt (a byte latched in, service request) is strapped on header W4: none | int | vi0..vi7. Canonical: vi6 (Parallel A) / vi5 (Parallel B) *(interrupt strap)*

Unit `pb` — `[board.unit.pb]`

Key	Kind	Default	Legal	Meaning
`connect`	string	`null`	text	The endpoint on this parallel port's line (CONNECT sets this): a file, socket, printer, in:/out: file, null. A WRITE latches out; a byte the far end sends is strobed into the input latch
`dav_bit`	enum	`none`	`none` | `0` | `1` | `2` | `3` | `4` | `5` | `6` | `7`	Data-bus bit a data-available flag is jumpered onto when this port is read (§3.2.2 status/data console): 0-7, or none
`dav_source`	enum	`self`	`self` | `sibling`	Whose service request the dav_bit shows: self (this port's) or sibling (the other parallel port's -- the status/data console reads the data port's flag at the status port)
`dav_active_low`	bool	`false`	`on` | `off`	Present the data-available flag active-low -- the dav_bit reads 0 when a byte is waiting (§3.2.2 "D0 going low"). Off = active-high
`int`	enum	`none`	`none` | `int` | `vi0` | `vi1` | `vi2` | `vi3` | `vi4` | `vi5` | `vi6` | `vi7`	Where this parallel input's interrupt (a byte latched in, service request) is strapped on header W4: none | int | vi0..vi7. Canonical: vi6 (Parallel A) / vi5 (Parallel B) *(interrupt strap)*

`pmmi`

PMMI MM-103: Bell 103 modem on an S-100 card, unit 'line'. Four ports at BASE+0..3 (default C0), read/write different registers. Transmit/receive over a ByteStream; CONNECT it to in:/out: files. No dialer; modem status is a fixed stub

Units: `line` (serial, CONNECT)

Board properties

Key	Kind	Default	Legal	Meaning
`port`	int	`0xC0`	`0x0` .. `0xFC`	Base address -- the 6-position DIP. Four ports at BASE..BASE+3; MUST be on a 4-port boundary. Default C0 (North Star alternative E0)
`connect`	string	`null`	text	The endpoint on the phone line (CONNECT sets this). in:/out: a file, ...
`dial`	string		text	Originate target host:port (empty = cannot dial). SH off-hook + DTR dials it; the guest's pulse digits are not decoded
`answer`	string		text	Auto-answer TCP port (empty/0 = will not answer). DTR arms the listener; an inbound call RINGS until the guest answers with RI
`rtsdtr`	bool	`false`	`on` | `off`	Mirror DTR onto RTS on a CONNECTed serial port (for cables that need RTS asserted to pass data). Default off
`frame`	string	—	—	Live UART frame (read-only), e.g. 8N1. Set by OUT BA+0 bits 2-6 **(read-only — not a key you may set)**
`baud`	int	—	—	Live line rate (read-only), 250000/(16*N) from the OUT BA+2 divisor **(read-only — not a key you may set)**
`uart`	string	—	—	Live UART status (read-only). CAPITALS = asserted: TBMT DAV **(read-only — not a key you may set)**
`lines`	string	—	—	Live modem lines (read-only). CAPITALS = asserted: SH RI DTR ST DT RING CTS AP (DT/RING/CTS/AP from the phone line + handshake state machine) **(read-only — not a key you may set)**

`propio`

S100Computers Console IO Board (Parallax-Propeller console), unit 'serial'. A strap-configurable serial subtype preset to the board's documented convention: status/data at 00/01, RX-ready = status bit 1, TX-ready = status bit 2, both active high. Every strap (status_port/data_port/dav/tbmt/inverter_gate) is still overridable -- the real board is jumpered. Polled, no interrupts. CONNECT it to a file, socket, serial port, in:/out:

Units: `serial` (serial, CONNECT)

Board properties

Key	Kind	Default	Legal	Meaning
`profile`	enum	`custom`	`custom` | `sior0` | `tuart` | `imsai-sio2` | `compupro-if2` | `compupro-ss1`	Built-in card to preset the straps from: custom, or a named board. Selecting one sets status_port/data_port/bits/inverter_gate (still overridable)
`status_port`	int	`0x0`	`0x0` .. `0xFF`	Status(read)/control(write) port. Control writes are discarded
`data_port`	int	`0x1`	`0x0` .. `0xFF`	Data port: receive(read)/transmit(write)
`dav`	int	`1`	`0` .. `7`	Status bit (0-7) carrying DAV, data available (a byte to receive)
`tbmt`	int	`2`	`0` .. `7`	Status bit (0-7) carrying TBMT, transmit buffer empty (ready to send)
`inverter_gate`	bool	`false`	`on` | `off`	Route both status bits through the inverter gate -- asserted DAV/TBMT read 0 (active low)
`baud`	int	`9600`	any	Line rate programmed onto a CONNECTed real serial port (8N1). Inert on a socket/file; does not pace the emulated line
`connect`	string	`null`	text	The endpoint on the serial line (CONNECT sets this): a file, socket, serial port, in:/out: file, null, loopback

`sbc`

SD Systems SBC-100/200: Z80 single-board computer. One 8-port block (78-7F): Intel 8251 console (unit 'tty', data 7C / status 7D, RxD->/DSR auto-baud for MSMONR21), Z80-CTC (78-7B) whose ch1 raises a mode-2 keyboard interrupt (vector 0x82) off the 8251 RxRDY, and a parallel port (7E/7F) whose OUT 7F bit 1 switches the onboard PROM out. Optional onboard boot PROM via [[board.socket]] (at+mount). variant=sbc100|sbc200

Units: `tty` (serial, CONNECT)

`[[board.socket]]` — a list you may add

Key	Kind	Legal	Meaning
`at`	int	any	Where the socket sits in the onboard window (E000 = monitor, F000 = disk BIOS)
`mount`	string	text	What is in the socket: builtin: or a HEX/BIN path. Relative to THIS FILE.

Board properties

Key	Kind	Default	Legal	Meaning
`variant`	enum	`sbc200`	`sbc100` | `sbc200`	Which board: sbc100 (2.4576 MHz) or sbc200 (4 MHz). The console, CTC and PROM behave alike here; the CPU crystal is set on the z80 card
`rxd2dsr`	bool	`true`	`on` | `off`	RxD strapped to /DSR (the SBC auto-baud jumper). Off = a plain 8251 /DSR
`port`	int	`0x7C`	`0x0` .. `0xFE`	Base I/O address (a card jumper). Data at BASE, status/command at BASE+1. The etch default is 7C

Unit `tty` — `[board.unit.tty]`

Key	Kind	Default	Legal	Meaning
`baud`	int	`9600`	`50` .. `76800`	Line rate. On the SBC the CTC generates it; here it paces the receive line and sizes the auto-baud bit. No free-running setting (min 50)
`interrupt`	enum	`none`	`none` | `int` | `vi0` | `vi1` | `vi2` | `vi3` | `vi4` | `vi5` | `vi6` | `vi7`	Where this port's IRQ is jumpered: none | int | vi0..vi7 (decoded; not yet honored) *(interrupt strap)*
`connect`	string	`null`	text	The endpoint on the other end of the line (CONNECT sets this)

`sio`

MITS 88-SIO: one COM2502 UART, unit 'tty'. Two ports at BASE+0..1. INVERTED status bits

Units: `tty` (serial, CONNECT)

Board properties

Key	Kind	Default	Legal	Meaning
`port`	int	`0x0`	`0x0` .. `0xFE`	Base address -- MUST BE EVEN. Control at BASE, data at BASE+1
`rev`	enum	`1`	`0` | `1`	Board revision. 1 = the factory errata mod: ready is bit 7 (out) and bit 0 (in), both inverted. 0 = as shipped, which also reports them true-sense on bits 5 and 1
`baud`	int	`9600`	`50` .. `25000`	Line rate. A JUMPER on the real card -- software cannot change it
`data_bits`	int	`8`	`5` .. `8`	Data bits per character. The NDB1/NDB2 pads
`stop_bits`	int	`1`	`1` .. `2`	Stop bits. The NSB pad: GND = 1, +V = 2
`parity`	enum	`none`	`none` | `odd` | `even`	The NPB/POE pads: none | odd | even
`in_int`	enum	`none`	`none` | `int` | `vi0` | `vi1` | `vi2` | `vi3` | `vi4` | `vi5` | `vi6` | `vi7`	Where the IN pad is soldered (RX): none | int | vi0..vi7 *(interrupt strap)*
`out_int`	enum	`none`	`none` | `int` | `vi0` | `vi1` | `vi2` | `vi3` | `vi4` | `vi5` | `vi6` | `vi7`	Where the OUT pad is soldered (TX): none | int | vi0..vi7 *(interrupt strap)*
`connect`	string	`null`	text	The endpoint on the other end of the line (CONNECT sets this)

`turnkey`

MITS 8800b Turnkey Module: phantom boot PROM (FC00-FFFF), integrated 6850 SIO (unit 'tty', default 0x10), sense switches at FF, and the Auto-Start JMP jam. Sockets via [[board.socket]]

Units: `tty` (serial, CONNECT)

`[[board.socket]]` — a list you may add

Key	Kind	Legal	Meaning
`at`	int	any	Where the socket sits in the window (FC00/FD00/FE00/FF00)
`mount`	string	text	What is in the socket: builtin: or a HEX/BIN path. Relative to THIS FILE.

Board properties

Key	Kind	Default	Legal	Meaning
`prom`	int	`0xFC00`	`0x0` .. `0xFC00`	PROM ADDR switches: base of the 1K boot-PROM window (FC00-FFFF normal)
`start`	int	`0xFC00`	`0x0` .. `0xFF00`	START ADDR switches: Auto-Start jams JMP here at reset (a multiple of 256)
`sense`	int	`0x0`	`0x0` .. `0xFF`	Sense switches (SW6/SW7), read at port FF
`sio_base`	int	`0x10`	`0x0` .. `0xFE`	Base address of the integrated 6850 SIO (0x10 = 2SIO Port A)

Unit `tty` — `[board.unit.tty]`

Key	Kind	Default	Legal	Meaning
`baud`	int	`9600`	`50` .. `76800`	Line rate. A JUMPER on the real card -- software cannot change it, and there is no free-running setting: the rate paces the line
`interrupt`	enum	`none`	`none` | `int` | `vi0` | `vi1` | `vi2` | `vi3` | `vi4` | `vi5` | `vi6` | `vi7`	Where this channel's IRQ is jumpered: none | int | vi0..vi7 *(interrupt strap)*
`dcd`	enum	`ground`	`ground` | `wired`	/DCD pin: grounded on the card, or wired to the connector
`cts`	enum	`ground`	`ground` | `wired`	/CTS pin: grounded on the card, or wired -- and then it gates the transmitter
`lines`	string	—	—	Live pin state (read-only). CAPITALS = asserted. in: DCD CTS, out: RTS BRK **(read-only — not a key you may set)**
`connect`	string	`null`	text	The endpoint on the other end of the line (CONNECT sets this)

Tape

`680kcacr`

Altair 680b KCACR audio-cassette interface: a 1602-family UART recording Kansas City Standard FSK, memory-mapped at F010 (status/control) and F011 (data), active-LOW. Adds software motor control (control D7=on, D6=off) and interrupt-driven transfer (D0/D1 enables pull the 6800 IRQ). Reuses the 88-ACR tape machinery -- MOUNT a tape, WIND/REWIND it

Units: `tape` (tape, MOUNT)

Board properties

Key	Kind	Default	Legal	Meaning
`baud`	int	`300`	`50` .. `25000`	Line rate. A JUMPER on the real card -- software cannot change it
`data_bits`	int	`8`	`5` .. `8`	Data bits per character. The NDB1/NDB2 pads
`stop_bits`	int	`2`	`1` .. `2`	Stop bits. The NSB pad: GND = 1, +V = 2
`parity`	enum	`none`	`none` | `odd` | `even`	The NPB/POE pads: none | odd | even
`motor`	enum	—	—	Tape-recorder motor relay (guest-driven: STA F010 7F = on, BF = off) **(read-only — not a key you may set)**

Unit `tape` — `[board.unit.tape]`

Key	Kind	Default	Legal	Meaning
`mode`	enum	`play`	`play` | `record`	Which way the bytes go: play loads from the file, record saves to it
`format`	enum	`auto`	`auto` | `raw` | `kcs300`	How to read the mounted file: auto | raw | fsk300
`leader`	int	`15`	`0` .. `120`	Seconds of idle tone before recorded data, when writing audio
`trailer`	int	`5`	`0` .. `120`	Seconds of idle tone after recorded data, when writing audio
`waveform`	enum	`square`	`square` | `sine`	Carrier shape when writing audio: square (like real hardware) | sine
`level`	int	`36`	`1` .. `100`	Recording level as a percent of full scale, when writing audio
`rate`	enum	`full`	`full` | `real`	Playback speed: full (as fast as the guest reads) | real (wall-clock baud)
`detected`	string	—	—	What the mounted tape turned out to be (empty if nothing is mounted) **(read-only — not a key you may set)**
`position`	string	—	—	Where the tape head is now: mm:ss / total (percent) -- read-only **(read-only — not a key you may set)**
`counter`	enum	`on`	`on` | `off`	Live tape counter on the console during a load: on | off
`stop`	string	`off`	off | end | mm:ss	Auto-stop playback at this time: off | end | mm:ss

`acr`

MITS 88-ACR: cassette. An 88-SIO B + an FSK modem, unit 'tape'. Brings the WIND/REWIND/EXTRACT verbs and a tape counter

Units: `tape` (tape, MOUNT)

Board properties

Key	Kind	Default	Legal	Meaning
`port`	int	`0x6`	`0x0` .. `0xFE`	Base address -- MUST BE EVEN. Control at BASE, data at BASE+1
`rev`	enum	`1`	`0` | `1`	Board revision. 1 = the factory errata mod: ready is bit 7 (out) and bit 0 (in), both inverted. 0 = as shipped, which also reports them true-sense on bits 5 and 1
`baud`	int	`300`	`50` .. `25000`	Line rate. A JUMPER on the real card -- software cannot change it
`data_bits`	int	`8`	`5` .. `8`	Data bits per character. The NDB1/NDB2 pads
`stop_bits`	int	`1`	`1` .. `2`	Stop bits. The NSB pad: GND = 1, +V = 2
`parity`	enum	`none`	`none` | `odd` | `even`	The NPB/POE pads: none | odd | even
`in_int`	enum	`none`	`none` | `int` | `vi0` | `vi1` | `vi2` | `vi3` | `vi4` | `vi5` | `vi6` | `vi7`	Where the IN pad is soldered (RX): none | int | vi0..vi7 *(interrupt strap)*
`out_int`	enum	`none`	`none` | `int` | `vi0` | `vi1` | `vi2` | `vi3` | `vi4` | `vi5` | `vi6` | `vi7`	Where the OUT pad is soldered (TX): none | int | vi0..vi7 *(interrupt strap)*

Unit `tape` — `[board.unit.tape]`

Key	Kind	Default	Legal	Meaning
`mode`	enum	`play`	`play` | `record`	Which way the bytes go: play loads from the file, record saves to it
`format`	enum	`auto`	`auto` | `raw` | `fsk300`	How to read the mounted file: auto | raw | fsk300
`leader`	int	`15`	`0` .. `120`	Seconds of idle tone before recorded data, when writing audio
`trailer`	int	`5`	`0` .. `120`	Seconds of idle tone after recorded data, when writing audio
`waveform`	enum	`square`	`square` | `sine`	Carrier shape when writing audio: square (like real hardware) | sine
`level`	int	`36`	`1` .. `100`	Recording level as a percent of full scale, when writing audio
`rate`	enum	`full`	`full` | `real`	Playback speed: full (as fast as the guest reads) | real (wall-clock baud)
`detected`	string	—	—	What the mounted tape turned out to be (empty if nothing is mounted) **(read-only — not a key you may set)**
`position`	string	—	—	Where the tape head is now: mm:ss / total (percent) -- read-only **(read-only — not a key you may set)**
`counter`	enum	`on`	`on` | `off`	Live tape counter on the console during a load: on | off
`stop`	string	`off`	off | end | mm:ss	Auto-stop playback at this time: off | end | mm:ss

`uio`

MITS 88-UIO: serial + cassette on one board. A 6850 (unit 'serial', default 0x10) and an 88-ACR cassette section (unit 'tape', default 0x06) with motor control and a SW-1 MITS/Kansas-City modulation switch. Defaults reproduce the standard 0x10 + 0x06 layout

Units: `tape` (tape, MOUNT), `serial` (serial, CONNECT)

Board properties

Key	Kind	Default	Legal	Meaning
`port`	int	`0x6`	`0x0` .. `0xFE`	Base address -- MUST BE EVEN. Control at BASE, data at BASE+1
`rev`	enum	`1`	`0` | `1`	Board revision. 1 = the factory errata mod: ready is bit 7 (out) and bit 0 (in), both inverted. 0 = as shipped, which also reports them true-sense on bits 5 and 1
`baud`	int	`300`	`50` .. `25000`	Line rate. A JUMPER on the real card -- software cannot change it
`data_bits`	int	`8`	`5` .. `8`	Data bits per character. The NDB1/NDB2 pads
`stop_bits`	int	`1`	`1` .. `2`	Stop bits. The NSB pad: GND = 1, +V = 2
`parity`	enum	`none`	`none` | `odd` | `even`	The NPB/POE pads: none | odd | even
`in_int`	enum	`none`	`none` | `int` | `vi0` | `vi1` | `vi2` | `vi3` | `vi4` | `vi5` | `vi6` | `vi7`	Where the IN pad is soldered (RX): none | int | vi0..vi7 *(interrupt strap)*
`out_int`	enum	`none`	`none` | `int` | `vi0` | `vi1` | `vi2` | `vi3` | `vi4` | `vi5` | `vi6` | `vi7`	Where the OUT pad is soldered (TX): none | int | vi0..vi7 *(interrupt strap)*
`serial_port`	int	`0x10`	`0x0` .. `0xFE`	Serial (6850) base -- SW-2. 0x10 = 2SIO Port A (default); SW-2 ON = 0x18
`standard`	enum	`mits`	`mits` | `kansas`	SW-1 tape modulation: mits (2400/1850) | kansas (Kansas City 2400/1200)
`motor`	enum	—	—	Tape-recorder motor relay (guest-driven: OUT 6,127 = on, OUT 6,191 = off) **(read-only — not a key you may set)**

Unit `tape` — `[board.unit.tape]`

Key	Kind	Default	Legal	Meaning
`mode`	enum	`play`	`play` | `record`	Which way the bytes go: play loads from the file, record saves to it
`format`	enum	`auto`	`auto` | `raw` | `fsk300`	How to read the mounted file: auto | raw | fsk300
`leader`	int	`15`	`0` .. `120`	Seconds of idle tone before recorded data, when writing audio
`trailer`	int	`5`	`0` .. `120`	Seconds of idle tone after recorded data, when writing audio
`waveform`	enum	`square`	`square` | `sine`	Carrier shape when writing audio: square (like real hardware) | sine
`level`	int	`36`	`1` .. `100`	Recording level as a percent of full scale, when writing audio
`rate`	enum	`full`	`full` | `real`	Playback speed: full (as fast as the guest reads) | real (wall-clock baud)
`detected`	string	—	—	What the mounted tape turned out to be (empty if nothing is mounted) **(read-only — not a key you may set)**
`position`	string	—	—	Where the tape head is now: mm:ss / total (percent) -- read-only **(read-only — not a key you may set)**
`counter`	enum	`on`	`on` | `off`	Live tape counter on the console during a load: on | off
`stop`	string	`off`	off | end | mm:ss	Auto-stop playback at this time: off | end | mm:ss

Unit `serial` — `[board.unit.serial]`

Key	Kind	Default	Legal	Meaning
`baud`	int	`9600`	`50` .. `76800`	Line rate. A JUMPER on the real card -- software cannot change it, and there is no free-running setting: the rate paces the line
`interrupt`	enum	`none`	`none` | `int` | `vi0` | `vi1` | `vi2` | `vi3` | `vi4` | `vi5` | `vi6` | `vi7`	Where this channel's IRQ is jumpered: none | int | vi0..vi7 *(interrupt strap)*
`dcd`	enum	`ground`	`ground` | `wired`	/DCD pin: grounded on the card, or wired to the connector
`cts`	enum	`ground`	`ground` | `wired`	/CTS pin: grounded on the card, or wired -- and then it gates the transmitter
`lines`	string	—	—	Live pin state (read-only). CAPITALS = asserted. in: DCD CTS, out: RTS BRK **(read-only — not a key you may set)**
`connect`	string	`null`	text	The endpoint on the other end of the line (CONNECT sets this)

Parallel and printer

`4pio`

MITS 88-4PIO: up to four 6820 PIAs, sections ja/jb.. per port. 16 ports from BASE (default 20). Software-set direction; CONNECT each section

Units: `ja` (serial, CONNECT), `jb` (serial, CONNECT)

Board properties

Key	Kind	Default	Legal	Meaning
`port`	int	`0x20`	`0x0` .. `0xF0`	Base address -- must be on a 16-address boundary. 16 ports from here
`ports`	int	`1`	`1` .. `4`	How many 6820 PIAs are populated (1..4 -- J, K, L, M)

Unit `ja` — `[board.unit.ja]`

Key	Kind	Default	Legal	Meaning
`connect`	string	`null`	text	The endpoint on the other end of this section (CONNECT sets this)

Unit `jb` — `[board.unit.jb]`

Key	Kind	Default	Legal	Meaning
`connect`	string	`null`	text	The endpoint on the other end of this section (CONNECT sets this)

`680uio`

Altair 680b Universal I/O: a second 6850 ACIA serial port ('serial') and a 6820 PIA parallel port (sections 'p1a/p1b', 'p2a/p2b' with pias=2) in an S9-relocatable window (default base F000: serial F006/F007, PIA F008-F00F), plus fixed switch inputs at F003 and a non-latched output at F010-F013. Memory-mapped, active-high

Units: `serial` (serial, CONNECT), `p1a` (serial, CONNECT), `p1b` (serial, CONNECT)

Board properties

Key	Kind	Default	Legal	Meaning
`base`	int	`0xF000`	`0xF000` .. `0xF0F0`	S9 window base (F000 + position*0x10); serial at base+6, PIAs base+8..+F
`pias`	int	`1`	`1` .. `2`	6820 PIAs populated: 1 (PIA-C only) or 2 (PIA-C + PIA-B)
`sense`	int	`0x0`	`0x0` .. `0xFF`	Switch inputs read at F003 (fixed, read-only tri-state)
`nlout`	bool	`true`	`on` | `off`	Decode the F010-F013 non-latched output (off = IC A1 removed, KCACR owns F010/F011)

Unit `serial` — `[board.unit.serial]`

Key	Kind	Default	Legal	Meaning
`baud`	int	`9600`	`50` .. `76800`	Line rate. A JUMPER on the real card -- software cannot change it, and there is no free-running setting: the rate paces the line
`interrupt`	enum	`none`	`none` | `int` | `vi0` | `vi1` | `vi2` | `vi3` | `vi4` | `vi5` | `vi6` | `vi7`	Where this channel's IRQ is jumpered: none | int | vi0..vi7 *(interrupt strap)*
`dcd`	enum	`ground`	`ground` | `wired`	/DCD pin: grounded on the card, or wired to the connector
`cts`	enum	`ground`	`ground` | `wired`	/CTS pin: grounded on the card, or wired -- and then it gates the transmitter
`lines`	string	—	—	Live pin state (read-only). CAPITALS = asserted. in: DCD CTS, out: RTS BRK **(read-only — not a key you may set)**
`connect`	string	`null`	text	The endpoint on the other end of the line (CONNECT sets this)

Unit `p1a` — `[board.unit.p1a]`

Key	Kind	Default	Legal	Meaning
`connect`	string	`null`	text	The endpoint on the other end of this PIA section (CONNECT sets this)

Unit `p1b` — `[board.unit.p1b]`

Key	Kind	Default	Legal	Meaning
`connect`	string	`null`	text	The endpoint on the other end of this PIA section (CONNECT sets this)

`c700`

MITS 88-C700: Centronics line-printer controller, unit 'prn'. Two ports at BASE+0..1 (default 02). Output-only; CONNECT it to a file, a socket, or a real printer queue

Units: `prn` (serial, CONNECT)

Board properties

Key	Kind	Default	Legal	Meaning
`port`	int	`0x2`	`0x0` .. `0xFE`	Base address -- MUST BE EVEN. Control/status at BASE, data at BASE+1
`connect`	string	`null`	text	The endpoint on the other end of the line (CONNECT sets this)
`interrupt`	enum	`int`	`none` | `int` | `vi0` | `vi1` | `vi2` | `vi3` | `vi4` | `vi5` | `vi6` | `vi7`	Where the printer's interrupt is soldered: none | int (pin 73) | vi0..vi7 *(interrupt strap)*
`interrupt_after`	enum	`char`	`char` | `crlf`	SW2 #4: raise the interrupt after every 'char' or only after a 'crlf'

`d7a`

Cromemco D+7A: analog + parallel I/O. Eight ports from BASE (default 18): one parallel port + seven two's-complement A/D-in/D/A-out channels. Reads 1-2 JS-1 joysticks from the host

Board properties

Key	Kind	Default	Legal	Meaning
`port`	int	`0x18`	`0x0` .. `0xF8`	Base of the 8-port block (A7..A3 jumpers): parallel at BASE, analog at BASE+1..7. A multiple of 8; default 18
`joystick1`	string	`auto`	text	Which host controller drives JS-1 console 1: 'none', 'auto' (the matching gamepad -- console 1->pad 0, console 2->pad 1 -- or the keyboard), 'keyboard', or a device index like 0
`joystick2`	string	`auto`	text	Which host controller drives JS-1 console 2: 'none', 'auto' (the matching gamepad -- console 1->pad 0, console 2->pad 1 -- or the keyboard), 'keyboard', or a device index like 0

`lpc`

MITS 88-LPC: 88-LP line-printer controller, unit 'prn'. Two ports at BASE+0..1 (default 02). Line-buffered: 6-bit codes + PRINT/LINE FEED/CLEAR. CONNECT it to a file, a socket, or a real printer queue

Units: `prn` (serial, CONNECT)

Board properties

Key	Kind	Default	Legal	Meaning
`port`	int	`0x2`	`0x0` .. `0xFE`	Base address -- MUST BE EVEN. Control/status at BASE, data at BASE+1
`connect`	string	`null`	text	The endpoint on the other end of the line (CONNECT sets this)

`pio`

MITS 88-PIO: 8-bit parallel port, units 'out'/'in'. Two ports at BASE+0..1 (default 04). CONNECT a printer, a keyboard, a socket

Units: `out` (serial, CONNECT), `in` (serial, CONNECT)

Board properties

Key	Kind	Default	Legal	Meaning
`port`	int	`0x4`	`0x0` .. `0xFE`	Base address -- MUST BE EVEN. Control/status at BASE, data at BASE+1

Unit `out` — `[board.unit.out]`

Key	Kind	Default	Legal	Meaning
`connect`	string	`null`	text	The endpoint on the other end of this line (CONNECT sets this)

Unit `in` — `[board.unit.in]`

Key	Kind	Default	Legal	Meaning
`connect`	string	`null`	text	The endpoint on the other end of this line (CONNECT sets this)

Video

`dazzler`

Cromemco Dazzler: color graphics from a framebuffer in main RAM. Two ports at BASE+0..1 (default 0E): control/status and format. 32x32 to 128x128, 16 colors/greys. Needs a Display

Board properties

Key	Kind	Default	Legal	Meaning
`port`	int	`0xE`	`0x0` .. `0xFE`	I/O base port -- control/status (BASE) and format (BASE+1). Even; default 0E
`width`	string	`auto`	text	Video window width in pixels: 'auto' (default) opens about half the screen wide, or a number like 1024. The height follows the board's own aspect, and the picture is a whole multiple of its pixels so it stays crisp
`video`	string	—	—	LIVE: whether the Dazzler is displaying -- OUT BASE D7 (on/off). Read-only **(read-only — not a key you may set)**
`resolution`	string	—	—	LIVE: picture size in elements, decoded from the format byte (D6 X4, D5 size): 32x32, 64x64 or 128x128. Read-only **(read-only — not a key you may set)**
`color`	string	—	—	LIVE: color vs black-and-white -- format D4. Read-only **(read-only — not a key you may set)**
`size`	string	—	—	LIVE: framebuffer footprint -- format D5: 512 bytes (one quadrant) or 2 KB (four quadrants). Read-only **(read-only — not a key you may set)**
`base`	int	—	—	LIVE: framebuffer start address in RAM -- OUT BASE D6-D0 << 9. Read-only **(read-only — not a key you may set)**

`vdb8024`

SD Systems VDB-8024: an 80x24 video terminal on one board -- the video console for an SBC-100/200 (the alternative to the 8251). Two I/O ports at BASE+0..1 (default 00): status/keyboard/display. Unit 'keyboard' (CONNECT). Optional keyboard-strobe interrupt strap (interrupt=vi0..vi7) for the SBC-200's CTC to vector -- what the SD video CBIOS needs; polled by default. Boots sdmonv21. Needs a Display

Units: `keyboard` (serial, CONNECT)

Board properties

Key	Kind	Default	Legal	Meaning
`port`	int	`0x0`	`0x0` .. `0xFE`	Low I/O port: status (IN base+0) / keyboard (IN base+1) / display (OUT base+1). The real card is fixed at 00
`cursor`	enum	`blink`	`off` | `blink` | `steady`	Cursor at the current cell: off, blink, or steady (the board default is a blinking cursor)
`video`	enum	`normal`	`normal` | `reverse`	Screen video polarity: normal (light on dark) or reverse
`interrupt`	enum	`none`	`none` | `int` | `vi0` | `vi1` | `vi2` | `vi3` | `vi4` | `vi5` | `vi6` | `vi7`	Keyboard-strobe interrupt strap: none = polled (default), or the S-100 VI line the keyboard raises while a byte waits (the SBC-200 CTC vectors it -- video CBIOS straps vi2) *(interrupt strap)*
`width`	string	`auto`	text	Video window width in pixels: 'auto' (default) opens about half the screen wide, or a number like 1024. The height follows the board's own aspect, and the picture is a whole multiple of its pixels so it stays crisp

Unit `keyboard` — `[board.unit.keyboard]`

Key	Kind	Default	Legal	Meaning
`connect`	string	`null`	text	The endpoint on the other end of the keyboard line (CONNECT sets this)

`vdm1`

Processor Technology VDM-1: memory-mapped 16x64 video, screen RAM at BASE (default CC00), scroll/status port (default CC). Needs a Display

Board properties

Key	Kind	Default	Legal	Meaning
`base`	int	`0xCC00`	`0x0` .. `0xFC00`	Screen-RAM base address -- 1 KB-aligned (16x64 = 1024 bytes)
`port`	int	`0xCC`	`0x0` .. `0xFC`	I/O port -- scroll (OUT) / status (IN). Low two bits are zero
`video`	enum	`normal`	`normal` | `reverse`	Video polarity (SW1/SW2): normal (light on dark) or reverse
`cursor`	enum	`blink`	`off` | `blink` | `steady`	Cursor for a byte with bit 7 set (SW3/SW4): off, blink, or steady
`width`	string	`auto`	text	Video window width in pixels: 'auto' (default) opens about half the screen wide, or a number like 1024. The height follows the board's own aspect, and the picture is a whole multiple of its pixels so it stays crisp

Systems

`sol`

Processor Technology Sol-PC I/O: serial, keyboard, parallel, CUTS tape as one board. Seven ports F8..FE. Units serial/printer/keyboard (CONNECT) and tape1/tape2 (MOUNT). Brings the WIND/REWIND/EXTRACT verbs and a tape counter

Units: `serial` (serial, CONNECT), `printer` (serial, CONNECT), `keyboard` (serial, CONNECT), `tape1` (tape, MOUNT), `tape2` (tape, MOUNT)

Board properties

Key	Kind	Default	Legal	Meaning
`base`	int	`0xF8`	`0x0` .. `0xF8`	Base I/O port (decodes BASE+0..6). Fixed at F8 on a real Sol-PC

Unit `serial` — `[board.unit.serial]`

Key	Kind	Default	Legal	Meaning
`connect`	string	`null`	text	The endpoint on the other end of this line (CONNECT sets this)
`baud`	int	`9600`	`1` .. `1000000`	Serial line speed (the strap on the Sol-PC's serial UART)
`data_bits`	int	`8`	`5` .. `8`	Serial word length: 8, 7, or 6 (the Sol-PC DIP)

Unit `printer` — `[board.unit.printer]`

Key	Kind	Default	Legal	Meaning
`connect`	string	`null`	text	The endpoint on the other end of this line (CONNECT sets this)

Unit `keyboard` — `[board.unit.keyboard]`

Key	Kind	Default	Legal	Meaning
`connect`	string	`null`	text	The endpoint on the other end of this line (CONNECT sets this)

Unit `tape1` — `[board.unit.tape1]`

Key	Kind	Default	Legal	Meaning
`mode`	enum	`play`	`play` | `record`	Which way the bytes go: play loads from the file, record saves to it
`format`	enum	`auto`	`auto` | `raw` | `cuts1200` | `kcs300`	How to read the mounted file: auto | raw | cuts1200 | kcs300
`leader`	int	`3`	`0` .. `120`	Seconds of idle tone before recorded data, when writing audio
`trailer`	int	`2`	`0` .. `120`	Seconds of idle tone after recorded data, when writing audio
`waveform`	enum	`square`	`square` | `sine`	Carrier shape when writing audio: square (like real hardware) | sine
`level`	int	`36`	`1` .. `100`	Recording level as a percent of full scale, when writing audio
`rc`	int	`4000`	`1000` .. `20000`	Edge-rounding low-pass corner in Hz, when writing CUTS audio
`rate`	enum	`full`	`full` | `real`	Playback speed: full (as fast as the guest reads) | real (wall-clock baud)
`detected`	string	—	—	What the cassette in this deck turned out to be (empty if none) **(read-only — not a key you may set)**
`position`	string	—	—	Where this deck's head is now: mm:ss / total (percent) -- read-only **(read-only — not a key you may set)**
`counter`	enum	`on`	`on` | `off`	Live tape counter on the console during a load: on | off
`stop`	string	`off`	text	Auto-stop playback at this time: off | end | mm:ss

Unit `tape2` — `[board.unit.tape2]`

Key	Kind	Default	Legal	Meaning
`mode`	enum	`play`	`play` | `record`	Which way the bytes go: play loads from the file, record saves to it
`format`	enum	`auto`	`auto` | `raw` | `cuts1200` | `kcs300`	How to read the mounted file: auto | raw | cuts1200 | kcs300
`leader`	int	`3`	`0` .. `120`	Seconds of idle tone before recorded data, when writing audio
`trailer`	int	`2`	`0` .. `120`	Seconds of idle tone after recorded data, when writing audio
`waveform`	enum	`square`	`square` | `sine`	Carrier shape when writing audio: square (like real hardware) | sine
`level`	int	`36`	`1` .. `100`	Recording level as a percent of full scale, when writing audio
`rc`	int	`4000`	`1000` .. `20000`	Edge-rounding low-pass corner in Hz, when writing CUTS audio
`rate`	enum	`full`	`full` | `real`	Playback speed: full (as fast as the guest reads) | real (wall-clock baud)
`detected`	string	—	—	What the cassette in this deck turned out to be (empty if none) **(read-only — not a key you may set)**
`position`	string	—	—	Where this deck's head is now: mm:ss / total (percent) -- read-only **(read-only — not a key you may set)**
`counter`	enum	`on`	`on` | `off`	Live tape counter on the console during a load: on | off
`stop`	string	`off`	text	Auto-stop playback at this time: off | end | mm:ss

PROM programmer

`pb1`

SSM PB1: 2708/2716 EPROM programmer + on-board EPROM board. A 4K programming-socket window (default D000, sockets U22=2708/U23=2716) and one control port (default 10): OUT arms the board and picks the chip (D0=2708, D1=2716), then a window write burns a byte and a window read disarms it. Save the burn to a host hex file with `SAVE file window`. Optional read-only on-board EPROM area via [[board.prom]] (at + mount). The board ships no firmware -- run any 2708/2716 burner; SSM's own from the PB1 manual is in examples/pb1

Units: `u22` (rom, MOUNT), `u23` (rom, MOUNT)

`[[board.socket]]` — a list you may add

Key	Kind	Legal	Meaning
`chip`	enum	`2708` | `2716`	Which programming socket: 2708 (U22, 1K) or 2716 (U23, 2K)
`mount`	string	text	A source or erased chip image to put in the socket. Relative to THIS FILE.

`[[board.prom]]` — a list you may add

Key	Kind	Legal	Meaning
`at`	int	`0x8000` .. `0xFFFF`	Where the on-board EPROM sits. An address at or above 8000.
`mount`	string	text	The firmware image: a file (relative to THIS FILE), or builtin:.

Board properties

Key	Kind	Default	Legal	Meaning
`port`	int	`0x10`	`0x0` .. `0xF0`	Control port: an OUT here arms the board and picks 2708 (D0) or 2716 (D1). Only A4-A7 decode, so it must be an x0H address (00, 10, .. F0)
`window`	int	`0xD000`	`0x0` .. `0xF000`	The 4K programming-socket window, on a 4K boundary (0000, 1000, .. F000). The guest writes bytes here to burn, and reads here to disarm

Other

`fp`

Altair front panel: the SENSE switches a guest reads at IN 0FFH -- a configured byte (SET fp0 sense= or TOML), not toggled here. No OUT

Board properties

Key	Kind	Default	Legal	Meaning
`sense`	int	`0x0`	`0x0` .. `0xFF`	The SENSE switches, SA8..SA15 -- what IN 0FFH reads

`hostbridge`

Host Bridge: guest <-> host file transfer, sandboxed. OUR OWN BOARD, not a period one. Two ports at BASE+0..1. R.COM/W.COM/HDIR.COM

Board properties

Key	Kind	Default	Legal	Meaning
`port`	int	`0xB0`	`0x0` .. `0xFE`	Base port. Two ports: BASE+0 command/status, BASE+1 data
`hostdir`	string		text	The sandbox root. Guest names resolve here and CANNOT escape it. Empty = the shell's working directory
`hostdir_root`	string	—	—	LIVE: the sandbox root as RESOLVED -- the actual directory the guest is fenced into. Read-only; `hostdir` is what was written. **(read-only — not a key you may set)**
`readonly`	bool	`false`	`on` | `off`	Refuse OPEN_WRITE and DELETE -- the guest may read the host, not change it

`ss1`

CompuPro System Support 1: multifunction S-100 board. Dual 8259A interrupt controllers in a master/slave cascade (master/slave at base+0..+3; master watches VI0-6 and drives pin 73, slave takes the timer OUTs and the UART's Rx/TxRDY), an 8253 interval timer (three counters + control at base+4..+7, 2 MHz clock), the OKI MSM5832 battery-backed real-time clock/calendar (command/data at base+10/+11) and a 2651 UART serial channel (base+12..+15); base default 50H. The 9511/9512 math socket is unpopulated

Units: `serial` (serial, CONNECT)

Board properties

Key	Kind	Default	Legal	Meaning
`base`	int	`0x50`	`0x0` .. `0xF0`	Base port of the 16-port I/O block. CompuPro standard is 50H
`clock`	string	—	—	LIVE: the date/time the MSM5832 is showing, and its offset from host time **(read-only — not a key you may set)**
`timer`	string	—	—	LIVE: the three 8253 counters -- mode, current count and OUT level **(read-only — not a key you may set)**
`pic`	string	—	—	LIVE: the master/slave 8259A -- IRR/ISR/IMR and the master's INT pin **(read-only — not a key you may set)**

Unit `serial` — `[board.unit.serial]`

Key	Kind	Default	Legal	Meaning
`baud`	int	`9600`	`50` .. `19200`	Line rate. The 2651 generates it from Mode Register 2, so the guest's MR2 write overwrites this; it seeds the line before then (min 50)
`interrupt`	enum	`none`	`none` | `int` | `vi0` | `vi1` | `vi2` | `vi3` | `vi4` | `vi5` | `vi6` | `vi7`	Where this port's IRQ is jumpered: none | int | vi0..vi7. The chip raises it on RxRDY; the standard board routes it through the 8259A instead *(interrupt strap)*
`connect`	string	`null`	text	The endpoint on the other end of the line (CONNECT sets this)

`virtc`

MITS 88-VI/RTC: vectored interrupts (VI0-VI7 -> RST n) and a real-time clock. One port at FE

Board properties

Key	Kind	Default	Legal	Meaning
`port`	int	`0xFE`	`0x0` .. `0xFF`	Control port. 0xFE (376 octal) on the real card -- write only
`rtc_source`	enum	`line`	`line` | `clock`	RTC clock source jumper: the 60 Hz line, or 10 kHz off the 2 MHz clock
`rtc_divide`	enum	`1`	`1` | `10` | `100` | `1000`	RTC divider jumper: source frequency / 1, 10, 100 or 1000
`rtc_interrupt`	enum	`none`	`none` | `vi0` | `vi1` | `vi2` | `vi3` | `vi4` | `vi5` | `vi6` | `vi7`	Where the RTC's interrupt ("RI") is jumpered: none | vi0..vi7. Leave it `none` to run the PS2 package *(interrupt strap)*
`vi_enabled`	bool	—	—	LIVE: is the 88-VI structure enabled? (control bit 7; POC clears it) **(read-only — not a key you may set)**
`level_live`	bool	—	—	LIVE: is the current-level comparison in circuit? (control bit 3) **(read-only — not a key you may set)**
`rtc_pending`	bool	—	—	LIVE: has the RTC's interrupt flip-flop set? (cleared by control bit 4) **(read-only — not a key you may set)**
`current_level`	int	—	—	LIVE: the current interrupt level (control bits 0-2, ones-complement on the wire). Nothing at this level or below may interrupt while level_live **(read-only — not a key you may set)**

The built-in machines

A built-in is a machine file that lives **inside the binary** — the same format you would write yourself, with nothing special about it. Name one and you get it in any directory on earth:

```
$ altairsim basic4k
```

`altairsim --list` prints this table, and `altairsim -x 'SHOW MACHINE' <name>` shows what is actually in one.

Machine	What it is
`8085`	A minimal 8085 machine: an `8085` CPU, 64K of RAM, and a 2SIO console.
`acuter`	ACUTER at F000 -- CUTER on a plain Altair, with a terminal instead of a VDM.
`altair680`	The Altair 680b -- MITS's second machine, and a different animal from the 8800.
`altmon`	An Altair with a monitor in ROM and a terminal on it.
`amon`	AMON 3.1 in a 4K EPROM at F000 -- Martin Eberhard's full-featured Altair monitor.
`bankmem`	A bank-switched RAM machine: a Z80, a console, and a Vector Graphic 64K bankmem.
`basic4k`	The machine Altair 4K BASIC was sold to run on: an 88-SIO Teletype, a cassette in the ACR.
`basic8k`	The machine Altair 8K BASIC was sold to run on: an 88-2SIO terminal, a cassette in the ACR.
`cdbl`	The `default` machine with the Combo Disk Boot Loader in the PROM socket.
`compupro`	A stock Altair with a CompuPro System Support 1 board for its clock/calendar.
`cuter`	CUTER 1.3 driving a Processor Technology VDM-1 -- the real Sol/CUTS monitor.
`dazzler`	A Cromemco Dazzler in an Altair -- the bench for the S-100's first color graphics card.
`default`	The machine you get when you name none: 56K, and the DBL boot PROM at FF00.
`dualide`	S100Computers "IDE-AB CF" machine -- a V2 Z80 CPU board booting CP/M 3 off a CompactFlash card.
`dualidesd`	S100Computers "IDE-AB CF+ESP32" machine -- both boards, CP/M 3 on CF drives A:/B: and SD drives C:/D:.
`dualsd`	S100Computers "Dual SD" machine -- a V2 Z80 CPU board booting CP/M 3 off a microSD card.
`icom`	iCOM FD3712 8" floppy machine -- boots CP/M 2.2 off a single-density iCOM disk.
`lineprinter-lpc`	The `default` machine with an 88-LPC line printer at port 02; CONNECT `lpt0:prn` to a file or the console.
`lineprinter`	The `default` machine with an 88-C700 line printer at port 02; CONNECT `lpt0:prn` to a file or the console.
`minidisk`	The Altair Minidisk: an 88-MDS at 08 and the MDBL boot PROM. You supply the 5.25" disk.
`original`	The Altair as it actually left Albuquerque.
`parallel`	The `default` machine with two MITS parallel boards: an 88-PIO and an 88-4PIO.
`ps2`	The machine MITS Programming System II ran on: basic8k's cards, but not basic8k's bootstrap.
`ps2int`	MITS Programming System II, WITH INTERRUPTS. `ps2` with A9 down and an 88-VI/RTC in it.
`rombasic`	MITS Extended ROM BASIC 16K -- Extended BASIC that runs directly out of ROM.
`sbc200`	SD Systems SBC-200 -- a 4 MHz Z80 single-board computer running the MSMONR21 monitor.

`sbc200v`	The SD Systems SBC-200 with a VDB-8024 VIDEO console: SDMONV21 on an 80x24 screen.
`sol20`	The Processor Technology Sol-20 -- an integrated 8080 machine, running SOLOS.
`tarbell`	Tarbell #1011 single-density floppy machine -- boots CP/M 2.2 the moment a disk is in it.
`tarbelldd`	Tarbell #2022 double-density floppy machine -- boots CP/M 2.2 the moment a disk is in it.
`turnkey`	The MITS 8800bt -- an Altair with a Turnkey Module where the front panel used to be.
`vdm1`	A Processor Technology VDM-1 in an Altair, and a demo that draws on it.
`z80`	A minimal Z80 machine: a `z80` CPU, 64K of RAM, and a 2SIO console.